Cristina-Sorina Stângaciu
Eugenia-Ana Capota

REAL TIME SYSTEMS PROGRAMMING

From Theory to Practice

EDITURA POLITEHNICA
TIMIŞOARA - 2025

Dedicated to our students.

CONTENTS

PREFACE

Real-time systems are a special category of computational systems, for which the correctness criterion does not involve only that the system provides a correct output, but also that this output is provided in specific time frames.

While this field has known a high scientific development, the theoretical knowledge is poorly reflected in the practical industrial field. For example, there are tens of theoretical real-time scheduling algorithms (over 20 implemented in SimSo simulator), but only very few are used in the industry. Such gaps between theory and industry are not new in this field: e.g. the Earliest Deadline First (EDF) algorithm was developed in 1973 but introduced in Linux only in 2013.

In this context, bridging the gap between theory and practice is important.

This book offers an introduction in the field of real-time system programming and it is dedicated to Computers and Information Technology students and enthusiasts which posses basic knowledge in C programming.

Its purpose is to introduce to the students the main concepts of real-time systems and real-time systems programming, and to bridge the gap between theory and practice, by providing a series of step-by-step lessons with explanations regarding the theoretical principals and how they can be implemented in practice.

Structure of book

This book is structured into five chapters. The first chapter contains the theoretical foundations of real-time systems, the second offers an introduction to task scheduling using a well-known simulation environment. The next chapters focus on embedded programming and practical real-time pro-

gramming on a real-time operating system including system analysis and benchmarking. The last chapter proposes a series of projects.

All practical lessons contain the same structure which includes the objectives, the theoretical foundations on which the applications are build, application examples, ending with bibliographic references and proposed exercises for a better understanding of the theoretical concepts and for developing practical real-time programming skills.

Timişoara, Iulie 2025 Authors

1. REAL-TIME SYSTEMS

1.1. Introduction into real-time system

In order for a system to be real-time, it must produce a physical effect or deliver a response within a specific time frame. For a computer, this refers to the capacity to perform the necessary computation in a given time. If multiple events occur at the same time, the computer will have to schedule the computations so that each event is treated within its required time bounds. Is this context, the events that occur in a real-time system are called tasks.

Each task is characterized by a set of timing properties, which should be considered when scheduling in a real-time system [1, 2, 3]:

- Arrival (or release) time - the earliest time at which a task can be processed.
- Period - the arrival interval between two consecutive instances of the same task.
- Deadline - the time by which execution of the task should complete, after the task is released.
- Worst case execution time (WCET) - the maximum time it takes to complete the task after its release.
- Priority - the level of urgency (or importance) of the task.

The parameters of a task instance (job) are presented in Figure 1.1.

Some examples of real-time systems include [4, 5]: air traffic control, autonomous driving systems, process control systems, robotics, road traffic control, etc.

The objective of a real-time system is to handle several demands concurrently and efficiently, within a limited time frame, where each demand has a different time available for response. These demands can have different levels of importance (e.g., in an autonomous driving system, safety-related tasks are more important than the air conditioning system). The real-time system must allocate available resources so that all demands are met before

their respective deadlines. This is done by a scheduler which implements a scheduling policy.

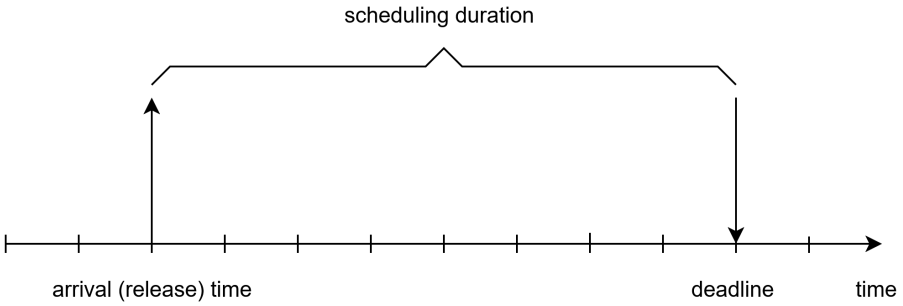

Figure 1.1. Task instance parameters.

1.2. Real-time scheduling

Scheduling in real-time systems is the process of planning and controlling the execution order of tasks so that each task meets its required timing constraints [1]. In a real-time environment, multiple tasks compete for limited CPU time, and the scheduler decides which task runs at each moment to satisfy all deadline requirements. In practice, scheduling works hand-in-hand with feasibility analysis: before runtime, system designers analyze the task set to verify that a valid schedule exists such that all tasks can indeed meet their deadlines.

Feasibility, in the context of real-time systems, refers to whether a given set of tasks can be scheduled to meet all their deadlines [1]. A task set is considered feasible if there exists some scheduling order in which every task finishes before its deadline (i.e. a schedule where all timing constraints are satisfied). Verifying feasibility, often through feasibility analysis [1, 2, 3] (also called schedulability analysis), is a critical step in real-time system design. It involves checking the task parameters (such as execution times and periods) against the available processing capacity to ensure that no deadlines will be missed. This analysis provides a guarantee that the system can handle its workload under worst-case conditions.

When scheduling a real-time system, the following should be taken into account [3]:

- Ensuring all timing constraints are met.

- Minimizing context switch overhead due to preemption.
- Avoiding concurrent access to shared resources.
- Reducing communication overhead in distributed real-time systems.

Real-time systems are those where correctness depends not only on the output, but also on when it is produced. Failing to meet timing constraints is considered a system failure. To avoid this, the system must be predictable, meaning task completion times can be determined with certainty.

Let us consider a system with a set of tasks, $\{\tau_1, \tau_2, ..., \tau_n\}$, where the worst case execution time of each task is C_i. The scheduling problem involves determining an execution plan that ensures all task instances (jobs) complete before their deadlines. We should take into account the specific characteristics of the system and the type of tasks it runs. These characteristics include [1, 2, 3]:

Soft/Firm/Hard real-time tasks

1. **Soft real-time tasks** introduce a penalty function $P(\tau_i)$. The penalty function is greater if the task execution exceeds its corresponding deadline (D_i). If $C_i \leq D_i$, the penalty function $P(\tau_i)$ is zero, otherwise $P(\tau_i) > 0$. Missing a soft deadline is not catastrophic; it may degrade the quality of service but not the functional correctness of the system. Soft real-time tasks are common in multimedia and consumer applications (e.g., video playback, data acquisition for logging, etc.) where occasional delays are tolerable. These tasks continue to have utility even after a deadline miss (unlike firm tasks), so the system will try to complete them even if late.

2. **Firm real-time tasks** should ideally meet their deadlines, but the system can tolerate occasional misses by simply discarding late results (e.g., a video frame arriving too late in a live stream might be dropped). This means that the usefulness of a firm real-time task output becomes zero after its deadline, even though sporadic misses do not damage the system. Many papers [6, 7, 8] formalize this concept using (m, k)-firm constraints, which require that at least m out of any k consecutive task instances meet their deadlines.

3. **Hard real-time tasks** must finish executing before every deadline D_i, ($C_i \leq D_i$). A single missed deadline is considered a system failure and may lead to catastrophic consequences. Hard real-time tasks arise in safety-critical functions (e.g., engine control, pacemakers, etc.) where timing guarantees are absolute.

Soft real-time tasks may occasionally miss deadlines without compro-

mising the overall functionality of the system. However, such delays often bring penalties. Firm real-time tasks should meet most deadlines but can drop the value of late results. Hard real-time tasks must always meet their deadlines, as any missed deadline can result in unacceptable or even catastrophic consequences for the system. A real-time task set $\{\tau_1, \tau_2, \ldots, \tau_n\}$ may be a combination of soft, firm and hard real-time tasks.

Periodic/Aperiodic/Sporadic real-time tasks

1. **Periodic real-time tasks** repeat at regular time intervals (once per period, T). Each instance (job) of a periodic task is released exactly every T time units. Periodic tasks model activities like sampling sensors or actuator updates that occur on a fixed schedule. Because of their regularity, periodic tasks are often used for hard real-time activities with stringent deadlines equal to the period.

2. **Aperiodic real-time tasks** do not have a fixed or predictable release and are not bound by a minimum inter-arrival time. They can execute in response to external or unpredictable events (e.g., a user input or an asynchronous message). The only time constraint is usually a deadline, D by which these tasks must finish. In many designs, aperiodic tasks are treated in a soft real-time manner, which means they may not have strict deadlines, or missing some deadlines is acceptable.

3. **Sporadic real-time tasks** are a special case of aperiodic tasks with a minimum inter-arrival time, which means there is a minimum interval of time between two successive instances of the same task. The only time constraint is usually a deadline, D. A sporadic task can occur at unpredictable times like an aperiodic task, but it is constrained, so that successive releases have at least a certain gap (e.g., "no more than one release every 50 ms"). This minimum inter-arrival time serves as a period for worst-case analysis. Sporadic tasks are often treated as hard real-time tasks (like periodic tasks) in schedulability analysis, because the known minimum inter-arrival time allows deterministic worst-case load assumptions. In practice, most real-time scheduling theory today uses the sporadic task model.

Periodic and sporadic tasks are well-suited for off-line feasibility analysis because they have known minimum inter-arrival times, allowing the system to assume a maximum request rate. In contrast, aperiodic tasks lack such guarantees, making them less predictable and harder to analyze. To manage them, real-time systems often use specialized mechanisms (such as polling [9] or deferrable servers [10]) to handle aperiodic requests without

disrupting the timely execution of periodic tasks. For instance, a system might run hard real-time periodic control loops while servicing operator commands (aperiodic) using a background server. In summary, periodic and sporadic tasks follow predictable patterns that support formal analysis, whereas aperiodic tasks require dynamic scheduling to achieve reasonable responsiveness.

Preemptive/Non-preemptive real-time tasks

Real-time tasks can either be scheduled preemptively or non-preemptively, which refers to whether a running task can be interrupted to start another task. This is a property of the scheduling approach, but it effectively creates two categories of task execution:

1. **Preemptive real-time task** execution can be interrupted if another task of higher priority is released. Preemption improves responsiveness to high-priority or urgent tasks. Most hard real-time systems allow preemption to ensure critical deadlines are met. For instance, a low-priority task computing a log can be preempted the instant a high-priority control task is released. Preemptive scheduling can minimize response time to events, but it requires considerable computational resources for scheduling.

2. **Non-preemptive real-time tasks** should complete executing, once started, without any interruption. While this can simplify the system, it can increase worst-case response times. A high-priority task may be forced to wait for a lower priority task to finish. Non-preemptive scheduling is generally less efficient in meeting deadlines, except in cases where tasks are very short or carefully ordered. In fact, it is known that in most scenarios, allowing preemption leads to better schedulability.

As it is necessary for ensuring hard deadlines, most real-time operating systems support preemptive priority-based scheduling by default. However, limited non-preemptive segments are sometimes used (e.g., when tasks disable preemption briefly to protect a critical section, or in certain communication scheduling).

Fixed/Dynamic priority real-time tasks

1. **Fixed-priority real-time tasks**. In fixed-priority scheduling, each task is assigned a priority that remains constant. An example is Rate Monotonic (RM) scheduling [11], which assigns higher priority to tasks with shorter periods.

2. **Dynamic priority real-time tasks**. For dynamic priority sched-

uling priorities are calculated during the execution of the system. For instance, Earliest Deadline First (EDF) [11] scheduling always gives highest priority to the job with the nearest deadline.

This leads to different performance characteristics: EDF can theoretically achieve 100% processor utilization, whereas RM guarantees schedulability only up to about 69% utilization in the worst case [11, 12]. Other algorithms that follow these models are: Deadline Monotonic (DM) [13] (a fixed-priority scheme for deadlines shorter than periods) and Least Slack Time First (LST) [14] (a dynamic scheme using remaining slack).

Single processor/Multiprocessor/Distributed real-time systems

The number of processors (CPUs) plays an important role when scheduling tasks in real-time systems. For multiprocessor and distributed systems, tasks have to first be allocated to processors and then scheduled on each individual processors. The scheduling algorithm should prevent simultaneous access to shared resources and devices.

1. **Single processor real-time systems** (or uniprocessor systems) contain a single CPU executing all tasks. Despite the hardware simplicity, scheduling such systems requires careful design to ensure all tasks meet their deadlines.

Single processor real-time systems avoid the complexities of task migration or communication delays, focusing instead on CPU scheduling and local resource management.

In this case, challenges include priority inversion when tasks share resources (if a high-priority task is indirectly superseded by a lower priority task) and processor overload if the cumulative CPU demand occasionally exceeds the available 100% (in which case deadlines are missed).

2. **Multiprocessor real-time systems** (or multicore systems) allow tasks to execute in parallel on different CPUs, offering greater computing capacity. However, this comes at the cost of significantly increased scheduling complexity.

There are two fundamental approaches to scheduling on m processors:

- In **partitioned scheduling**, each task is statically assigned to one processor. Therefore, each CPU schedules its tasks independently (using single processor scheduling algorithms, for example RM or EDF). Partitioning simplifies analysis (reducing the problem to multiple single-CPU problems) but the partitioning itself is an NP-hard

bin-packing problem, often requiring heuristics [15].

- In **global scheduling** tasks are pooled and a global scheduler can dispatch any task on any available processor. Tasks can migrate between processors or even be preempted and resumed on different processors. Global scheduling is more flexible and can, in theory, achieve higher utilization, but introduces overhead from task migrations and a more complex schedulability analysis.

There are also hybrid approaches (e.g., semi-partitioned [16] or cluster-based scheduling [17]) that try to balance the two.

3. **Distributed real-time systems** consist of multiple processors connected by a communication network. Tasks may be distributed across nodes, and end-to-end real-time functionality is achieved through a chain of task executions and message transmissions. Distributed real-time systems are common in automotive networks (multiple ECUs communicating via CAN/FlexRay), avionics (ARINC bus systems), or IoT sensor networks [18]. They introduce two main challenges on top of local scheduling: communication scheduling and global time coordination.

In a distributed system, tasks on different nodes often communicate via network messages that must also be scheduled. For instance, a sensor node might send a message with latest readings to an actuator node which runs a control task. Networks used in real-time systems have their own medium access control protocols that determine when messages are sent [19]. Ensuring message latencies are bounded is as important as scheduling CPU tasks. A common strategy is to assign priorities to messages (e.g., fixed-priority in CAN) and analyze worst-case queuing delays on the network similar to task scheduling. In time-triggered protocols (TTP [20], TTEthernet [21]), message transmissions are scheduled at predetermined global time slots, providing very predictable communication at the cost of flexibility [19].

In essence, distributed real-time systems combine the challenges of local scheduling on each node with the challenges of real-time networking. Each node might use fixed-priority or EDF scheduling locally, and the network might use fixed-priority message scheduling or a static schedule. The overall system timing correctness depends on both.

Single-processor systems demand optimal CPU scheduling, multiprocessor systems add complexity of parallel scheduling and shared resources, and distributed systems further add network timing and partitioning of functionality. In all cases, the goal is to guarantee that every critical task finishes by its deadline, but the methods to ensure this vary with the platform.

1.3. Task models in real-time systems

To analyze and guarantee timing properties, real-time tasks are described using formal task models. A task model encapsulates the key parameters of tasks (like execution time, period, deadline, etc.) and any behavioral rules (such as dependencies or modes).

1.3.1. The Liu and Layland periodic task model

The most widely used task model was introduced by C. L. Liu and J. Layland in 1973 [22]. In this model a task (τ_i) is represented by an ordered pair of parameters, $\tau_i = (C_i, T_i)$, where C_i is the worst case execution time (WCET) and T_i is the period. The first job of a task can arrive at any time instant, however the arrival times of two consecutive jobs must be at least T_i time units apart. The Liu and Layland task system consists of a finite number of independent tasks, executing on the same platform. Scheduling is preemptive, and context switch overheads are ignored. Under these assumptions, Liu and Layland proved fundamental results such as the EDF and RM optimality, and derived a utilization-based schedulability criterion for RM scheduling. For example, they showed that a set of n periodic tasks scheduled by RM is guaranteed to meet all deadlines if $\sum_{i=1}^{n} C_i/T_i \leq n(2^{1/n} - 1)$ [23]. As n grows large, this bound approaches $\ln(2) \approx 0.693$ (about 69% CPU utilization), whereas EDF can go up to 100% utilization.

Despite its popularity, the model has some limitations. It is very restrictive in the types of task systems it can represent. Key limitations include: no variation in inter-arrival times (tasks are strictly periodic), deadlines equal periods (cannot model tasks with deadlines shorter or longer than their period), no inter-task interactions or release jitter, all tasks are recurrent (they repeat forever, no transient tasks). Real-world systems often violate these assumptions (e.g., a task might have a sporadic trigger or a deadline less than its period).

The model's rigidity prevents handling mode changes or conditional behavior. As Stigge et al. [24] noted, the Liu and Layland model "does very well on the analysis efficiency side but is very restrictive on the type of tasks allowed".

Many industrial standards (e.g., automotive OSEK, ARINC 427) assume periodic or sporadic tasks scheduled with fixed priorities, essentially using Liu-Layland model assumptions for timing analysis [25]. The model also provides a baseline against which more complex task models are compared.

1.3.2. Extensions and advantaced task models

The sporadic task model, introduced by Mok in 1983 [26] is an extension of the periodic model. A sporadic task is characterized by (C_i, T_i, D_i) and can release jobs at irregular intervals, as long as successive jobs are at least T_i units apart. Here T_i is interpreted as a minimum inter-arrival time, not a fixed period. This extension allows modeling of event-driven tasks (e.g., interrupts). Sporadic tasks are widely used in modern real-time analysis. Most schedulability tests (like Response-Time Analysis for fixed-priority [27]) assume sporadic task arrivals for worst-case (since sporadic is more general than periodic).

Real-time tasks can have a relative deadline D different from period T (if the task is periodic/sporadic). Constrained-deadline models allow $D \leq T$, and arbitrary-deadline models allow D to be any value (even greater than T). These are not separate "models", but extensions to the parameters of periodic/sporadic tasks. Analysis techniques such as utilization bounds or exact schedulability tests often become more complex for arbitrary deadlines, but they reflect reality in systems where, for example, a task may have to complete within 50 ms but only runs every 100 ms ($D = 50 < T = 100$), or conversely a task might accumulate results over several periods ($D > T$).

The multiframe task model, proposed by Mok et al. [28], allows the execution time of a task to vary in a cyclic pattern from instance to instance. Instead of a single C_i, a multiframe task has a sequence of execution times $C_i^0, C_i^1, ..., C_i^{(N-1)}$ that repeats every N invocations. This can model tasks whose computational demand fluctuates regularly (e.g., a control task that does a heavy computation every 10th cycle). The generalized multiframe model further allows each frame to have its own deadline and period offset.

Stigge et al. (2011) [24] proposed the digraph real-time task model, which is an extension of the directed acyclic graph task model introduced by Baruah in 1998 [29]. In this task model the sequence of job inter-arrival times for a task is represented as an arbitrary directed graph rather than a fixed period or simple cycle. Each node in the graph represents a type of job (with a given execution time and deadline), and edges carry minimum separation times. The task can follow different paths in the graph, capturing alternating behaviors, branches, or modes. Essentially, the digraph model generalizes multiframe and recurring task models and can even represent unbounded loops by cycles in the graph. Mode-switching tasks [30] (where tasks change their periods or execution profile on-the-fly when the system mode changes) can be modeled as a digraph or as multiple task sets with a mode-change

protocol controlling them. These advanced models acknowledge that real systems often are not strictly periodic forever, they go through phases (e.g., startup vs. normal operation) or have conditional execution paths.

A significant body of work in the past decade has focused on mixed-criticality systems (MCSs), introduced by Vestal (2007) [31]. In a MCS, tasks have different levels of criticality (e.g., some are safety-critical, others are low-criticality). Vestal observed that higher-criticality tasks typically require more conservative timing assumptions (larger WCETs) due to stricter validation processes. The mixed-criticality model assigns each task multiple WCET estimates: $C_i(LO)$ for normal operation and $C_i(HI)$ (a larger value) for the high-criticality scenario. The system is analyzed in multiple modes or levels. Initially, all tasks are assumed to execute within their low-criticality bounds. If a high-criticality task overruns its LO bound, the system switches to HI-criticality mode: low-criticality tasks may be dropped and high-criticality tasks execute within their HI bound, $C_i(HI)$. Formally, a MCS task might be defined as $(C_i(LO), C_i(HI), T_i, D_i, L_i)$ where L_i is the criticality level. Analysis of MCSs is quite complex. It involves verifying schedulability in multiple modes and designing scheduling algorithms (fixed-priority or EDF variants) that know how to drop or handle low-criticality tasks on a mode switch. Mixed-criticality models have been influential in domains like avionics, where multiple functions of different criticality share hardware. Research surveys by Burns and Davis (2022) [32] detail many variations and improvements on Vestal's basic model.

Figure 1.2 illustrates the evolution of task models in real-time scheduling. The arrows represent generalizations or simplifications of certain task models.

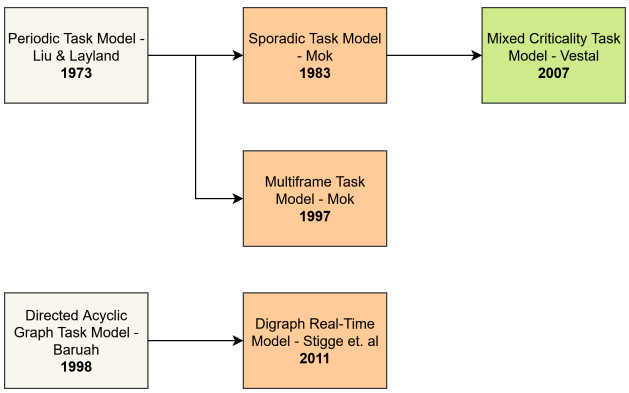

Figure 1.2. Task models in real-time scheduling.

1.4. Scheduling algorithms in real-time systems

Based on the moments when the scheduler decides which task to launch for execution, known as scheduling points, scheduling algorithms can be classified into clock-driven or event-driven.

One of the most used clock driven algorithms are the table-driven ones [33, 34], relying on precomputed schedules stored in tables to determine the order and timing of task execution. These algorithms often involve timers to trigger tasks at specific intervals based on the table schedule. They are typically used when the system workload is predictable and fixed, allowing for efficient and deterministic task scheduling. By looking up task execution details in the table, they can minimize runtime overhead and ensure timely task completion. These algorithms are particularly useful in environments where deadlines must be met consistently, such as embedded systems or industrial control applications. However, their flexibility is limited, as they are less suited for dynamic or unpredictable workloads.

In event-driven scheduling [35, 36], on the other hand, task priorities are determined in response to external events, such as the arrival of new tasks or changes in system state. This approach is more adaptable, allowing systems to respond to unforeseen events or workloads. While event-driven algorithms offer more flexibility, they can introduce additional complexity and require more sophisticated runtime management. Event-driven algorithms for real-time scheduling can be divided into fixed-priority and dynamic priority. Figure 1.3 highlights the most popular real-time scheduling algorithms.

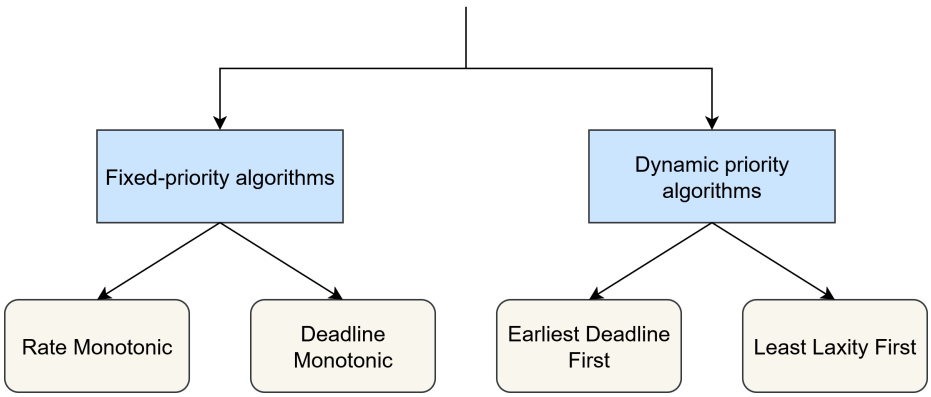

Figure 1.3. Real-time scheduling algorithms.

1.4.1. Fixed-priority algorithms

1.4.1.1. Rate Monotonic

The Rate Monotonic (RM) [11, 37] scheduling is a fixed-priority scheduling algorithm for periodic tasks. Under RM, tasks are assigned static priorities before runtime based on their rates. Tasks with short periods receive high priority, while tasks with long periods get lower priority.

Assumptions in the classic RM scheduling model include [11]:

- Periodic, independent tasks: Each task τ_i triggers jobs at a fixed period T_i, with deadline equal to its period.
- Preemptive scheduling: A running job can be temporary stopped with negligible context-switch overhead. This ensures that high-priority periodic tasks can meet tight deadlines.
- No resource contention: In the basic RM theory, tasks do not share critical resources (or if they do, they employ synchronization protocols to avoid priority inversion).
- Deterministic execution times: Each task worst-case execution time (WCET) C_i is known, and the CPU utilization of task τ_i is $U_i = = C_i/T_i$. The system total utilization is the sum $U = \sum_{i=1}^{n} \frac{C_i}{T_i}$, which is used for schedulability tests.

In their seminal 1973 paper [22], Liu and Layland proved that assigning priorities in Rate Monotonic order is an optimal strategy for fixed-priority scheduling. If any static priority assignment can schedule the task set feasibly, then the RM assignment will also yield a feasible schedule. In other words, no other fixed-priority policy can schedule a task set that RM fails to schedule (where deadlines are equal to periods).

Schedulability analysis and utilization bounds

A central question in real-time scheduling is determining whether a given task set will meet all deadlines under a particular algorithm. A sufficient schedulability condition for RM was derived, which states that a set of n periodic tasks will always be schedulable (all deadlines met) if the total utilization satisfies equation 1.1:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n \left(2^{1/n} - 1\right) \tag{1.1}$$

This inequality is often called the Liu-Layland bound. For example, for $n = 1$ tasks, it gives $U \leq 1$ (trivially, one task can use 100% CPU). For $n = 2$, it yields $U \leq 2(2^{1/2} - 1) \approx 0.828$ (about 82.8% CPU). As n grows larger, the bound approaches $\ln(2) \approx 0.693$ (69.3%). This means that in the worst-case task configuration, RM scheduling can guarantee all deadlines if total utilization is 69% or less.

It is important to emphasize that this utilization test is sufficient, but not necessary. Many task sets with utilizations higher than the bound are, in fact, schedulable under RM. For instance, if tasks have harmonic periods [38] (each period is an integer multiple of shorter periods), RM can schedule them up to 100% utilization with no missed deadlines. Empirical studies have shown that randomly chosen task sets can often be scheduled under RM at utilization levels around 88%, on average [39]. The Liu-Layland bound, however, provides a simple conservative test: if $U \leq n(2^{1/n} - 1)$, we are assured of schedulability without needing further analysis.

Rate Monotonic offers advantages like simplicity and predictability, and in practical systems the achievable utilization under RM is often quite high (well above 90% in many cases). Moreover, RM has the benefit of graceful degradation under overload: if tasks exceed the CPU capacity, the highest priority (typically most critical) tasks will still get CPU time at the expense of lower priority ones. This makes RM appropriate for safety-critical hard real-time systems where guaranteed deadlines for critical tasks are required even in overload scenarios.

Implementation in real-time systems

Implementing RM in a real-time operating system (RTOS) is straightforward because many RTOS kernels support fixed-priority preemptive scheduling as a basic service. The developer simply assigns each periodic task a priority according to its period and ensures that tasks are released periodically. The OS scheduler then preempts and dispatches tasks according to these fixed priorities. For example, the POSIX real-time scheduling class SCHED_FIFO [40] can be used to implement RM by appropriate priority assignment: one would configure the highest-rate task with the highest priority (often priority 1 in POSIX, or 255 in some systems), the next with the next-highest, and so on.

RM has been recommended as a standard practice in several industry standards and projects (e.g., the U.S. Navy Futurebus+ [41] system guidelines and NASA Space Station real-time software guidelines) due to its

robustness and analyzability.

However, applying RM in practice involves a few additional considerations that extend beyond the basic theoretical model [42]:

- Task period management: Periodic tasks in a RTOS can be implemented using hardware timers or a tick interrupt plus software delays. Common patterns include having each task suspend itself until its next release time (using sleep or timed wait calls), or a dedicated periodic scheduler task that releases jobs at fixed intervals. The timer resolution of the system can introduce release jitter (e.g., if the OS tick is 1 ms, period releases might jitter by that amount). Minimizing jitter is important for RM assumptions, since uneven spacing of jobs can create bursts that resemble critical instants. A high-resolution timer or hardware timer is often used for precise periodic releases.
- Context-switch and preemption overhead: In theoretical analysis, context switch times are assumed to be zero. In a real OS, each preemption incurs some overhead (saving registers, cache effects, etc.). If tasks have very short periods or execution times, frequent preemptions can degrade overall CPU available for useful work. This is sometimes handled by slightly reducing the utilization bound to include a margin for context-switch costs, or by limiting RM to task sets where WCETs are not too small relative to system tick or context-switch time.
- Priority inversion and resource sharing: A known limitation of fixed-priority scheduling is priority inversion, which occurs when a lower priority task holds a resource needed by a higher priority task, thereby blocking the high-priority task. In RM, priority inversion can violate schedulability assumptions (tasks are no longer independent) and lead to missed deadlines. To safely use RM with shared resources, real-time systems employ protocols like Priority Inheritance [43] or Priority Ceiling [44]. Priority Inheritance Protocol (PIP) temporarily elevates the priority of a low-priority task that holds a resource to the priority of the highest task waiting for that resource. This prevents a medium-priority task from preempting the resource-holding task and prolonging the inversion. Priority Ceiling Protocol (PCP), on the other hand, assigns a nominal "ceiling" priority to each resource (equal to the highest priority of any task that may use it) and forces tasks to acquire resources in a way that avoids lower priority tasks holding a lock on a resource that a higher priority task will

need. These protocols ensure that any priority inversion is bounded in duration. Modern RTOSs typically provide such mechanisms. In summary, careful synchronization design is necessary for RM to function predictably when tasks share resources.

- Deadline miss handling: In hard real-time systems, a missed deadline is a critical failure. With RM, if analysis shows a task set is not schedulable, system designers must adjust the task set or scheduling approach. Remedies can include reducing task execution times, increasing periods, or splitting/redistributing work to other CPUs (if a multiprocessor platform is available). In systems where firm or soft real-time requirements exist, a deadline miss by a low-priority task might be tolerable. RM will naturally let the lowest priority tasks miss deadlines first in an overload situation, preserving CPU for higher priority tasks.

In summary, implementing RM on an OS involves assigning static priorities according to task periods and then relying on the RTOS preemptive scheduler.

Rate Monotonic task scheduling example

Let us consider a task set with three periodic tasks running on a single CPU (Table 1.1) scheduled under the Rate Monotonic algorithm:

Table 1.1. Three-task set example scheduled under RM.

Task	T_i	D_i	C_i
1	5	5	2
2	10	10	2
3	20	20	3

All tasks arrive (release their first jobs) at time 0. The total utilization is $U = 2/5 + 2/10 + 3/20 = 0.4 + 0.2 + 0.15 = 0.75$ (75%). According to the Liu-Layland criterion for $n = 3$, the bound is $3(2^{1/3} - 1) \approx 0.78$. Here $U = 0.75$ is under the bound, so we expect the task set to be schedulable by RM. Following the RM priority rule, Task 1 (period 5) has the highest priority, Task 2 (period 10) is medium-priority, and Task 3 (period 20) has the lowest priority.

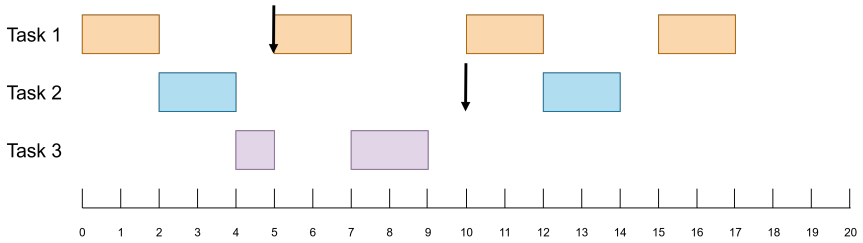

Figure 1.4. Rate Monotonic (RM) scheduling of a three-task set example.

Figure 1.4 presents the RM execution schedule across the first 20 time instants, which is the hyperperiod value (i.e. the least common multiple of the periods). At time 0, all three tasks release their first job simultaneously. Task 1, being highest priority, runs first. Task 1 executes for 2 time instants (from time 0 to 2). By time 2, Task 1 completes its job. At that moment, Task 2 and Task 3 are both ready. Task 2 has higher priority than 3, so Task 2 runs next. Task 2 runs for 2 time instants (2 to 4) and completes at time 4. Now Task 3 (lowest priority) finally gets to run at time instant 4, after Tasks 1 and 2 have finished their first jobs. The computation time for Task 3 is 3 time instants, but it will not get to complete it straight through because Task 1 has a period of 5, which means Task 1 releases its second job at time 5.

At time 5, the second release of Task 1 occurs. Task 1 preempts Task 3 immediately. Task 1 (job 2) runs from 5 to 7 time instants and completes its execution. Task 3 is still pending (2 time instants left) and the next release of Task 2 has not arrived yet. Thus, from 7 to 10 time instants, the processor returns to Task 3, allowing it to finish. At time instant 10, both Task 1 and Task 2 release new jobs. In RM, Task 1 has higher priority than Task 2, so the third job of Task 1 starts at time instant 10 and runs until time 12 when it completes. Task 2 (second job) now runs from time instant 12 to 14. By time 14, Task 2 finishes its execution.

After 10 time instants, Task 3 first job is done. Task 1 next release is at time 15, and Task 2 at time 20. So in the interval 14–15, the processor is actually idle. At time 15, Task 1 releases its fourth job and immediately executes from 15 to 17. At time 17, Task 1 finishes. The CPU goes idle again from 17 to 20 because Task 2 third job and Task 3 second job both arrive at time 20. At time 20, the cycle would repeat with new releases (Task 1 at 20, 2 at 20, 3 at 20, with Task 1 and 2 preempting Task 3 again). Thus, we have observed one full hyperperiod of scheduling. Throughout this schedule, all tasks met their deadlines: Tasks 1 and 2 finished each job before their next

release, and Task 3 finished its first job at time 10, well before its time 20 deadline. The RM policy ensured that at each release, the highest priority task runs first, which is why Task 3 execution got sliced by preemption.

This example highlights several important aspects of RM scheduling. Priority assignment by period truly determines the execution ordering: high-rate Task 1 frequently preempts the others, while low-rate Task 3 only runs when Tasks 1 and 2 are idle. Preemption can cause the execution of a task to be split (Task 3 was preempted once), but as long as the total CPU demand does not exceed what RM can handle, each task still completes by its deadline. This example confirms the schedulability test: with a total utilization of $U = 0.75$, all deadlines were indeed met.

RM scheduling is optimal, meaning no other static priority scheme could schedule a task set that RM cannot. Thus, for periodic task management on a single processor, RM scheduling provides a theoretically solid and practically supported strategy. RM serves as the baseline against which more complex scheduling algorithms are compared.

1.4.1.2. Deadline Monotonic

Deadline Monotonic (DM) [13, 45, 46] scheduling is a foundational fixed-priority algorithm for real-time systems that assigns task priorities according to their deadlines. In DM, the task with the earliest deadline (shortest relative deadline) is given the highest priority. This approach generalizes the well-known Rate Monotonic (RM) scheduling (which prioritizes tasks by period) to better handle tasks whose deadlines are not equal to their periods. By using deadlines as the priority metric, DM directly addresses scenarios where tasks have deadlines shorter than their release intervals, a common situation in many real-time applications. It can be proven that DM is an optimal fixed-priority scheduling algorithm for tasks with deadlines less than or equal to their periods, meaning if any static priority assignment can schedule such a task set, the deadline monotonic assignment will also succeed.

DM operates under classical real-time scheduling assumptions [46]: tasks are periodic or sporadic with known worst-case execution times and can preempt each other if necessary. Under these conditions, DM offers a strong guarantee: if tasks can be ordered by deadlines and each task worst-case response time is within its deadline, all deadlines will always be met at run-time.

Deadline Monotonic scheduling is a fixed-priority preemptive scheduling policy defined on a set of recurring real-time tasks. DM assigns priorities

to tasks based monotonically on their deadlines: a task with a shorter relative deadline is given a higher priority. Formally, for any two tasks τ_i and τ_j, if $D_i < D_j$, then $priority(\tau_i) > priority(\tau_j)$ [13]. This priority assignment is static, meaning it is determined before run-time and does not change during execution. At run-time, the scheduler always selects the job with the highest priority (shortest-deadline task among those released and not finished) to execute next, preempting lower priority tasks if necessary. This preemptive, highest priority first execution ensures that tasks with the most urgent deadlines get CPU time whenever they need it.

The theoretical significance of DM lies in its optimality under constrained deadlines. Constrained deadlines refer to systems where each task deadline D_i is less than or equal to its period T_i ($D_i \leq T_i$). In 1973, Liu and Layland proved that Rate Monotonic [22] (which is equivalent to Deadline Monotonic when $D_i = T_i$ for all tasks) is the optimal static priority assignment for periodic tasks with deadlines at period end. This result was later extended to the more general case of constrained deadlines: DM is an optimal static priority scheduling algorithm for periodic task sets with $D_i \leq T_i$. Optimality here means that if a set of tasks with constrained deadlines can possibly be scheduled by any fixed-priority assignment, then assigning priorities according to deadlines will successfully schedule them as well. In other words, no other static priority scheme can feasibly schedule a task set that DM cannot, given the same task model assumptions.

Schedulability analysis and utilization bounds

When deadlines are not equal to periods, the simple Liu-Layland RM utilization bound [22] no longer directly applies. Task sets under DM may achieve higher utilization in some cases or may miss deadlines even at lower utilization if deadlines are very tight relative to periods. Therefore, schedulability analysis for DM generally relies on response-time analysis or time-demand analysis rather than a single utilization threshold. A classic schedulability test for fixed-priority scheduling (including DM) is the Worst-Case Response Time (WCRT) test [47]. In this approach, for each task τ_i (in order of priority from highest to lowest), one computes the worst-case response time R_i, which is the maximum time from its release to completion under preemption by higher priority tasks. The task set is schedulable under DM if and only if every task worst-case response time is less than or equal to its deadline ($R_i \leq D_i$ for all i). Mathematically, R_i can be found by solving the recurrence equation 1.2 [47]:

$$R_i = C_i + \sum_{\tau_j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j \tag{1.2}$$

where $hp(i)$ is a set of tasks with higher priority (shorter deadlines) than τ_i. This formula accounts for the worst-case interference from all higher priority tasks that may execute during τ_i response interval. It assumes a synchronous release at time 0 of all tasks which maximizes contention for the CPU. The equation is typically solved by iteration, starting with $R_i = C_i$ and iterating until R_i converges to a stable value or exceeds D_i. If for some task the iteration yields $R_i > D_i$ (meaning that task would miss its deadline), the task set is declared not schedulable under DM. If all tasks satisfy $R_i \leq D_i$, the system is schedulable, and the analysis also provides the WCRT of each task. This response-time analysis is a necessary and sufficient schedulability test for DM on a single processor platform.

Deadline Monotonic task scheduling example

To better understand the Deadline Monotonic algorithm, a scheduling example is considered: a task set with three periodic tasks running on a single processor platform (Table 1.2).

Table 1.2. Three-task set example scheduled under DM.

Task	T_i	D_i	C_i
1	5	5	1
2	6	4	2
3	10	6	3

Under Deadline Monotonic, we first order tasks by their relative deadlines: Task 2 ($D_1 = 4$) has highest priority, Task 1 ($D_2 = 5$) is next, and Task 3 ($D_3 = 9$) has lowest priority, so the order is Task 2, Task 1 and Task 3.

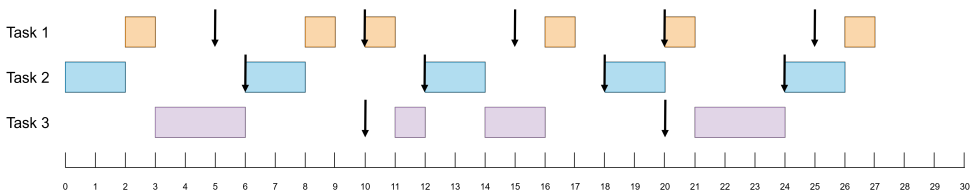

Figure 1.5. Deadline Monotonic (DM) scheduling of a three-task set example.

Figure 1.5 illustrates the Deadline Monotonic task scheduling. All 3 tasks release simultaneously, at time instant 0. DM scheduling will execute the first release of each task in priority order: Task 2 (until time 2), Task 1 (until time 3) and Task 3 (until time 6, which is also its deadline). The next job release with earliest deadline is that of Task 2 (must complete before time 10). Task 2 runs for 2 time instants and finishes executing at time 8. Next, the second release of Task 1 executes until time instant 9 (with a deadline at time 10).

The processor is idle for 1 time instant because the next job is released at time 10 (Task 1 with a deadline at time instant 15). Since the next release of the highest priority task from the set is at time 12, Task 3 (which is the lowest priority task) can execute from time instant 11 to 12, before it is preempted by Task 2. Task 2 (job 3) runs from time 12 to 14. Now, the task with the earliest deadline is Task 3 (job 2), which already executed for 1 time instant. The next release of Task 1 waits until Task 3 finishes executing (with a deadline at time 16). Then, job 4 of Task 1 executes from time instant 16 to 17. Task 2 will run its next job from time 18, until it finishes (with a deadline at time 22).

At time instant 20, the next job of Task 1 releases and executes immediately because the final job of Task 3 has a later deadline (both jobs release at the same time, but the deadline for Task 1 is at time 25, while the deadline for Task 3 is at time 26). Job 5 of Task 1 executes entirely, followed by the final release of Task 3, which also runs to its completion. The last release of Task 2 executes next (highest priority task, with a deadline at time instant 28), while job 6 of Task 1 is pending. Job 6 of Task 1 has the latest deadline (at time 30) and executes right after job 5 of Task 2 (from time instant 26 to 27).

The above example demonstrates how DM schedules tasks based on urgency (deadlines) and can succeed where purely rate-based scheduling might fail. Many RTOS products allow tasks to be assigned fixed priorities. System designers employing DM will determine task priorities offline according to deadlines and then configure the RTOS with those priority levels. The algorithm does not require complex runtime computations because it leverages the priority queue of the RTOS.

1.4.2. Dynamic priority algorithms

1.4.2.1. Earliest Deadline First

Earliest Deadline First (EDF) [11, 35] is a dynamic priority scheduling algorithm widely used in real-time systems. In EDF, the priority of each task is determined by its deadline (the time by which it must complete). At any scheduling decision point (such as when a task arrives or finishes), the EDF scheduler selects the task whose absolute deadline is earliest (the task with the least time remaining until its deadline) for execution. As tasks complete or new tasks arrive, the scheduler may preempt the current task if another task has an earlier deadline. In a system of periodic or sporadic tasks, EDF effectively maintains a priority queue ordered by deadlines, ensuring that at each moment the task with the most urgent deadline runs next.

If any scheduling policy can meet all deadlines for a given task set, then EDF can too, making it optimal for single processor scheduling of independent preemptive tasks. This optimality means EDF will successfully schedule all tasks to meet their deadlines whenever such a schedule is at all possible under any algorithm.

Schedulability analysis and utilization bounds

The foundational work of Liu and Layland (1973) [22] established key properties of EDF in the context of periodic task systems. For a set of independent periodic tasks where each task deadline equals its period, Liu and Layland proved that EDF will schedule all tasks to meet their deadlines as long as the total CPU utilization does not exceed 100%. In other words, a feasible EDF schedulability exists if (equation 1.3):

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1 \tag{1.3}$$

where each task i has worst-case execution time C_i and period (equal to its deadline) T_i. This is a simple schedulability test for EDF: if the CPU utilization is at most 100%, EDF can guarantee all deadlines are met.

The optimality of EDF on a single processor also extends beyond periodic tasks. In a more general sense, EDF is optimal for any set of independent jobs with deadlines on a single processor: if any scheduling algorithm can schedule the jobs to meet all deadlines, EDF can do so as well. This optimality was first highlighted by Dertouzos (in 1974) [48] and is sometimes

called the EDF scheduling theorem. Additionally, ordering tasks by earliest deadline is a proven optimal strategy for minimizing maximum lateness, meaning EDF also tends to minimize how far past their deadlines tasks run if deadlines cannot all be met.

It should be noted, however, that the simple utilization-based schedulability test is sufficient but not necessary for EDF. It holds exactly for implicit-deadline systems (where deadlines are equal to periods). When task relative deadlines are shorter or longer than their periods, more complex schedulability analyses (e.g., time-demand analysis [49]) are used to determine if EDF can meet all deadlines. In such cases, EDF is still optimal, but one must account for the specific deadline parameters in the schedulability test.

Implementation in hard real-time systems

In hard real-time systems, timing constraints are absolute, which a missed deadline is considered a system failure. Therefore, using EDF in a hard real-time context requires stringent analysis to guarantee that every deadline will always be met under worst-case conditions. The advantage of the EDF utilization bound means that, in principle, as long as the total workload does not exceed the processor capacity, a set of hard real-time tasks with implicit deadlines will all meet their deadlines. This makes EDF efficient for maximizing resource usage [36]. Schedulability tests (like the utilization test or more exact tests) are performed offline to ensure the task set is feasible. If the task set passes these tests, EDF can be confidently used in a hard real-time system to guarantee all deadlines.

Despite its optimality, EDF has not been as widely adopted in industry for hard real-time systems as fixed-priority schemes. One practical concern is that the dynamic priority nature of EDF can make system behavior and timing less transparent or predictable, especially during overloads [36]. In an overload (when tasks collectively require more CPU time than available), EDF does not have a built-in rule about which tasks to drop. This means the set of deadlines missed can be unpredictable, depending on the exact task timing and arrival pattern. By contrast, a fixed-priority scheme like RM will predictably let the lower priority (typically less critical) tasks miss their deadlines first, while the highest priority tasks continue to meet theirs. This property is often desirable in safety-critical hard real-time systems, where one would prefer to gracefully degrade (drop less critical tasks) rather than unpredictably miss a critical deadline. Moreover, implementing EDF

can be more complex because it requires maintaining an ordered queue by deadlines and handling frequent priority changes [11, 36]. As a result, many certified hard real-time operating systems (for avionics, automotive, etc.) have favored fixed-priority or static scheduling, which are simpler to verify and analyze in practice.

Another drawback of EDF scheduling is that it does not directly address resource sharing between tasks. Since EDF prioritizes tasks solely based on their deadlines, integrating a resource-sharing policy can be challenging. The lack of built-in mechanisms for coordinating access to shared resources may lead to issues such as priority inversion or deadlocks [36], requiring additional synchronization strategies to ensure safe and efficient task execution.

Nevertheless, EDF can be, and is used, in hard real-time contexts when its benefits outweigh these concerns. Modern real-time kernels and standards have started to include EDF-based scheduling classes. For example, the Linux kernel SCHED_DEADLINE policy (merged in Linux 3.14) [50, 51] is an EDF scheduler augmented with constant bandwidth servers for isolation. It allows tasks to declare deadlines and execution budgets, and the kernel schedules them by EDF, while enforcing that each task does not exceed its reserved CPU bandwidth. This is a hard real-time scheduling feature, and when used with proper admission control, it provides deadline guarantees on Linux. The introduction of EDF in such general-purpose OS demonstrates that, with careful design, EDF can meet hard real-time requirements while offering the advantage of high utilization.

Another consideration in hard real-time EDF usage is jitter and stability [52]. However, research has shown that many such concerns (e.g., that EDF causes more jitter or is harder to analyze) can be mitigated. With modern analysis techniques (e.g., response-time analysis for EDF or time-demand analysis) and enforcement mechanisms like bandwidth reservation [53], EDF can be made as deterministic as needed for hard deadlines.

In summary, for hard real-time systems, EDF offers theoretical optimality and full resource utilization, but system designers must ensure schedulability and consider overload scenarios. When applied with proper analysis and combined with reservation servers [54] or priority inheritance [43] for resource access, EDF can meet hard real-time requirements effectively.

Implementation in soft real-time systems

In soft real-time systems, deadlines are important but occasionally missing a deadline is tolerable and does not constitute a total system failure. Instead, the performance may degrade gracefully when deadlines are missed (for example, a video frame arrives late, causing a brief glitch, but the system continues to operate). Because soft real-time systems allow some deadline misses, they are often run at higher average utilizations and with less pessimistic worst-case provisioning. This means the system might sometimes be overloaded (demand exceeds capacity), and not every deadline can be met. EDF is an efficient scheduling algorithm for such cases due to its optimality properties [55, 56]: when the system is not overloaded, EDF will meet all deadlines (as it would in a hard real-time system), and when overloads occur, EDF will make the best effort to minimize deadline misses or lateness.

One way to view EDF in a soft context is that it maximizes the number of deadlines met (or equivalently minimizes total deadline miss time) when not all tasks can be scheduled. Since EDF always schedules the task with the earliest deadline, it tends to ensure that tasks miss their deadlines by the smallest possible margin if misses are inevitable. In fact, EDF minimizes the maximum lateness of tasks in a schedule, which is a desirable property for soft real-time performance. For example, if tasks occasionally overload the CPU such that not all can finish on time, EDF will try to complete as many as possible by their deadlines, and any that miss will be the ones with the latest deadlines (hence presumably less urgent). This behavior is appropriate in scenarios like multimedia streaming, where it is better for the scheduler to drop or delay a frame with a far deadline than one with an imminent deadline [57].

Soft real-time systems also often employ EDF alongside overload management techniques. In an overload scenario, EDF by itself, simply keeps running the earliest deadlines, and if overload persists some deadlines of other tasks will pass. For instance, a system might implement value-based scheduling (assigning value to each task and skipping less-important tasks during heavy load) on top of EDF, or use a feedback scheduler [58] that deliberately sheds load when it detects sustained overload. Research on soft real-time scheduling includes concepts like imprecise computation and Quality of Service (QoS) guarantees [59], where tasks can degrade their workload if needed to avoid catastrophic overload. EDF provides an excellent baseline for these because of its optimal use of the processor: system designers can

focus on which tasks to drop or degrade, knowing that EDF will efficiently schedule the rest.

Another aspect is variability in execution and arrival times in soft real-time systems. Since deadlines can be missed occasionally, tasks might not have strict worst-case execution times enforced. EDF can dynamically adjust priorities as deadlines. Systems like online transaction processing, multimedia, or telecommunications often leverage EDF to schedule tasks or packets because it naturally prioritizes the timeliest data. For example, packet scheduling in networks can use EDF to ensure time-sensitive packets (like voice or video frames) get transmitted by their deadlines, improving the quality of service [60].

Overall, in soft real-time systems, EDF is a flexible and efficient scheduling algorithm. It does not require conservative worst-case analysis to the same extent as hard real-time systems, instead, the system can be loaded optimistically and EDF will do its best to meet deadlines. Empirical studies have shown that many alleged drawbacks of EDF (such as unpredictable behavior under overload or higher overhead) can be managed. In practice, EDF often outperforms fixed-priority schedulers in soft real-time environments by delivering lower average response times and higher resource utilization.

Earliest Deadline First task scheduling example

To understand how EDF operates, let us consider a simple scenario with three tasks (Table 1.3). We will track how EDF schedules these tasks on a single processor system.

Table 1.3. Three-task set example scheduled under EDF.

Task	T_i	D_i	C_i
1	6	6	2
2	8	8	3
3	12	12	2

The processor utilization is less than 1, therefore the task set is schedulable: $U = 1/4 + 2/6 + 3/8 = 0.25 + 0.333 + 0.375 = 0.95$, according to equation 1.3 ($U = 0.95 \leq 1$).

Figure 1.6 illustrates this scheduling sequence.

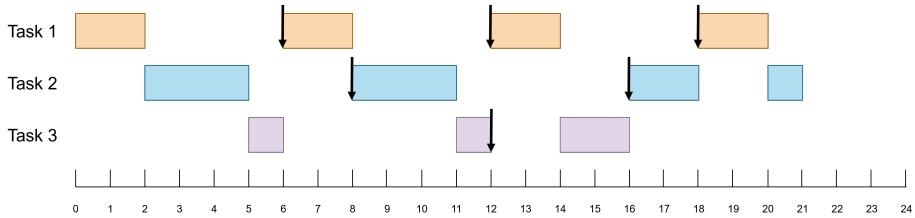

Figure 1.6. Earliest Deadline First (EDF) scheduling of a three-task set example.

Tasks 1, 2 and 3 arrive at the same time but are ordered according to their deadlines (the task with the earliest deadline is always executed first). Therefore, Task 1 will start executing at time instant 0 and complete at time instant 2. Next, Task 2 will execute from time instant 2 to 5 (before its deadline at time instant 8). Task 3 has the latest deadline, so it executes after Tasks 1 and 2, from 5 to 6. The next job of Task 1 arrives at time instant 6, and since its deadline is earlier than Task 3 (with deadline at time 12), it preempts Task 3 and executes from 6 to 8. At time instant 8, Task 2 arrives, and, because it also has an earlier deadline than Task 3, executes from time instant 8 to 11. At time instant 11, there is no other task in the ready queue, so Task 3 finishes executing by running from 11 to 12.

After 12 time instants the third release of Task 1 takes place. Task 1 executes from 12 to 14. Next, the only task available to execute is Task 3 (Task 2 did not arrive yet). Task 3 runs from time instant 14 to 16 and finishes executing. At time instant 16, Task 2 arrives and executes from 16 to 18, before it is preempted by Task 1, which has an earlier deadline. Task 1 runs from 18 to 20 and finishes. Finally, Task 2 executes from time instant 20 to 21. At this point, a full hyperperiod of scheduling was completed and all tasks met their deadlines.

Throughout this timeline, EDF dynamically adjusted to arrivals. Therefore, whenever a new task with an earlier deadline appeared, it was scheduled immediately, ensuring no imminent deadline was missed. Later-arriving tasks with later deadlines were halted. This behavior exemplifies how EDF always keeps the processor busy with the most time-critical task.

EDF is optimal for single-processor scheduling of deadline-constrained tasks and can achieve full processor utilization while ensuring all deadlines are met. It is applicable to both hard and soft real-time systems, albeit with different considerations: in hard real-time systems, ensuring that no deadlines are missed requires thorough schedulability analysis and effective safeguards against system overloads, whereas in soft real-time systems, EDF is often used to maximize throughput and minimize lateness, while occasional deadline misses may occur. Modern operating systems and standards are increasingly incorporating EDF-based scheduling due to these benefits.

1.4.2.2. Least Laxity First

Least Laxity First (LLF) [61, 62, 63], also known as Least Slack Time First (LST), is a dynamic priority scheduling algorithm used in real-time systems. In LLF, the priority of each task is determined by its laxity (or slack time), which represents how much leeway the task has before missing its deadline. Formally, for a task (or job) with absolute deadline d, current time t, and remaining execution time c', the laxity s, is defined as equation 1.4 [61]:

$$s = d - t - c' \tag{1.4}$$

This value is essentially the spare time left for the task: it indicates how long the task can afford to wait and still finish by its deadline. A task with zero laxity has no slack and must execute immediately to avoid missing its deadline. LLF scheduling continuously evaluates the laxity of all ready tasks and always selects the task with the least laxity (smallest slack time) to run next. This reduces the likelihood of deadline misses and gives highest priority to the most time-critical tasks at each moment.

A LLF scheduler will preempt running tasks whenever another task's laxity becomes smaller. This dynamic adjustment means task priorities can change at runtime based on the remaining computation and time to deadline. If two or more tasks share the same laxity

value, one could choose the task with the earlier deadline or simply the one that arrived first. Because laxity depends on knowing each task execution time, LLF assumes the system can estimate or measure remaining execution times for tasks. This makes LLF more complex to implement, as it requires continually updating laxities and may rely on worst-case execution time (WCET) estimates for each job.

Working principle of Least Laxity First algorithm

LLF is categorized as a dynamic priority algorithm because priorities are determined online, based on the current state of tasks. This is in contrast to static priority algorithms (like Rate Monotonic scheduling) where task priorities are fixed in advance.

The basic operation of a LLF scheduler involves the following steps, repeated whenever scheduling decisions are made (on task completions or arrivals) [61, 62, 63]:

1. **Calculate laxity for each ready task:** The scheduler computes each task current slack time $s = d - t - c'$. Tasks that are not yet released or already completed are excluded.

2. **Select task with minimum laxity:** The ready task with the smallest laxity (most urgent, least slack) is chosen to run next on the processor.

3. **Execute tasks and update slack:** The chosen task runs, typically, for a short time frame or until a new scheduling event. As time progresses and tasks execute, the scheduler updates the remaining execution times and recomputes laxities. If a different task has smaller laxity, the scheduler will preempt the current task in favor of the new task.

If multiple tasks have equal laxity, the scheduler can use a secondary rule (for example earliest deadline) to decide which to run.

Because laxities can change frequently, LLF may trigger frequent preemptions. In the worst case, two or more tasks with nearly equal slack time can cause the scheduler to swap back and forth between them at each time tick or small interval. Due to these numerous preemptions LLF can suffer from high runtime overhead. Therefore, vari-

ous refinements to basic LLF (such as Modified LLF [61] or algorithms like Earliest Deadline until Zero Laxity [64]) have been proposed to reduce unnecessary context switches by not switching tasks until absolutely necessary.

LLF can produce a feasible schedule "if and only if" a feasible schedule exists for the task set (assuming tasks can be preempted). Like EDF, LLF can therefore achieve 100% processor utilization for schedulable task sets, unlike fixed-priority approaches that might leave CPU capacity unused. The trade-off is that continuous priority adjustments make it less predictable and more expensive to run in a real-time system (due to the computational cost of computing laxities and the runtime cost of context switches).

Least Laxity First task scheduling example

This section presents an example of LLF scheduling on a single processor system. A set of three independent tasks is considered, with the following parameters (Table 1.4):

Table 1.4. Three-task set example scheduled under LLF.

Task	T_i	D_i	C_i
1	5	5	2
2	8	8	2
3	10	10	3

All tasks arrive at time instant 0 (for simplicity, all tasks are ready simultaneously). The task scheduling is presented in Figure 1.7:

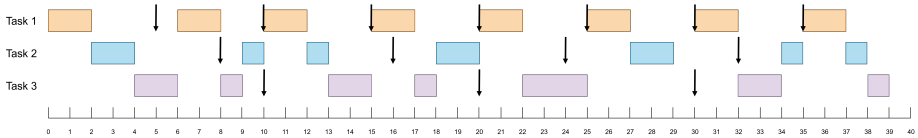

Figure 1.7. Least Laxity First (LLF) scheduling of a three-task set example.

At $t = 0$ the laxity of each task is calculated by using equation 1.4:

$s_1 = 5 - 0 - 2 = 3$
$s_2 = 8 - 0 - 2 = 6$
$s_3 = 10 - 0 - 3 = 7$

As Task 1 has least laxity, it will execute first, with highest priority. Similarly, at $t = 1$, the laxity is calculated again for each task: $s_1 = 3$, $s_2 = 5$ and $s_3 = 6$. Due to least laxity, Task 1 continues to execute.

At $t = 2$, the first instance of Task 1 finishes executing, so the laxities of Tasks 2 and 3 are compared:
$s_2 = 8 - 2 - 2 = 4$
$s_3 = 10 - 2 - 3 = 5$

The first job of Task 2 starts executing because it has less laxity than Task 3. At $t = 3$, both s_2 and s_3 are 4. If two tasks have the same laxity, run the task with the earliest deadline. Therefore, Task 2 continues to execute until it finishes. At $t = 4$ there is no other ready task in the system, so Task 3 executes until $t = 5$.

At $t = 5$ Task 1 is released, thus, laxities are calculated again:
$s_1 = 10 - 5 - 2 = 3$
$s_3 = 10 - 5 - 2 = 3$

Task 3 is allowed to continue its execution until $t = 6$, because Tasks 1 and 3 have the same laxity and deadline.

At $t = 6$, laxities are updated: $s_1 = 2$ and $s_3 = 3$. Therefore, Task 3 is preempted and the second instance of Task 1 executes until $t = 7$.

At $t = 7$: $s_1 = 2$ and $s_3 = 2$. Task 1 finishes executing at $t = 8$.

The next instance of Task 2 arrives at $t = 8$. The laxities are:
$s_2 = 16 - 8 - 2 = 6$
$s_3 = 10 - 8 - 1 = 1$

Task 3 has the smallest laxity value, so it executes until $t = 9$.

At $t = 9$, since there is no other ready task, job 2 of Task 2 executes until $t = 10$.

Now, all 3 tasks have instances ready to execute:
$s_1 = 15 - 10 - 2 = 3$
$s_2 = 16 - 10 - 1 = 5$
$s_3 = 20 - 10 - 3 = 7$

Task 1 has the least laxity, so it preempts Task 2 and executes until $t = 11$. At $t = 11$: $s_1 = 3$, $s_2 = 4$ and $s_3 = 6$. Task 1 runs until it finishes at $t = 12$.

The schedule continues based on the same procedure:
$t = 12$: $s_2 = 3$ and $s_3 = 5$, run Task 2
$t = 13$ to $t = 15$: run Task 3
$t = 15$: $s_1 = 3$ and $s_3 = 4$, run Task 1
$t = 16$: $s_1 = 3$, $s_2 = 6$ and $s_3 = 3$, run Task 1
$t = 17$: $s_2 = 5$ and $s_3 = 2$, run Task 3
$t = 18$ to $t = 20$: run Task 2
$t = 20$ $s_1 = 3$ and $s_3 = 7$, run Task 1
$t = 21$ $s_1 = 3$ and $s_3 = 6$, run Task 1
$t = 22$ to $t = 24$: run Task 3
$t = 24$: $s_2 = 6$ and $s_3 = 5$, run Task 3
$t = 25$: $s_1 = 3$ and $s_2 = 5$, run Task 1
$t = 26$: $s_1 = 3$ and $s_2 = 4$, run Task 1
$t = 27$ to $t = 29$: run Task 2
$t = 29$ to $t = 30$: idle
$t = 30$: $s_1 = 3$ and $s_3 = 7$, run Task 1
$t = 31$: $s_1 = 3$ and $s_3 = 6$, run Task 1
$t = 32$: $s_2 = 6$ and $s_3 = 5$, run Task 3
$t = 33$: $s_2 = 5$ and $s_3 = 5$, run Task 3
$t = 34$: $s_2 = 4$ and $s_3 = 5$, run Task 2
$t = 35$: $s_1 = 3$, $s_2 = 4$ and $s_3 = 4$, run Task 1
$t = 36$: $s_1 = 3$, $s_2 = 3$ and $s_3 = 3$, run Task 1
$t = 37$: $s_2 = 2$ and $s_3 = 2$, run Task 2
$t = 38$: run Task 3

Least Laxity First demonstrates an optimal approach to scheduling by always focusing on the task with the most urgent remaining workload. In practice, simple LLF is rarely deployed due to the difficulties of continuously computing laxity and the potential for excessive preemptions. Nonetheless, the concept of laxity influences many modern scheduling techniques. Some real-time operating systems and research frameworks incorporate laxity-based decisions in

hybrid algorithms (for instance, scheduling policies that behave like EDF until tasks become critical, then switch to LLF to avoid any slack running out). LLF is also important in handling aperiodic tasks or mixed-criticality systems where task execution times vary significantly.

2. REAL-TIME TASK SCHEDULING WITH MATLAB

MATLAB, and especially Simulink, provides a powerful environment to simulate real-time task scheduling for embedded systems [65, 66, 67]. Unlike pure algorithm simulations which often ignore timing, Simulink can incorporate scheduling effects (task periods, priorities, preemption, etc.) into the model. This helps detect timing problems (like missed deadlines or task overruns) during simulation rather than discovering them on hardware. MathWorks offers dedicated tools (e.g., SoC Blockset and Simulink Coder [68]) that allow modeling and scheduling of tasks. This enables the testing of algorithms such as Rate Monotonic (RM) or Earliest Deadline First (EDF) under realistic conditions.

In MATLAB/Simulink, time-based (periodic) scheduling refers to tasks executed on a regular period, driven by a timer tick (like a hardware timer interrupt), whereas event-based (aperiodic) scheduling involves tasks triggered by asynchronous events. Simulink Coder (formerly Real-Time Workshop) supports both paradigms for code generation, and these can be simulated in Simulink. In practice, this means that periodic tasks can be modeled to run at fixed intervals and event-driven tasks can be modeled to run in response to external triggers (interrupts, messages, etc.). The simulation will comply with these timing specifications.

2.1. Modeling periodic and aperiodic tasks in Simulink

To simulate a scheduling algorithm, the first step is to model each task in the application. In Simulink, a common approach is to use subsystems or model references to represent tasks:

- Periodic (time-driven) tasks - each periodic task is modeled as an atomic subsystem (or referenced model) with a specified sample time equal to its period. In the *Block Parameters* of the subsystem, it

should be marked as a *Periodic partition* (treat as atomic unit, schedule as periodic) and given a name (partition name) corresponding to the task. This indicates to Simulink that the subsystem executes as a time-triggered task with that period.

- Aperiodic (event-driven) tasks - each event-driven task is modeled as a function-call subsystem (or an atomic subsystem marked as *Aperiodic partition*) that executes when triggered. Instead of a fixed sample time, these tasks have an input trigger port. An external event source (e.g., a *Simulink Trigger* block or an *Async Interrupt* block in more advanced setups) will fire a function-call signal to activate the task. Each such task should also be named (for identification in the scheduler).

Using these modeling patterns, the system can be partitioned into distinct tasks, each with its own context. All these task subsystems are typically placed inside a single processor model (often a referenced model representing the processor or CPU). This allows a central scheduler component to manage them collectively.

2.2. The Task Manager and Schedule Editor in SoC Blockset

MathWorks SoC (System on Chip) Blockset [68] provides a high-level way to simulate task scheduling via the *Task Manager* block. The Task Manager block is designed to create and manage task execution in a Simulink model. All the tasks are listed in this block (using the GUI or programmatically). Each task has a name, type (periodic or event-driven), period (if periodic), and other parameters like priority and CPU core assignment. The Task Manager will then simulate the concurrent execution of these tasks as if they were scheduled on a real processor. Key features of the Task Manager include [68]:

- Scheduling by priority and core - it executes tasks based on their configured priority and assigned processor core. By default, time-driven periodic tasks have their priority determined automatically (for RM, a faster period implies higher priority).

Static priorities can also be assigned if needed. Event-driven tasks (aperiodic) have their priority set explicitly since they do not have a fixed rate. The Task Manager will simulate preemption: a higher priority task on the same core will preempt a lower priority one if it becomes ready to run. Tasks can also be assigned to different cores to simulate parallel execution on a multi-core processor (tasks on different cores run concurrently in the simulation).

- Task execution timing and overruns - to simulate real-time behavior, the execution time (duration) of each task must be provided. The Task Manager block allows several methods to specify task duration: a statistical distribution for execution time can be defined in its parameters (e.g., a normal distribution with mean and variance) or an external input signal or data file with execution time measurements can be supplied. The scheduler uses these durations to decide if a task overruns its period. The Task Manager can model what happens on overruns (for instance, to drop a task instance if it has not finished before the next release, or to let it overrun and attempt to catch up later).
- Schedule visualization and analysis - the SoC Blockset also provides a textitSchedule Editor tool and built-in profiling. The Schedule Editor is a GUI that integrates with the Task Manager, which displays task partitions (atomic subsystems) and enables configuration of their execution order and priorities in a timeline view. The Task Manager can optionally be set to follow the ordering specified by the Schedule Editor, which is useful in scenarios involving multiple tasks with identical periods or priorities where a specific execution sequence or core assignment is required. During simulation, the Task Manager logs task execution events (including start, finish, and preemption) which can be visualized using the *Simulation Data Inspector* or dedicated plotting functions. This facilitates analysis of the scheduling behavior, including task execution timing, duration, and any missed deadlines.

The Task Manager block combined with the Schedule Editor provides a way to simulate an RTOS-like scheduler within Simulink. It conforms to task parameters like period, priority, and core, and simulates preemptive multitasking accordingly.

2.3. Implementing and simulating a scheduling algorithm

This subchapter lists the general steps to implement a scheduling algorithm and simulate it in MATLAB/Simulink. The version used for the screen captions is MATLAB R2024b.

1. **Define tasks in the model**

Let us partition our system into separate tasks. For each periodic task, we will create a Subsystem (Figure 2.1).

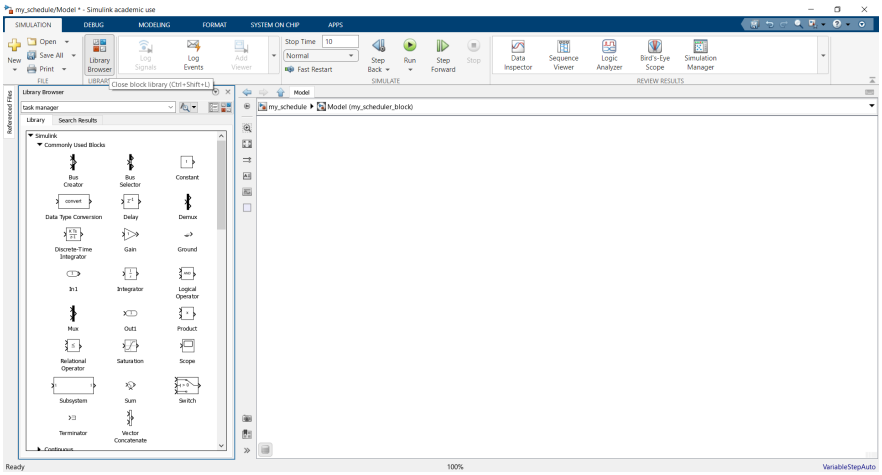

Figure 2.1. Create a Subsystem block.

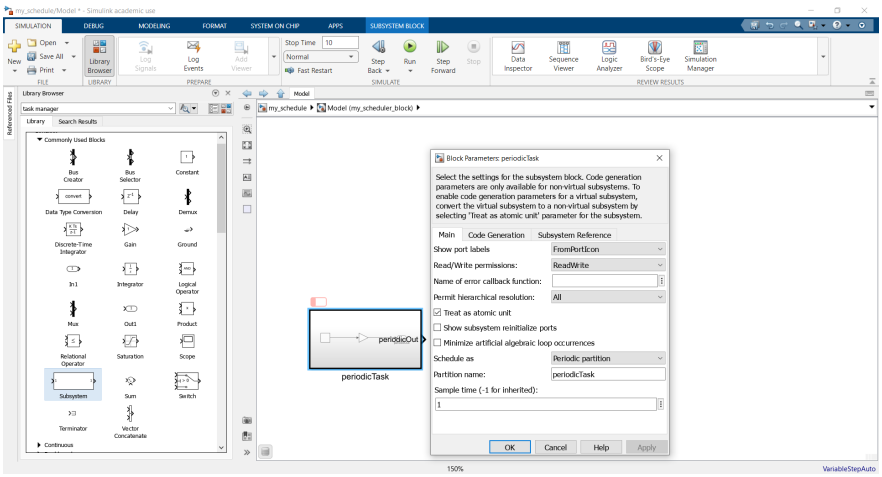

Figure 2.2. Set Block Parameters for a periodic task.

Right-click on the *Subsystem -> Block Parameters*, check *Treat as atomic unit* and set its *Sample Time* to the desired period (Figure 2.2). Mark it as a *Periodic partition* (in *Block Parameters*, set *Schedule* as *Periodic partition*). Name each partition ("Task1", "Task2", etc.).

Double click on the newly created block and the structure should look like Figure 2.3. It contains a *Counter Free-Running* block, a *Multiply* block and an *Outport*.

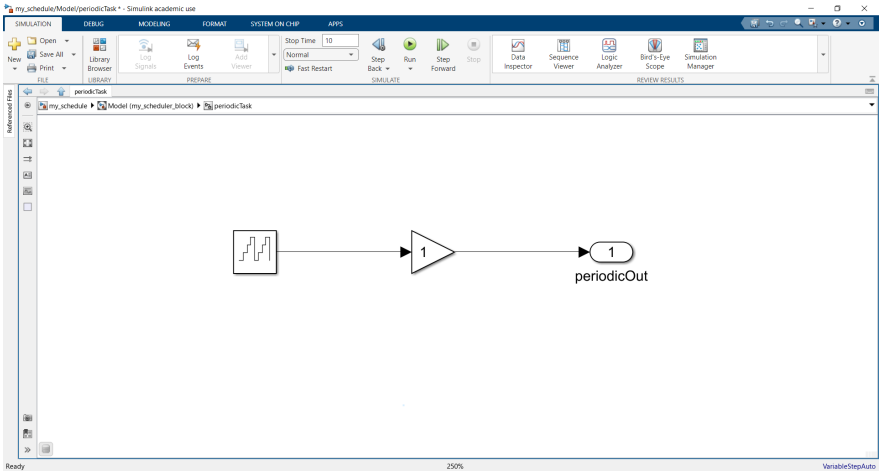

Figure 2.3. Structure example of a Subsystem block.

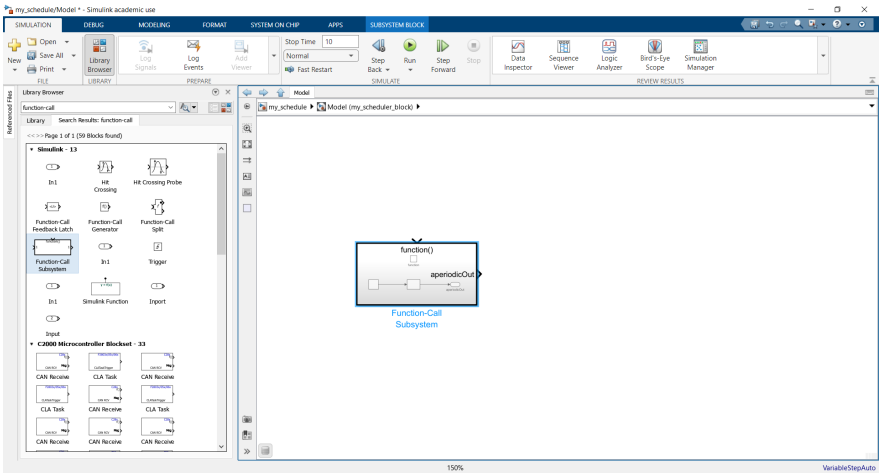

Figure 2.4. Create a Function-Call Subsystem block.

For each aperiodic task, use a *Function-Call Subsystem* (Figure 2.4) or

atomic subsystem with *Schedule* as *Aperiodic partition* and no fixed sample time. This will have an input trigger port for events.

The *Functional-Call Subsystem* comprises of a *Counter Free-Running* block, an *Add Constant* block, an *Outport* and a *TriggerPort* block (Figure 2.5).

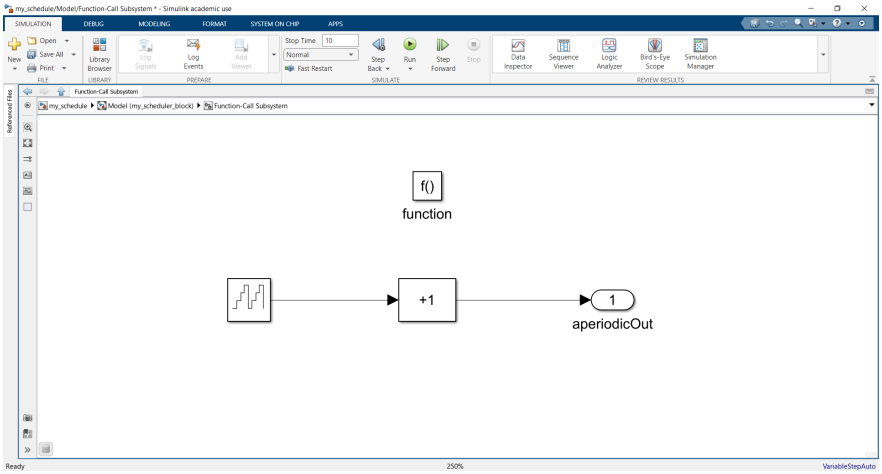

Figure 2.5. Structure example of a Functional-Call Subsystem block.

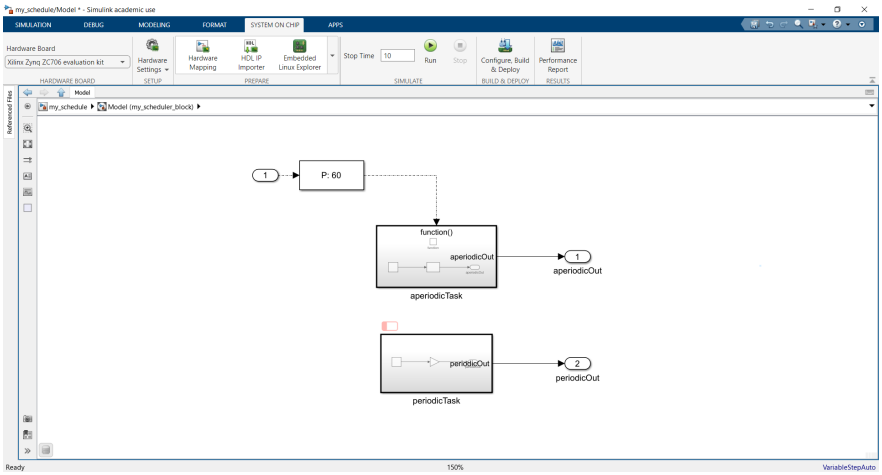

Figure 2.6. Two-task set implementation example.

The final setup of the two tasks is shown in Figure 2.6. An *Inport* and an *Asynchronous Task Specification* block was added for the aperiodic task. *Outports* were added for both tasks.

The task set created can be saved from the *Simulation* tab as a ".slx" file for future use.

2. **Insert the Task Manager block**

In our top-level model (the "CPU" model), we will add a *Task Manager* block (found in *SoC Blockset -> Processor Task Execution* library, Figure 2.7).

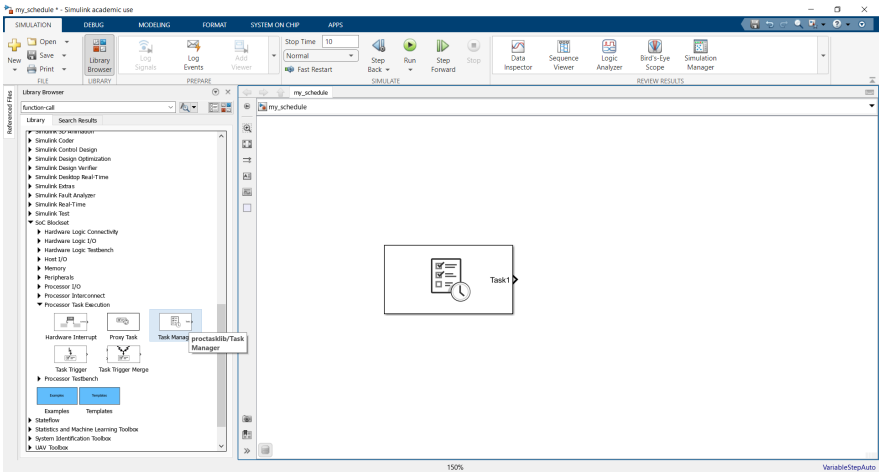

Figure 2.7. Create a Task Manager block.

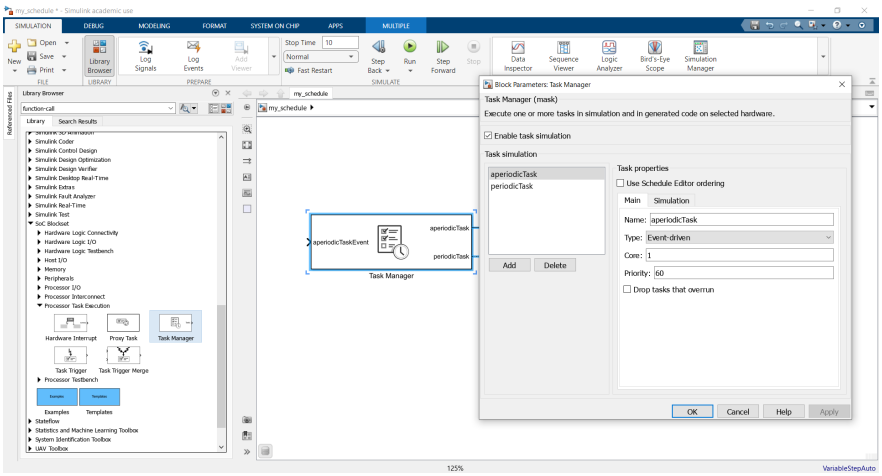

Figure 2.8. aperiodicTask entry in the Task Manager dialog box.

In the *Task Manager* dialog (double-click on the *Task Manager* block), click *Add* to create an entry for each defined task. Set each task *Name*

exactly to match the partition (subsystem) name. For each entry, choose the *Type* (timer-driven or event-driven) and if timer-driven, set its period (this should match the subsystem sample time, Figures 2.8 and 2.9).

For event-driven tasks, no period is needed, they will run when triggered. In this case, if we want to manually assign priorities, we can set a *Priority* value for each task (Figure 2.8). Otherwise, we may leave periodic task priority on "auto", which by default gives shorter-period tasks higher priority. We can also assign each task to a *Core* number if simulating a multi-core scenario (tasks on the same core will preempt each other by priority, tasks on different cores run concurrently).

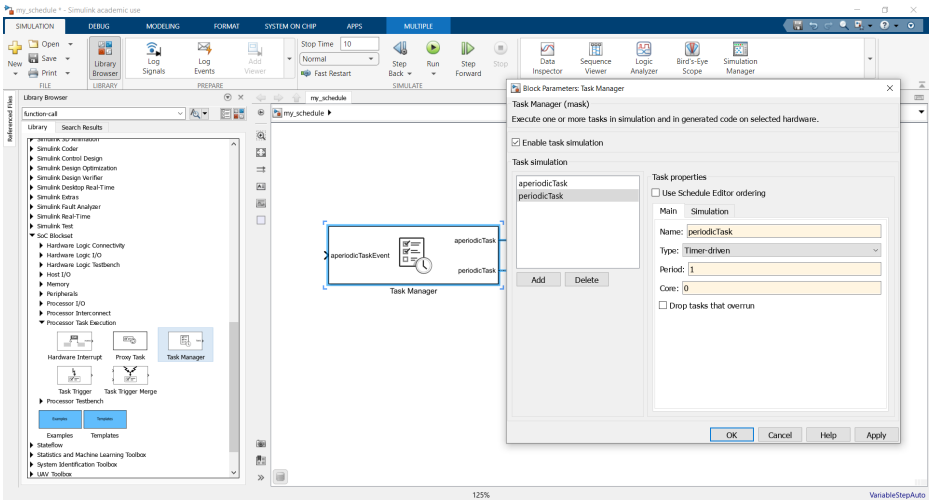

Figure 2.9. periodicTask entry in the Task Manager dialog box.

Ensure the *Task Manager* input and output ports (one per task) are connected (Figure 2.11): each periodic task will correspond to a rate port on the *Model* block for the processor, and each event-driven task port connects to the *Function-Call* trigger of its *Subsystem*. The task set ".slx" file created previously can be uploaded through a *Model* block (Figure 2.10). An *Event Source* block is used for the aperiodic task.

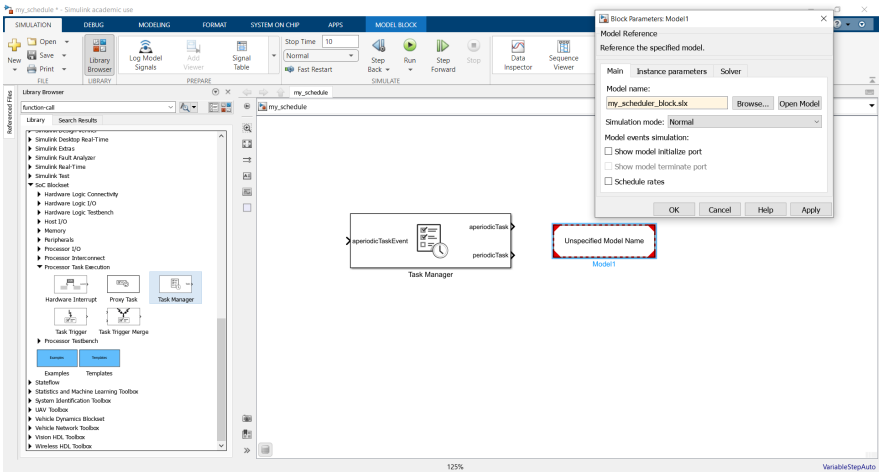

Figure 2.10. Option to reference a specific task set file in the Block Parameters dialog box.

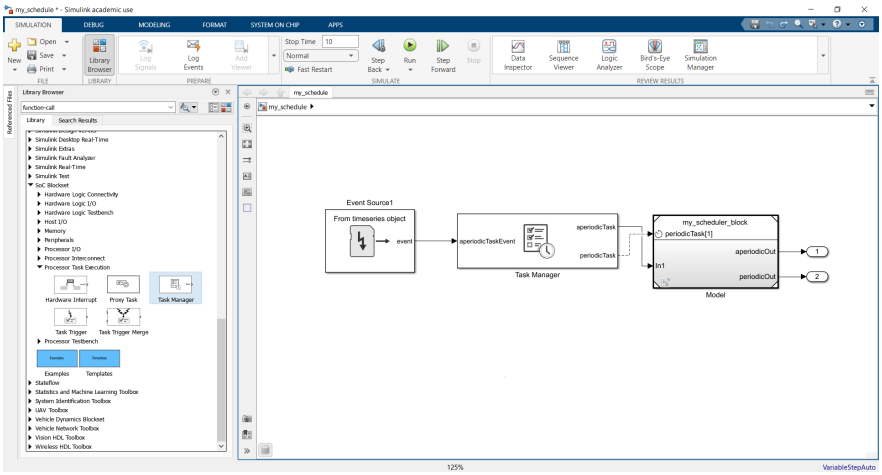

Figure 2.11. Task Manager output ports connections.

3. **Provide task execution times**

Specify how long each task takes to execute, so the simulation can determine preemption and overlaps. Open the *Task Manager Simulation* tab. There are tree options to specify a duration for each task:

- Dialog (statistical, Figure 2.12) - Enter a probability distribution for execution time (e.g., a mean and standard deviation for a normal

distribution). The *Task Manager* will sample from this to model variability.

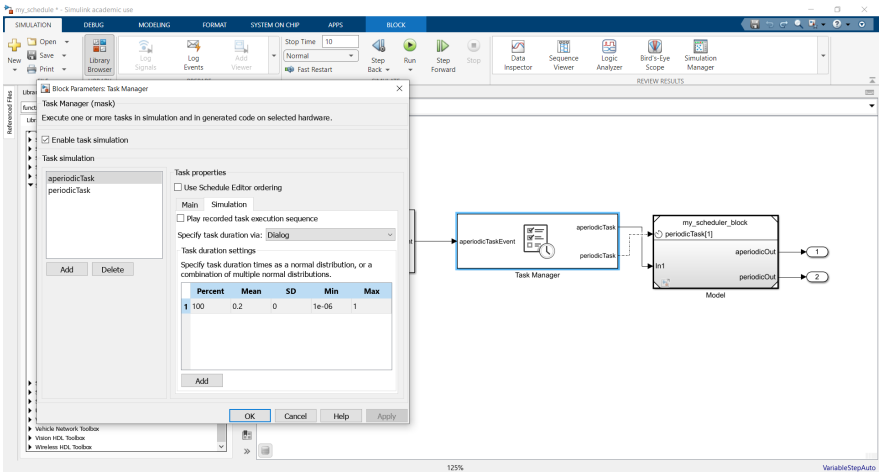

Figure 2.12. Dialog option in the Task Manager dialog box.

- From file (Figure 2.13) - We can load a recorded profile of execution times (for example, data measured from a real system or a previous simulation run).

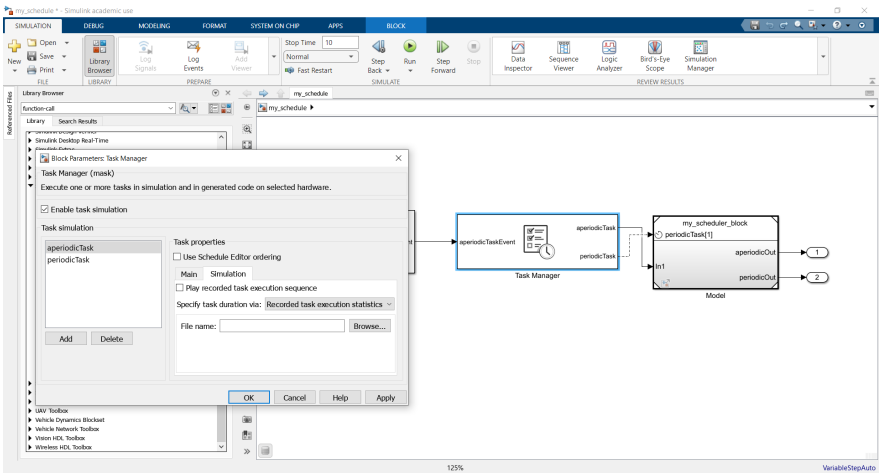

Figure 2.13. Recorded task execution statistics option in the Task Manager dialog box.

- Input port (Figure 2.14) - For maximum flexibility, choose to provide the duration via an input. This adds an input port for each task

execution time, where we can connect a signal or a custom logic that computes the execution time in real-time.

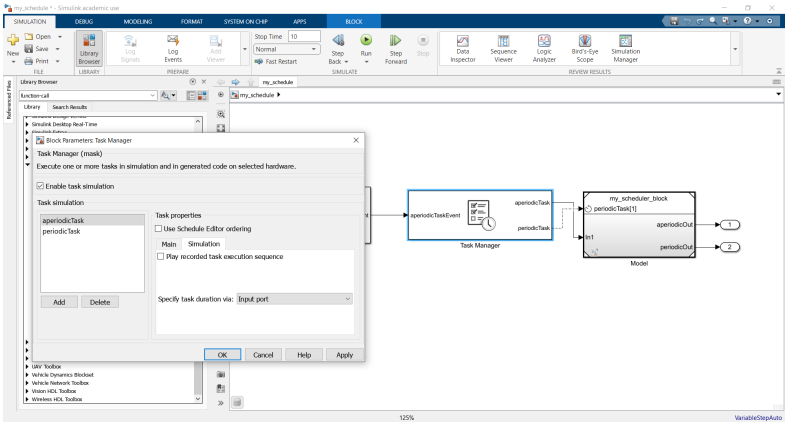

Figure 2.14. Input port option in the Task Manager dialog box.

If there is no exact data, a simple approach is to set a fixed duration or a fraction of the period (e.g., 50% of the period) for each task to see how the scheduler behaves. The execution duration is crucial because it enables the simulation of task overruns.

4. **Run the simulation**

Before starting the simulation (Figure 2.16), we must select the hardware board for our model. Simulation and core generation will be tailored to the selected board. To do this, go to the *System on Chip* tab, under *Hardware Settings* and select *Model Settings* (Figure 2.15).

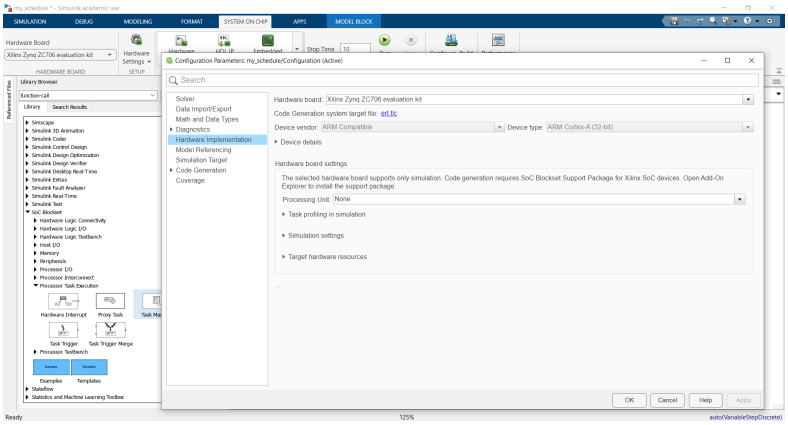

Figure 2.15. Set model Configuration Parameters.

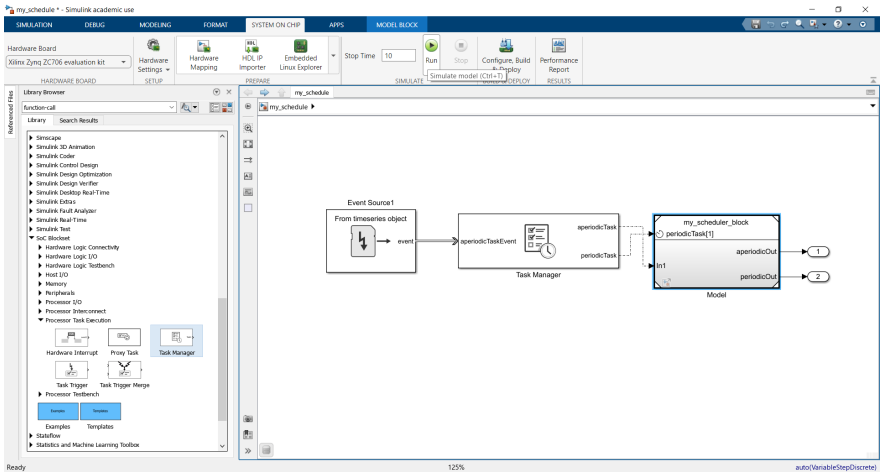

Figure 2.16. Run the task scheduling simulation.

The *Task Manager* will now orchestrate the task executions according to the specified periods, priorities, and cores:

- Rate Monotonic behavior - By default, if we have multiple periodic tasks, the one with the fastest period will preempt slower ones on the same core (*Simulink/Task Manager* essentially implements rate-monotonic scheduling automatically). For example, a 100 Hz task will interrupt a 10 Hz task if it becomes active, reflecting their priority order.
- Preemption and concurrency - Task execution is visible on the time line. If a high-priority task becomes ready while a lower priority task is running (on the same core), the lower one will pause and the high-priority task runs immediately (just as an RTOS would schedule an interrupt or higher priority thread). Meanwhile, tasks on different cores run in parallel.
- Event triggers - If we have event-driven tasks, we need to generate the events during simulation. This can be done with source blocks. For example, we might use a *Signal Generator* or a custom block to send a function-call trigger (or a message) to the *Task Manager* event input port for that task. The *Task Manager* will then schedule that task at that moment (if its core is free or by preempting a lower task). A timeline of events from a file or workspace can also be imported to simulate irregular arrival times.

5. **Observe and analyze results**

After (or during) the simulation, use the available tools to analyze scheduling behavior. The *Task Manager* block records each task execution instances.

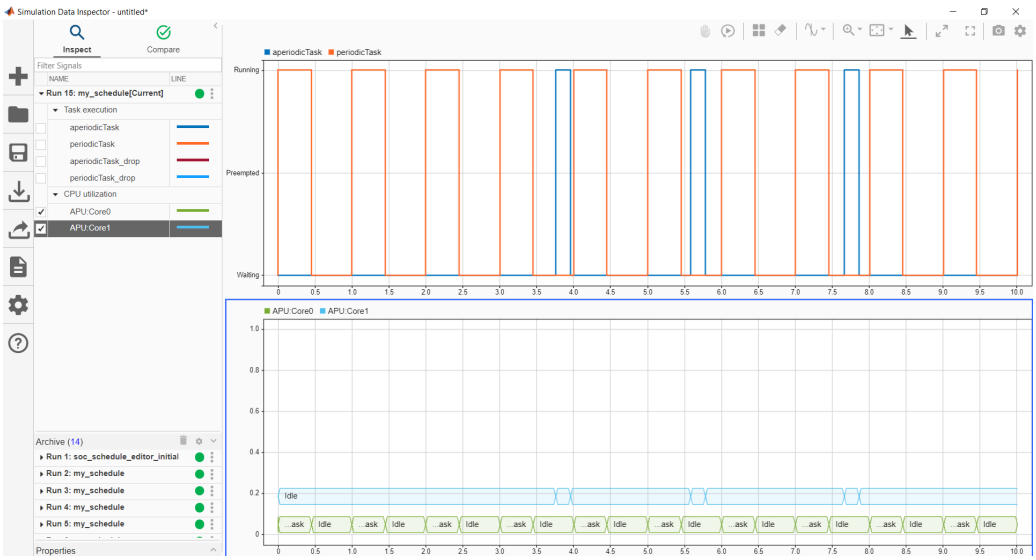

Figure 2.17. The Simulation Data Inspector tool.

We can open the *Simulation Data Inspector*, which is accessed through the *Simulation* tab, under *Review Results* (Figure 2.17), to see task timelines. Check if tasks met their deadlines (each periodic task should complete before its next period). If a task execution exceeded its period, the simulation will flag an overrun. Depending on the configuration, it might drop the next release of that task or execute it late. The system behavior must match the intended scheduling policy.

6. **Generate and access task scheduling C/C++ code**

The first step is to set up our model for code generation. To do this, go to the *Apps* tab, under *Embedded Coder*. In settings, choose *C/C++ Code generation settings*. Then, go to *Code Generation* and *System target file* (Figure 2.18). Select ert.tlc (Embedded Real-Time).

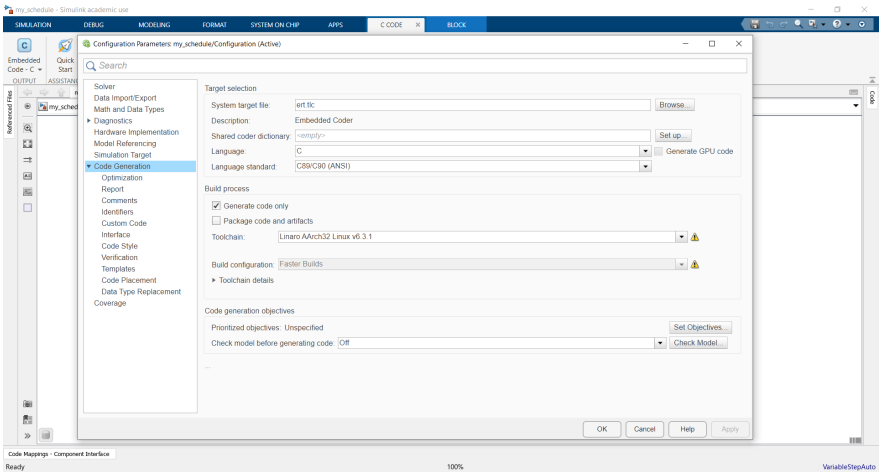

Figure 2.18. Set up model for code generation.

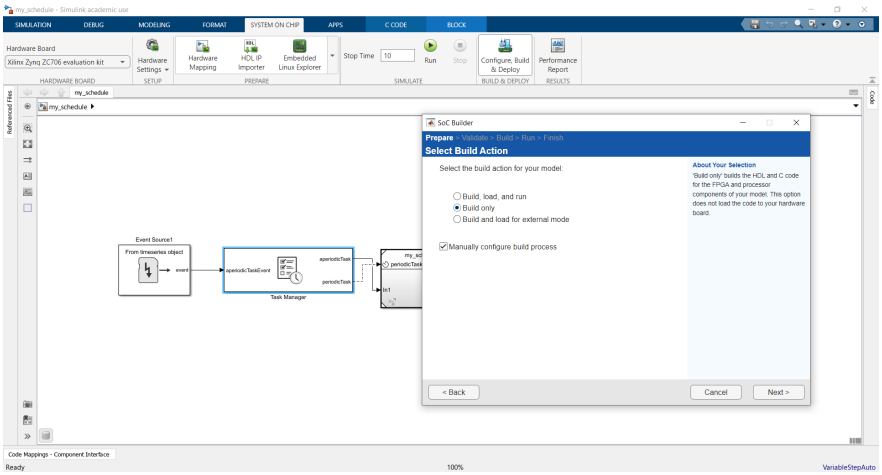

Figure 2.19. Configure Build & Deploy your model.

Next, click on *Configure Build & Deploy* in the top toolbar under *System on Chip*. Select *Build model* and *Build only* (Figure 2.19). After the build completes, our C/C++ model files are available in the generated folder. The code can also be viewed from the *C Code* tab, under *View Code*.

2.4. Scheduling tasks across multiple cores

This subchapter describes how to schedule tasks across multiple cores in a SoC Blockset model by using the Schedule Editor tool. Let us consider an example from the Simulink library, where two periodic tasks and two aperiodic tasks are scheduled:

- *periodicTask1*, which runs every 1 second;
- *periodicTask2*, which runs every 2 seconds;
- *aperiodicTask1* and *aperiodicTask2*, which run sporadically.

Task management without Schedule Editor

In order to inspect the model and its tasks run the following command (Figure 2.20): *open_system('soc_schedule_editor_initial')*.

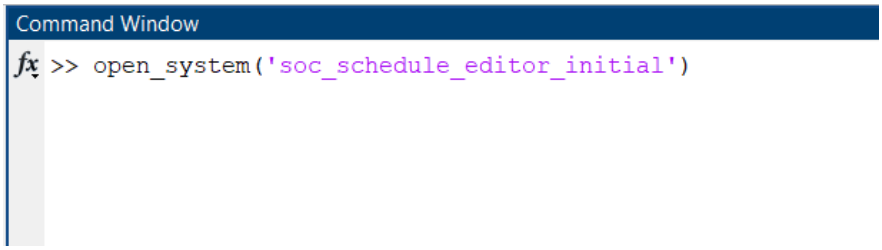

Figure 2.20. Command used to inspect the task scheduling on multiple cores example from Simulink library.

The command opens an example of a SoC application design (Figure 2.21):

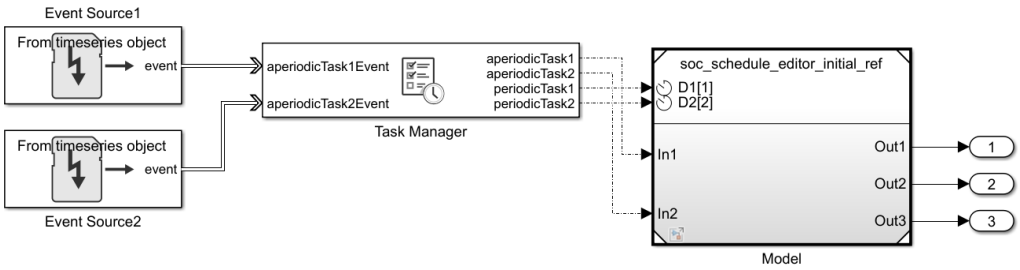

Figure 2.21. SoC application design example.

Let us assume that after the initial implementation and profiling of the task execution on the hardware board, we obtain the following task durations (Table 2.1):

Table 2.1. Task durations.

Task	period (s)	duration (s)
periodicTask1	1	0.45
periodicTask2	2	2.05
aperiodicTask1	N/A	0.35
aperiodicTask2	N/A	0.15

The second periodic task takes too long to run, overloading the processor. Therefore, an alternative implementation is required.

The second periodic task contains two independent data paths (Figure 2.22). Therefore, this task can be split into two periodic tasks, each running every 2 seconds: *periodicTask2a* and *periodicTask2b*. Profiling the tasks again produces updated task durations (Table 2.2).

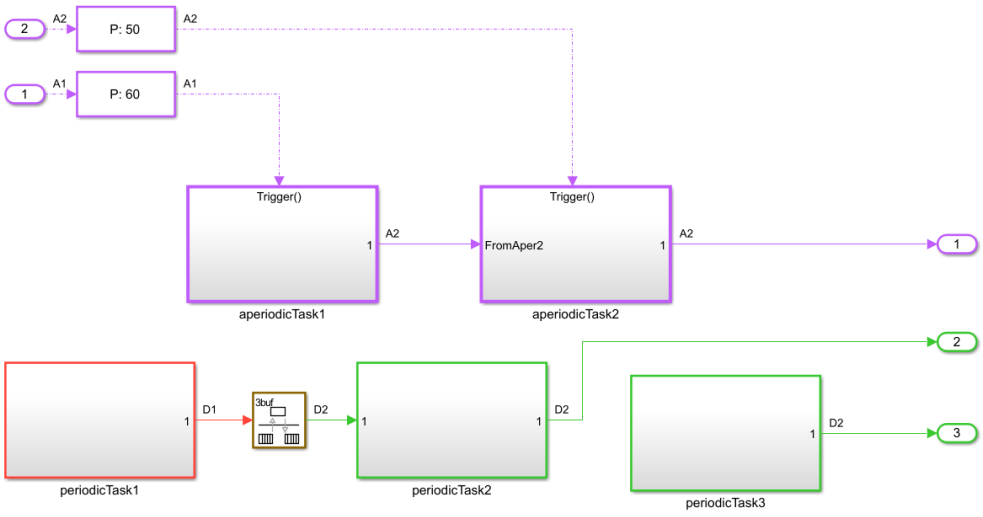

Figure 2.22. Model block structure.

Table 2.2. Updated task durations.

Task	period (s)	duration (s)
periodicTask1	1	0.45
periodicTask2a	2	0.50
periodicTask2b	2	1.55
aperiodicTask1	N/A	0.35
aperiodicTask2	N/A	0.15

Assigning *periodicTask2a* and *periodicTask2b* to separate processor cores results in a schedulable system. By default, Simulink does not allow multiple tasks for the same discrete rate. The Schedule Editor tool can be used to implement and explicitly define and manage the execution of *periodicTask2a* and *periodicTask2b*.

Task management with Schedule Editor

Task partitions can be created through the Schedule Editor (both periodic and aperiodic). Unlike the default Simulink rate grouping, multiple task partitions are able to share the same rate. We can create the two subtasks of *periodicTask2* and set them to run on separate cores. A partition is created by defining a Simulink atomic subsystem and assigning it a partition name.

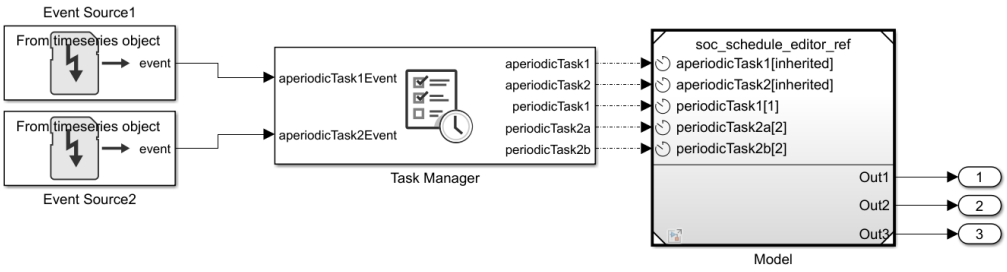

Figure 2.23. Updated SoC application design example.

In this model, two periodic partitions are configured with a period of 2, along with two aperiodic partitions (Figure 2.23). The new model can be accessed by running: *open_system('soc_schedule_editor')*.

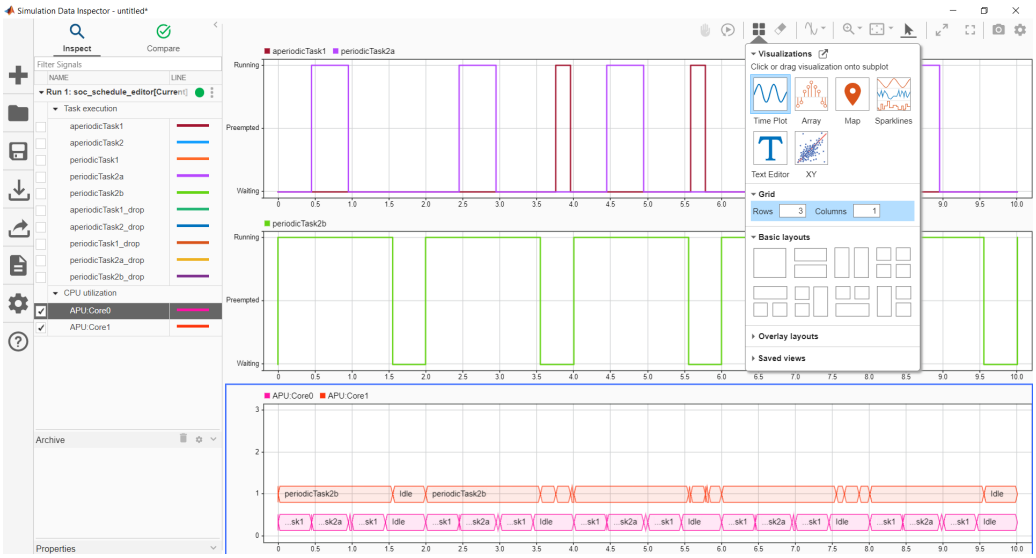

Figure 2.24. Result analysis using the Simulation Data Inspector.

Finally, we can simulate the model and analyze the results in the Data Inspector (Figure 2.24).

3. EMBEDDED PROGRAMMING

3.1. Working Environment Setup

This section will specify the setup needed for the practical applications described and proposed in this book. Having the proper setup and going through all the proposed applications is a necessary step for both understanding the theoretical concepts and for developing practical real-time programming skills.

For the practical part of this book, we will use an embedded development board, a real-time operating system, and several other software tools for tracing and debugging.

The recommended hardware consists of an STM development board, STM32F407G-DISC1 equipped with STM32F407VG processor based on ARM Cortex-M4 32-bit architecture, 1MB flash, 168 MHz CPU. It also includes ST-LINK embedded debug tool, accelerometer, microphone, LEDs, push-buttons, and a USB Micro connector.

The recommended development tools are:

- STM32CubeIDE (version 1.18.0)
- STM32CubeMX (version 6.14.0)
- Percepio Tracealyzer (version 4.10.3)

3.2. Lesson 1 - Blinky

3.2.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Checking the hardware setup
- Getting used with the IDE environment and project structure

The application requirements are:

- Implement an application for blinking an LED
- Implement and use software delays

After going through the proposed exercises the reader will be able to control the state of an LED and understand how a software delay loop works and how to influence the LED blinking frequency.

3.2.2. Theoretical Aspects

The LED is a light-emitting diode. In our case, it is connected to a general-purpose input/output (GPIO) pin. A GPIO is a signal pin, which may be used as an input, output, or both and can be controlled by software [69].

3.2.3. Hardware and Software Setup

What do you need?

- A STM32F407G-DISC1 board
  https://www.st.com/en/evaluation-tools/stm32f4discovery.html.
- A mini USB cable to connect the board
- STM32CUBE MX available for download at
  https://www.st.com/en/development-tools/stm32cubemx.html
- STM32CUBE IDE available for download at
  https://www.st.com/en/development-tools/stm32cubeide.html

For setting up the environment:

- Register on www.st.com and log in.
- Download the STM32CUBE MX from
  https://www.st.com/en/development-tools/stm32cubemx.html
  and install it on your host PC.
- Download STM32CUBE IDE from
  https://www.st.com/en/development-tools/stm32cubeide.html
  and install it on your host PC.
- Connect the STM32F407G-DISC1 board to your host PC through the mini USB cable.

3.2.4. Creating a Project

- Open STMCube IDE, and create a new STM32Cube project

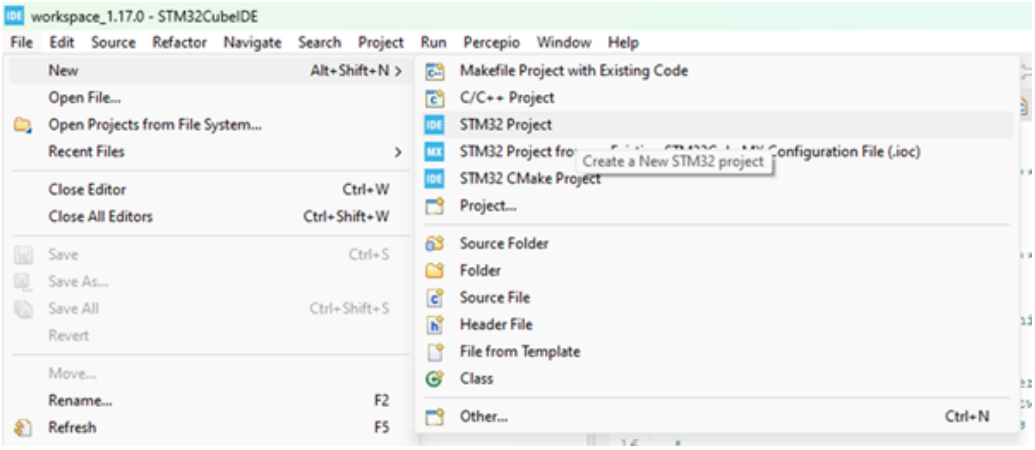

Figure 3.1. Create project

- In the Target Selection window, go to Board Selector and select the proper board

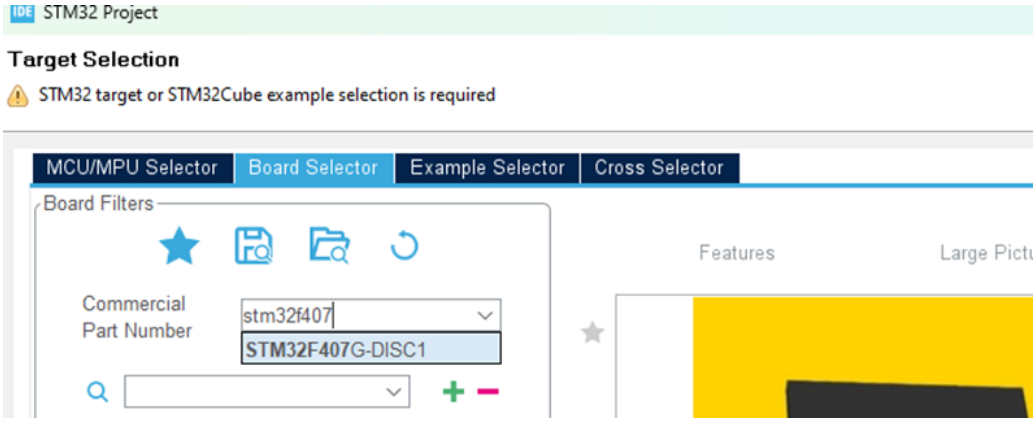

Figure 3.2. Board selector

- Select the board and click next

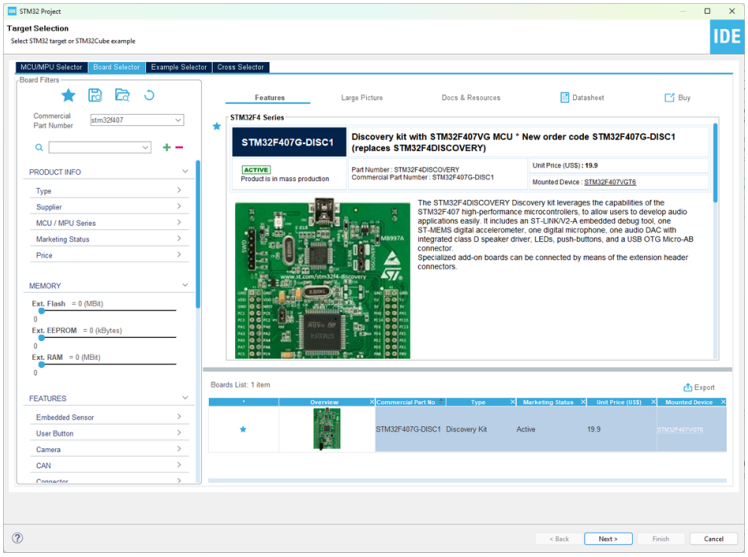

Figure 3.3. Select target

- In the pop-up window, type the project name and click Finish

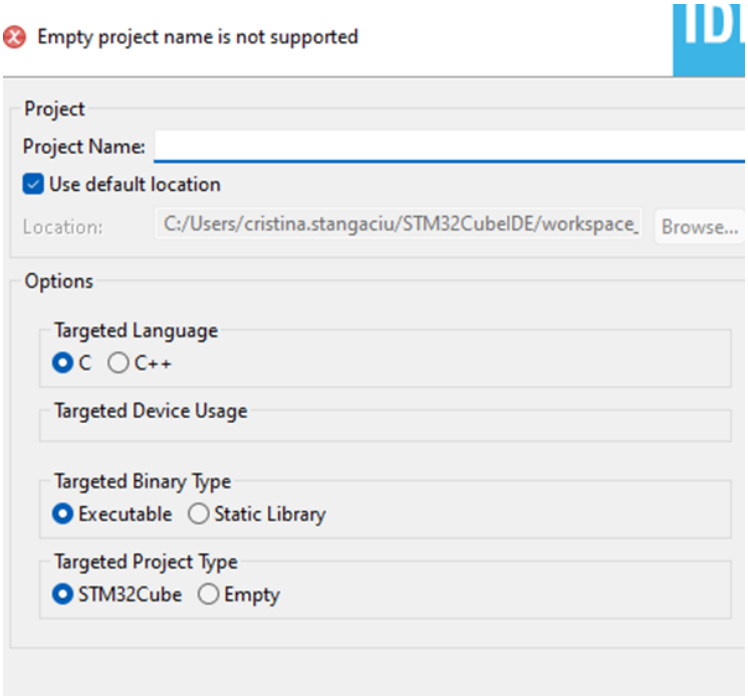

Figure 3.4. Project pop-up window

- Click yes when asked to initialize all peripherals in their default mode. For now, there is no need to make any changes in the configuration of the peripherals.

3.2.5. Implementing a Simple Application

- By opening the .ioc file, you can locate the needed GPIO ports and pins. You can also observe that the GPIO pins connected to the LEDs are already configured as output pins.

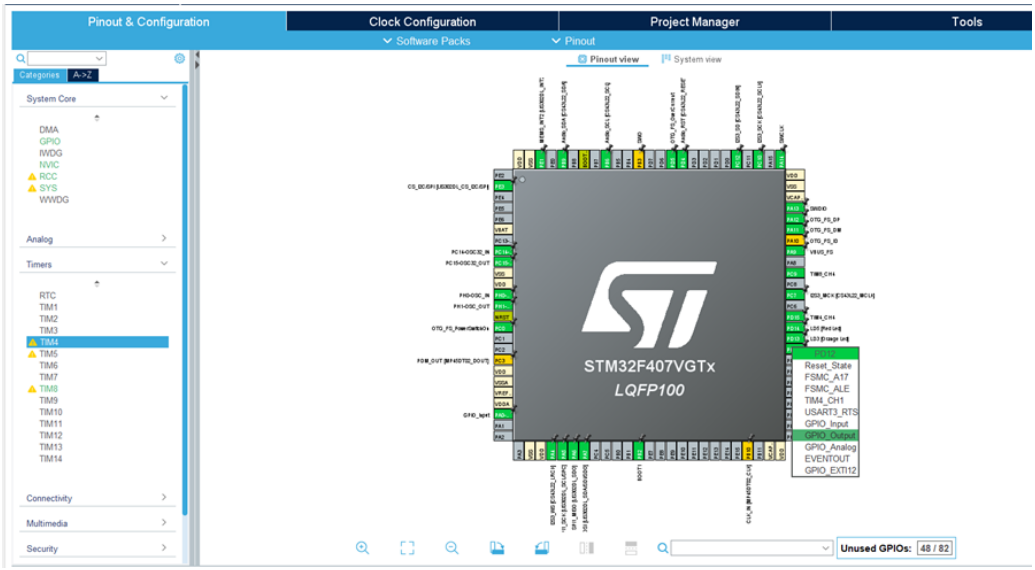

Figure 3.5. Pinout

- Locate the int main(void) function in the main.c file in the previously created project.
- STM32CubeMX can automatically generate code, thus remember to write code only between /* USER CODE BEGIN */ and /* USER CODE END */ comments, otherwise, it will be erased in the generation process by the STM32CubeMX. In the code files, you can make your own definitions for an easier location of the pins, but remember to write them in the proper location

```
1 /* Private define
     ---------------------------------------*/
2 /* USER CODE BEGIN PD */
```

```
#define GREEN_LED GPIO_PIN_12
#define BLUE_LED GPIO_PIN_15
#define RED_LED GPIO_PIN_14
#define ORANGE_LED GPIO_PIN_13
/* USER CODE END PD */
```

Listing 3.1: Defines

- Depending on the peripheral used, locate the proper functions in the STM32F4xx_HAL_Driver folder
- Write your code in the continuous loop located in the main function between /* USER CODE BEGIN */ and /* USER CODE END */

```
/* Private define
   ---------------------------------------*/
/* USER CODE BEGIN PD */
#define BLUE_LED GPIO_PIN_15
#define mainDELAY_LOOP_COUNT    ( 0xffffff )
/* USER CODE END PD */
/*...*/

/* USER CODE BEGIN 2 */
volatile uin32_t ul;
/* USER CODE END 2 */

/*...*/
while (1)
{
    /* USER CODE END WHILE */
    MX_USB_HOST_Process();

    /* USER CODE BEGIN 3 */
    HAL_GPIO_TogglePin(GPIOD, BLUE_LED);
    for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul++)
        ;
}
/* USER CODE END 3 */
}
```

Listing 3.2: Blink LED with software delay

3.2.6. Running and Debugging the Application

- Connect the board to the host PC, if you have not done this already, by using a mini USB cable
- Build the project, by right click on the project name and then -> Build Project

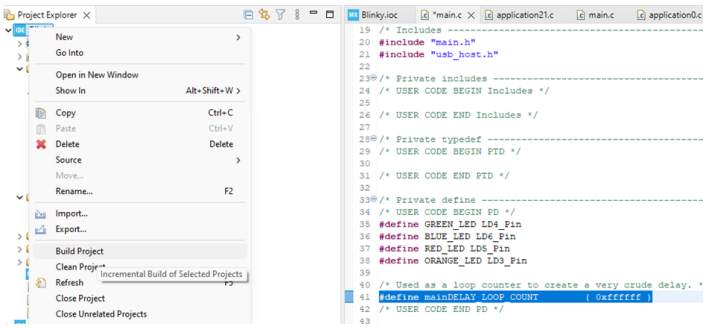

Figure 3.6. Build project

- If the build was successful (0 errors), right-click on the project and click on Run As -> 1 STM32 C/C++ Application

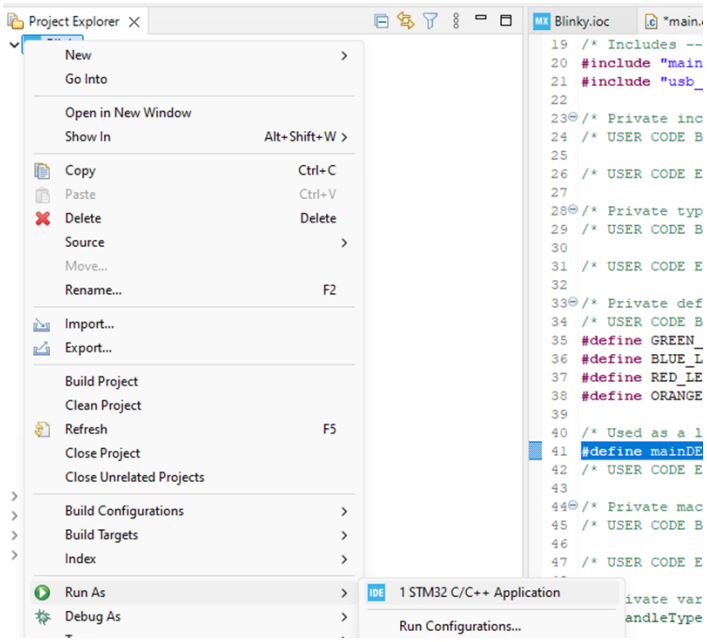

Figure 3.7. Run project

- Check that the application is running.
- You can use the debug mode by right-clicking on the project and then clicking on Debug As -> 1 STM32 C/C++ Application
- In debug mode, you can add and remove breakpoints by right-clicking at the left of the line number where you wish to pause the code execution
- In debug mode, use the proper instructions to navigate through the application

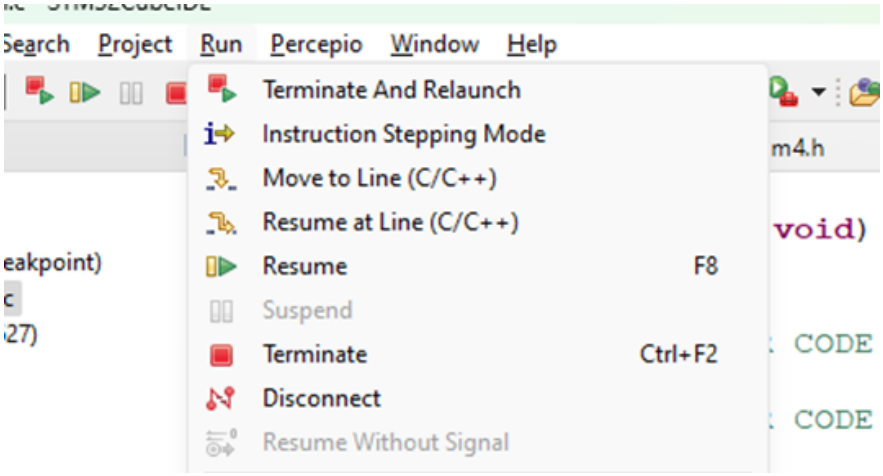

Figure 3.8. Debugging commands

3.2.7. Bibliography, Further Reading and Exercises

https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-gpio-tutorial-using-stm32cubeide/

Exercises:

- Starting from the previous application, change the program to blink the red LED.
- For better time control use void HAL_Delay(uint32_t Delay) function, from stm32f4xx_hal.h instead of the delay loop.
- Change the LED blinking frequency to 250ms.
- Blink all LEDs, one after another, with a blinking period of 250ms.

3.3. Lesson 2 - User Button

3.3.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Configuring GPIO pins
- Configuring hardware interrupts
- Getting used with the IDE environment and project structure

The application requirements are:

- Implement an application that detects the push of the user button and changes the LED status
- Use polling/interrupts

After going through the proposed exercises the reader will be able to use polling and interrupt mechanisms in order to detect events such as the press of a button.

3.3.2. Theoretical Aspects

In embedded systems, events such as the push of a button can be detected via polling mechanisms (periodically checking a GPIO pin status, for example) or by interrupts.

3.3.3. Using Polling

- Open STMCube IDE and create a new project following the steps from the first lesson
- By opening the .ioc file, you can locate the needed GPIO ports and pins. For the polling mechanism, configure the GPIO pin connected to the button as input.
- In the main.c file, include "stm32f4xx_hal_gpio.h" header file.
- In the main.c file, locate the main (void) function and poll the pin connected to the bush button [70]

```
/* Private define ------------------*/
/* USER CODE BEGIN PD */
#define BUTTON GPIO_PIN_0
/* USER CODE END PD */
/*...*/

/* USER CODE BEGIN 2 */
volatile uin32_t ul;
/* USER CODE END 2 */

/*...*/
while (1)
{
    /* USER CODE END WHILE */
    MX_USB_HOST_Process();

    /* USER CODE BEGIN 3 */
    if (HAL_GPIO_ReadPin(GPIOA, BUTTON) ==
       GPIO_PIN_SET)
        HAL_GPIO_WritePin(GPIOD, BLUE_LED,
           GPIO_PIN_SET);
    else
        HAL_GPIO_WritePin(GPIOD, BLUE_LED,
           GPIO_PIN_RESET);
    HAL_Delay(100);
}
/* USER CODE END 3 */
}
```

Listing 3.3: Polling

3.3.4. Using Interrupts

- Open STMCube IDE and create a new project following the steps from the first lesson
- Open the .ioc file and for the polling mechanism, configure the GPIO pin connected to the button as external interrupt (EXTI0)

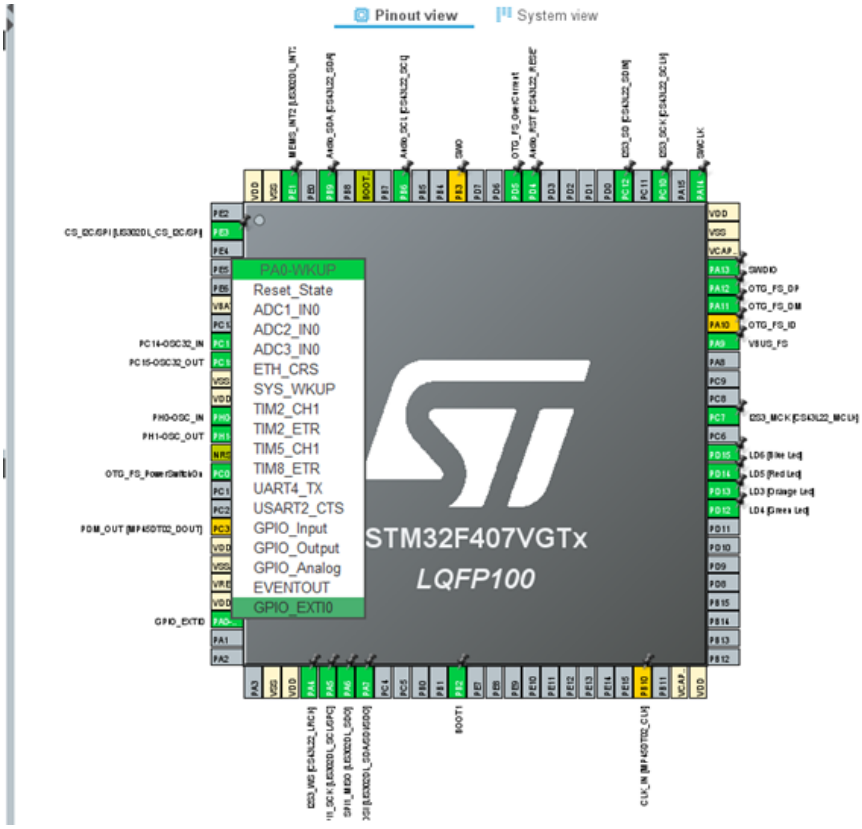

Figure 3.9. External interrupt

- Enable the interrupt for EXTI line 0, from NVIC

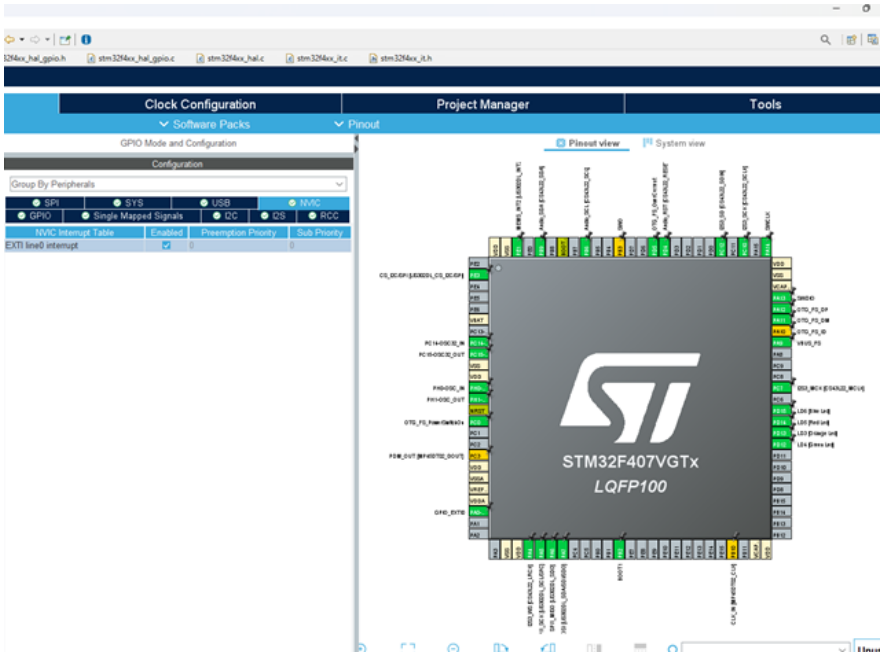

Figure 3.10. NVIC

- Generate the code and search for stm32f4xx_it.c file
- In stm32f4xx_it.c search for void EXTI0_IRQHandler(void) function and add the functionality for the interrupt handler

```
void EXTI0_IRQHandler(void)
{
        /* USER CODE BEGIN EXTI0_IRQn 0 */
        HAL_GPIO_TogglePin(GPIOD, BLUE_LED);
        /* USER CODE END EXTI0_IRQn 0 */
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
        /* USER CODE BEGIN EXTI0_IRQn 1 */
        /* USER CODE END EXTI0_IRQn 1 */
}
```

Listing 3.4: Interrupt

3.3.5. Bibliography, Further Reading and Exercises

https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-gpio-tutorial-using-stm32cubeide/#Push-button_Interfacing

Exercises

1. Starting from the previous applications, make the red LED blink when the button is pressed once and the green LED blink when the button is pressed twice.

3.4. Lesson 3 - Timers

3.4.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Getting used with hardware timers and interrupts
- Getting used with the IDE environment and project structure

The application requirements are:

- Implement an application that blinks an LED with a specific frequency using timer interrupts

After going through the proposed exercises the reader will be able to use timers for blinking LEDs with a specific frequency.

3.4.2. Theoretical Aspects

The timer is an essential peripheral in microcontrollers. It is represented by a counting register that increments/decrements each time it receives a pulse from a clock signal. It can be configured to function in different ways. The timer frequency depends on the clock, prescaler, and auto-reload parameters. It can be used to generate delays or periodic operations.

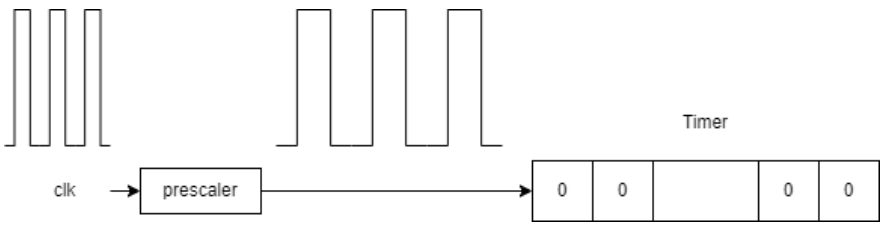

Figure 3.11. Timer

3.4.3. Configuring hardware timers

- Open STMCube IDE and create a new project following the steps from the first lesson
- Open the .ioc file and configure Timer 2 (TIM2) or another general-purpose timer.
- Configure the clock source, prescaler (PSC - prescaler counter) and counter period (ARR - auto-reload register). The ISR calls are generated at a rate that depends on the internal clock frequency, the PCS value, and the ARR value. While the internal clock frequency is fixed, the other two are configurable. The ISR frequency equals the internal clock frequency divided by (PSC * (ARR+1)). The internal clock source frequency can be seen and configured from the clock configuration tab [71].

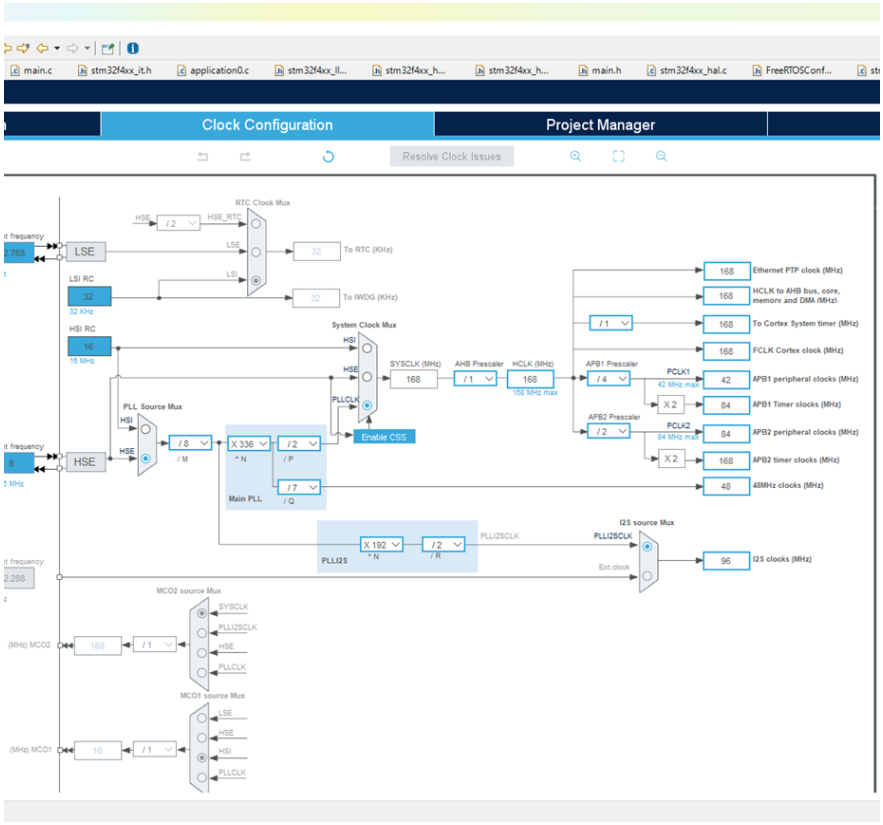

Figure 3.12. Clock configuration

- For a 1 Hz interrupt frequency, configure as following: - Clock Source: Internal Clock (for TIM2 is APB1 timer clock = 84MHz) - Prescaler: 1000 - Counter Period:84000-1 Auto-reload preload Enable

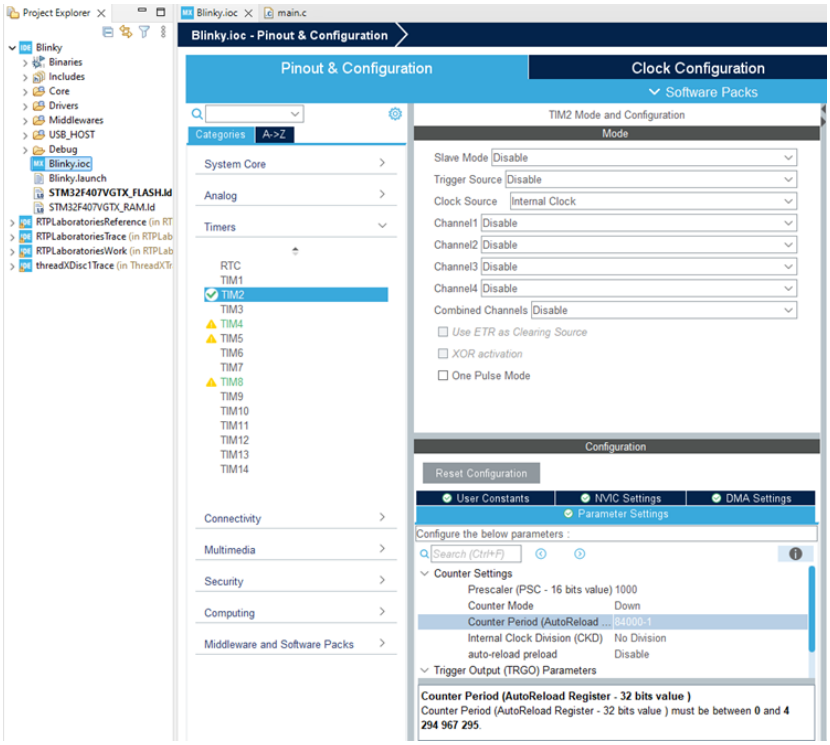

Figure 3.13. Timer configuration

- Go to NVIC Settings and enable the timer (TIM2 in this case) global interrupt

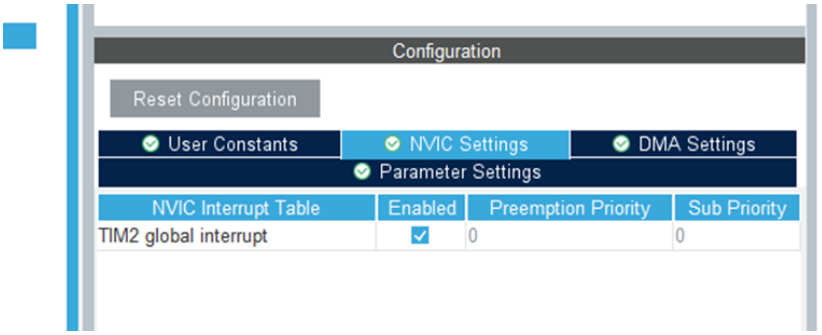

Figure 3.14. Timer interrupt

- Generate the code by saving the file agreeing, when asked to generate the code

3.4.4. Using Timer Interrupts

- Start the timer by calling HAL_TIM_Base_Start_IT(&htim2); in the main function before the continuous loop implemented by while (1), which should remain as generated.
- Write the functionality interrupt handler for Timer 2, in stm32f4xx_it.c

```
/* Private define ---------------------*/
/* USER CODE BEGIN PD */
#define BLUE_LED LD6_Pin
/* USER CODE END PD */
void TIM2_IRQHandler(void)
{       /* USER CODE BEGIN TIM2_IRQn 0 */
        HAL_GPIO_TogglePin(GPIOD, BLUE_LED);
        /* USER CODE END TIM2_IRQn 0 */
        HAL_TIM_IRQHandler(&htim2);
        /* USER CODE BEGIN TIM2_IRQn 1 */
        /* USER CODE END TIM2_IRQn 1 */}
```

Listing 3.5: Interrupt handler

- Follow the steps for running and debugging the applications presented in the first lesson.

3.4.5. Bibliography, Further Reading and Exercises

https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-timer-tutorial-using-stm32cubeide/

Exercises

1. Starting from the previous application, change the timer interrupt frequency to 100 Hz.

2. Activate four timers and blink all LEDs, each with a different frequency.

3.5. Lesson 4 - Pulse Width Modulation

3.5.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Configuring PWM pins
- Getting used with the IDE environment and project structure

The application requirements are:

- Implement an application that varies the LED brightness using PWM techniques

After going through the proposed exercises the reader will be able to use PWM technique to control the brightness of the LED.

3.5.2. Theoretical Aspects

PWM (Pulse Width Modulation) is a method for adjusting how long a rectangular signal stays at a high and a low level. It works by setting the signal frequency and the duty cycle, which determines how much time the signal spends at the high level.

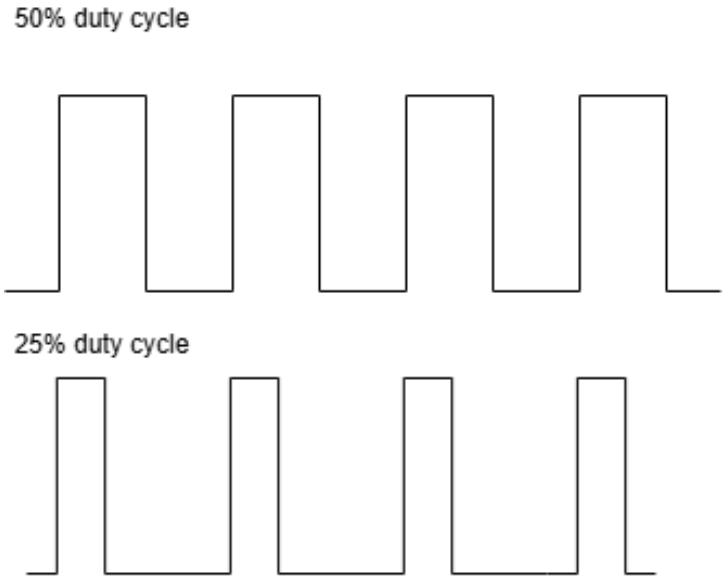

Figure 3.15. PWM

3.5.3. Configuring the PWM

- Open STMCube IDE and create a new project following the steps from the first lesson.
- Open the .ioc file and configure Timer 4 or any timer which supports PWM functionality.
- Configure the clock source as Internal Clock, Channel4 as PWM Generation CH4. The ISR calls are generated at a rate that depends on the internal clock frequency, the PCS value, and the ARR value. While the internal clock frequency is fixed, the other two are configurable. The ISR frequency equals the internal clock frequency divided by (PSC * (ARR+1)). The internal clock source frequency can be seen and configured from the clock configuration tab.
- For a 10 KHz frequency, configure the timer as following: - Clock Source: Internal Clock - Prescaler: 100 - Counter Period: 84-1 - Channel4: PWM Generation CH4
- Select the LED pin to be controlled by the selected PWM timer

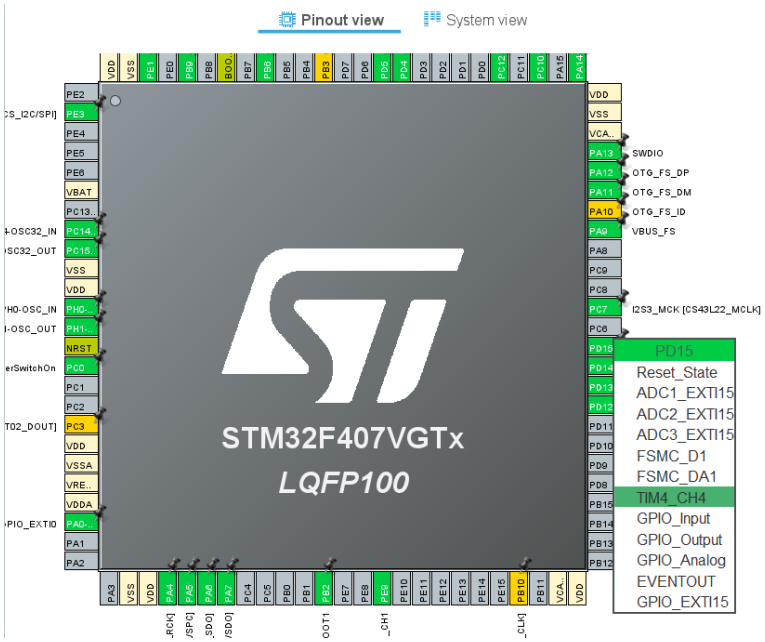

Figure 3.16. PWM controlled LED

- Generate the code by saving the file agreeing, when asked to generate the code

3.5.4. Using the PWM

- In the main.c file include "stm32f4xx_hal_tim.h"
- Start the timer by calling
  HAL_TIM_PWM_Start(&htim4,TIM_CHANNEL_4);
  and set the duty cycle by calling
  HAL_TIM_SET_COMPARE (&htim4,TIM_CHANNEL_4,50);
  where the last parameter represents the duty cycle. The function should be called in the main function before the continuous loop implemented by main loop, which should remain as generated.

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);
/* USER CODE END 2 */
while (1)
{
    /* USER CODE END WHILE */
    MX_USB_HOST_Process();

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Listing 3.6: PWM start

- For a visible variation of the LED intensity, caused by a variation of the duty cycle, change duty cycle gradually [72]

```
volatile uint8_t ul;
while (1)
{
        /* USER CODE END WHILE */
        MX_USB_HOST_Process();

        /* USER CODE BEGIN 3 */

        for (ul = 1; ul <= 100; ul++)
        {
```

```
11                  __HAL_TIM_SET_COMPARE(&htim4,
                        TIM_CHANNEL_4, ul);
12                  HAL_Delay(5);
13          }
14          for (ul = 100; ul > 0; ul--)
15          {
16                  __HAL_TIM_SET_COMPARE(&htim4,
                        TIM_CHANNEL_4, ul);
17                  HAL_Delay(5);
18          }
19
20  }
```

Listing 3.7: PWM gradient

- Follow the steps for running and debugging the applications presented in the first lesson.

3.5.5. Bibliography, Further Reading and Exercises

https://embetronicx.com/tutorials/microcontrollers/stm32/pwm-generation-in-stm32f407-using-stm32cubeide/

Exercises

1. Starting from the previous application, change the PWM frequency to 100 Hz and the duty cycle to 50
2. Starting from the previous application, use PWM timers to control the brightness for each of the four LEDs (set the red LED to 25%, the blue one to 50%, the orange one to 75%, the green one to 90%).

3.6. Lesson 5 - Serial Communication

3.6.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Using serial communication between the target board and PC
- Using serial communication for debugging
- Getting used with the IDE environment and project structure

The application requirements are:

- Implement functions for sending and receiving messages to and from the PC, via the serial interface.

After going through the proposed exercises the reader will be able to use serial communication for sending debugging messages.

3.6.2. Theoretical Aspects

Serial communication is used to interface the target board with other external devices or with the host PC. STM32F407G-DISC1 provides a USB-UART bridge to communicate with the host PC [73].

UART (Universal Asynchronous Receiver Transmit) is a fully duplex serial communication protocol. Being an asynchronous protocol, means that there is no need for an additional clock line. Instead, the hardware is set to transmit the bits at a specific frequency (baud rate). Besides the bits of data, some extra framing bits are added at the beginning and end of the transmitted packet. These bits along with the an optional parity bit are used for packet validation.

3.6.3. Configuring the UART

- Besides the mini-USB, also connect the micro-USB cable to the board
- Install a serial port terminal, such as DockLight, on your host PC.
- Open STMCube IDE and create a new project following the steps from the first lesson
- Open the .ioc file and select RCC from the System Core and make sure that the Crystal/Ceramic Resonator is selected
- In the Connectivity section select USB_OTG_FS and configure the mode as Device_Only. You can also disable Activate_VBUS. Make sure that the PA11 and PA12 pins are configured as OTG_FS_DM and OTG_FS_DP.
- In the Middleware section, select the USB_DEVICE and for the Class For FS IP, select Communication Device Class (Virtual Port Com).

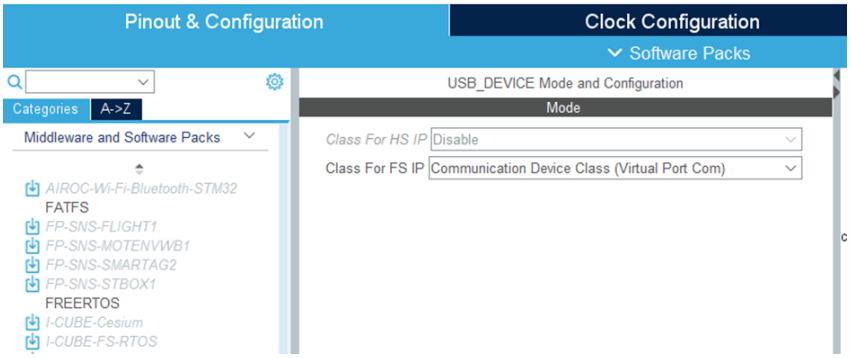

Figure 3.17. Virtual Port Com

- In the Project Manager tab, increase the Minimum Heap Size to 0x600.

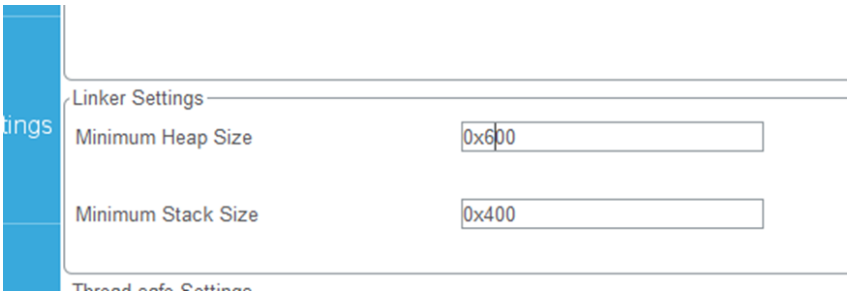

Figure 3.18. Heap Size

- Generate the code by saving the file agreeing, when asked to generate the code

3.6.4. Adding a serial communication transmitting function

- For using the serial communication and for implementing variadic functions similar with printf, include the following:

```
/* USER CODE BEGIN Includes */
#include "stm32f4xx_hal_gpio.h"
#include "usbd_cdc_if.h"
#include <stdarg.h>
#include <stdio.h>
/* USER CODE END Includes */
```

Listing 3.8: Include files

- In order to send messages to your host PC through the serial interface, implement a function similar with printf, but which uses the serial communication transmission function CDC_Transmit_FS(buf, len)

```
/* USER CODE BEGIN 0 */
#define LOG_BUFFER_SIZE 128
uint8_t buf[LOG_BUFFER_SIZE];
void vPrintSerial(const char* format, ...)
{
        va_list ap;
        va_start(ap, format);
        size_t len = vsnprintf((char*)buf, sizeof(
           buf), format, ap);
        va_end(ap);
        if (len > sizeof(buf) - 1)
                len = sizeof(buf) - 1;
        CDC_Transmit_FS(buf, len);
        /* Send string to serial interface */
}
/* USER CODE END 0 */
```

Listing 3.9: Include files

- In the main.c file, in the main() function, implement the infinite loop and call vPrintSerial to send messages to your host PC

```
/* USER CODE BEGIN WHILE */
uint32_t cnt = 0;
/* cnt is just a counter */
while (1)
{
    vPrintSerial("Iteration␣%d\n", ++cnt);
    HAL_Delay(1000);
}
/* USER CODE END WHILE */
```

Listing 3.10: Send message

3.6.5. Adding a serial communication receiving function

- In your project, locate the usbd_cdc_if.c file (USB_Device->App) and find the CDC_Receive_FS function and modify the serial communication reception function CDC_Receive_FS

```
1  static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t
       * Len)
2  {
3          /* USER CODE BEGIN 6 */
4          USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf
               [0]);
5          USBD_CDC_ReceivePacket(&hUsbDeviceFS);
6          if ((Buf[0]=='O') || (Buf[0]=='o'))
7          {       /* Blink the orange LED */
8                  HAL_GPIO_TogglePin(GPIOD,
                       GPIO_PIN_13);
9          }
10         else if ((Buf[0]=='G')||(Buf[0]=='g'))
11         {       /* Blink the green LED */
12                 HAL_GPIO_TogglePin(GPIOD,
                       GPIO_PIN_12);
13         }
14         else if ((Buf[0]=='R')||(Buf[0]=='r'))
15         {       /* Blink the red LED */
16                 HAL_GPIO_TogglePin(GPIOD,
                       GPIO_PIN_14);
17         }
18         else if ((Buf[0]=='B')||(Buf[0]=='b'))
19         {       /* Blink the blue LED */
20                 HAL_GPIO_TogglePin(GPIOD,
                       GPIO_PIN_15);
21         }
22         else
23         {
24                 CDC_Transmit_FS((unsigned char*)"
                       WRONG␣Option!\n", 14);
25         }
26         return (USBD_OK);
27         /* USER CODE END 6 */
28 }
```

Listing 3.11: Reception function

- Follow the steps for running and debugging the applications presented in the first lesson.

3.6.6. Using the serial communication between the target and PC

- Identify the virtual com device in Device Manager

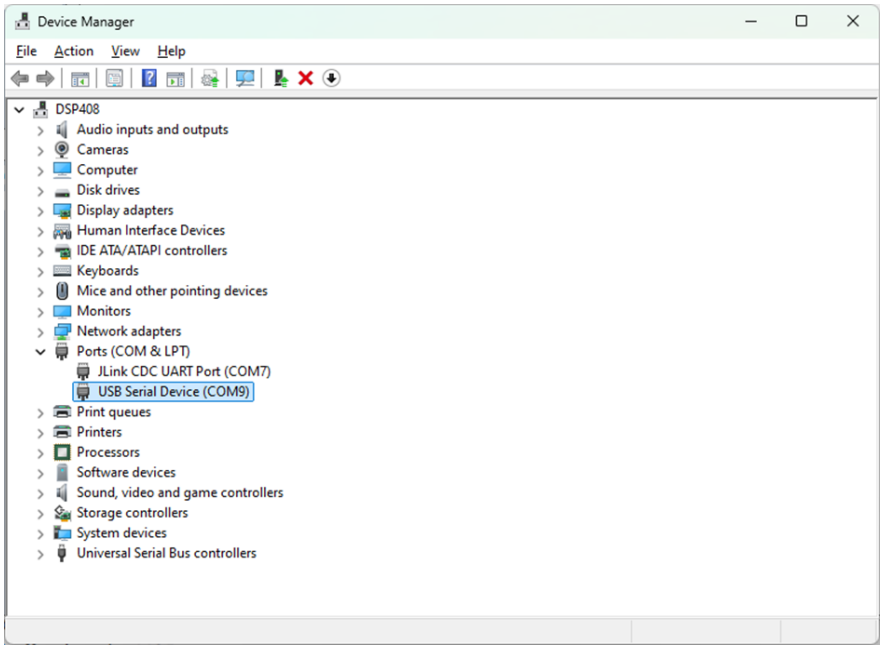

Figure 3.19. Device Manager

- In the serial port terminal (e.g. Docklight), configure the communication by double-clicking on the COM in the upper right corner
- Select the proper COM number, and set the highest Baud rate: 230400, Data bits 8, Parity None, Stop bits 1.
- Send and receive messages through the port terminal, by clicking on Start communication (F5) in Docklight.

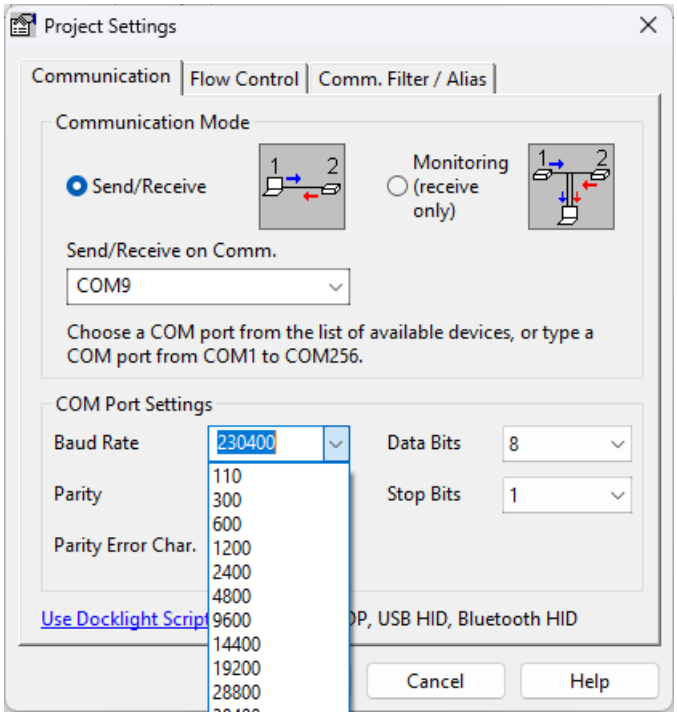

Figure 3.20. Docklight

- Type the commands and watch the received messages in the Communication tab.

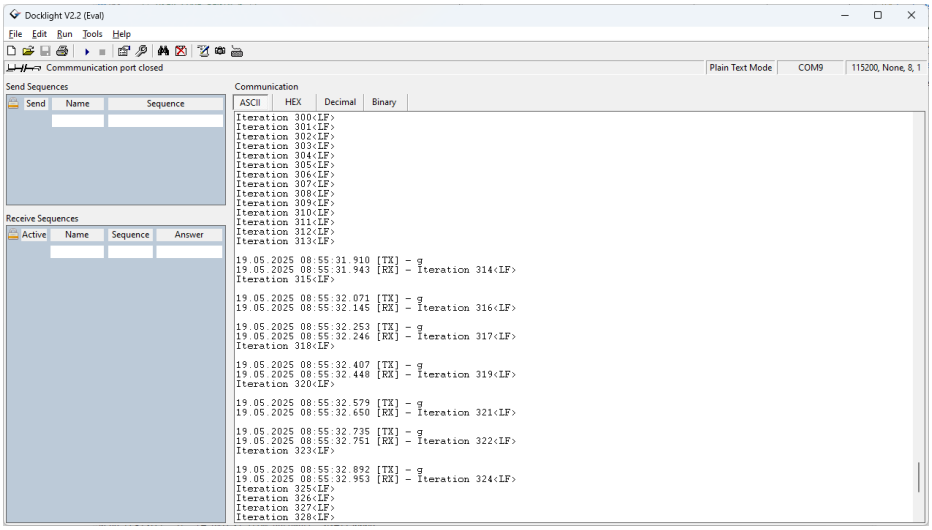

Figure 3.21. Docklight Communication

3.6.7. Bibliography, Further Reading and Exercises

https://willfrank.co.uk/stm32-vcp.html
Exercises

1. Blink an LED by using the HAL_Delay() function and control its blinking frequencies through commands sent via the serial interface.
2. Use the previously defined vPrintSerial function for debugging the previous application by sending back the received command.

4. REAL-TIME PROGRAMMING IN FREERTOS

4.1. Working Environment Setup

The setup for the next lessons is similar with the one proposed in the previous chapter.

It consists of the same STM32F407G-Disc1 development board and the recommended development tools are:

- STM32CubeIDE (version 1.18.0)
- STM32CubeMX (version 6.14.0)
- Percepio Tracealyzer (version 4.10.3)

Additionally, for the following lessons we will also use a real-time operating system. The one used in this book is FreeRTOS (Free Real-Time Operating System). FreeRTOS is an open-source real-time operating system kernel designed specifically for embedded systems, supporting a wide range of microcontrollers, including ARM Cortex architectures. It is a priority-based multithreading environment that offers features such as task management, inter-task communication, synchronization mechanisms, and software timers through dedicated API functions, which also include synchronization mechanisms like semaphores, mutexes, and queues for task communication and synchronization. One of the reasons for choosing this operating system, besides being open-source, is its small footprint, which makes it suitable for resource-constrained embedded systems. Additionally, it is well-documented and actively maintained.

4.2. Lesson 1 - Task Creation

4.2.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Getting familiar with the FreeRTOS project structure
- Getting familiar with the FreeRTOS API
- Creating a task in FreeRTOS using xTaskCreate function

The application requirements are:

- Create a task that blinks an LED using FreeRTOS APIs. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to create tasks in FreeRTOS and be introduced in multi-threading programming.

4.2.2. Theoretical Aspects

In real-time systems, applications are composed of tasks that represent basic units of execution. They are generated in response to some events, that may be either internal or external to the system and usually recur a large number of times, at different instances of time, competing for processor execution time and sometimes for accessing other resources. Real time tasks have different timing constraints which are usually given by their interaction with the environment.

Even if not all embedded systems are designed with an operating system, some of them require as RTOS (Real-Time Operating System) for task scheduling, resource management and other real-time services.

An example of such a system is FreeRTOS, an open-source operating system developed for embedded platforms. It offers an API which includes functions for task creation and control, software timers creation and control, usage of semaphores, mutextes, queues and other kernel objects, etc.

In FreeRTOS each task executes within its own context and has its own stack. The scheduler is responsible for deciding which task should be executed on the processor at any time reference point which is given by the system time reference called SysTick [74].

4.2.3. How to setup a FreeRTOS project on STM32F470G-DISC1 Board using STM32Cube MX

- Open STMCube IDE and create a new project following the steps from the first lesson
- Open the .ioc file and make the following configurations
- Choose a timing source for the SysTick (e.g. TIM1)

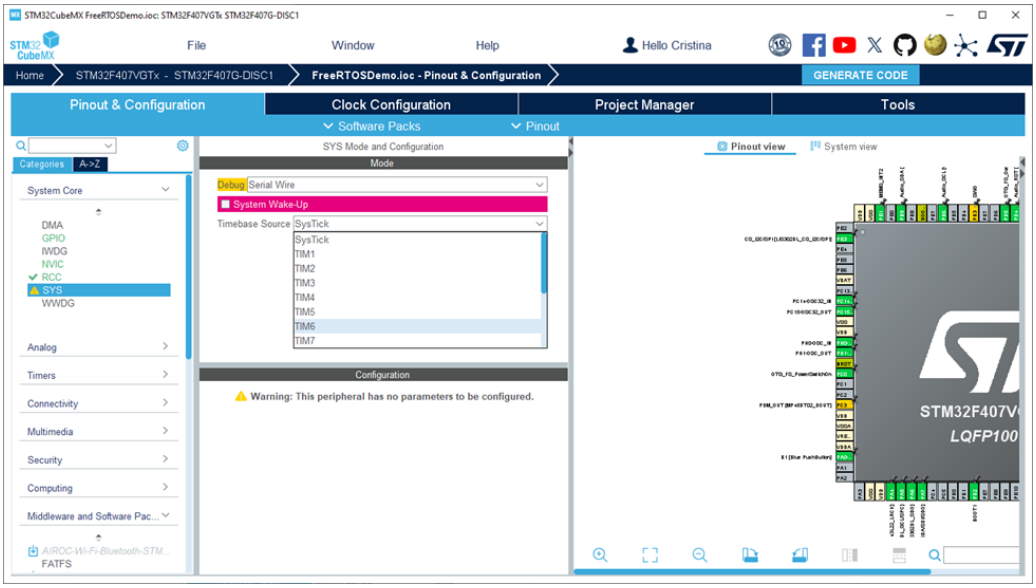

Figure 4.1. Clock source

- Enable FreeRTOS by choosing one of the CMSIS versions of the interface. Even if we will not use the CMSIS API (e.g. CMSIS_V2), for automated code generation from STM32Cube MX, it is needed.
- Enable USE_NEWLIP_REENTRANT from advanced settings

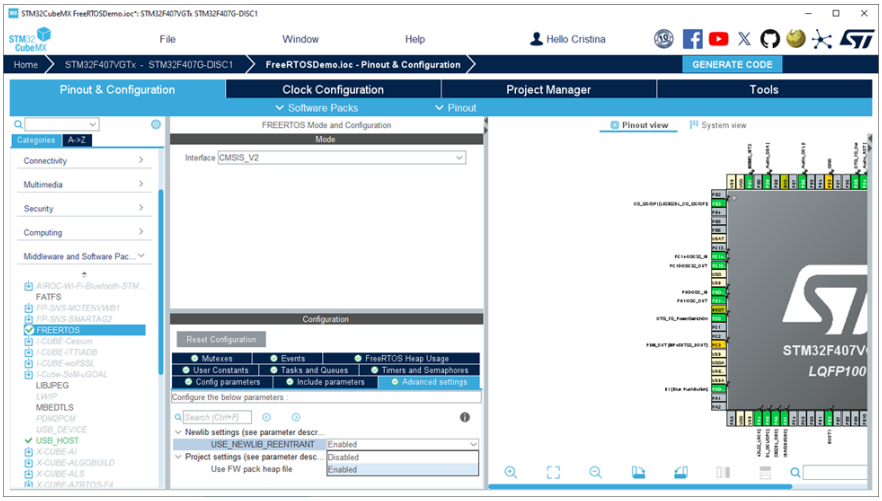

Figure 4.2. CMSIS version

- Go to Project Manager tab and choose a name and a path for your project and configure the project
- Choose STM32CubeIDE as Toolchain/IDE

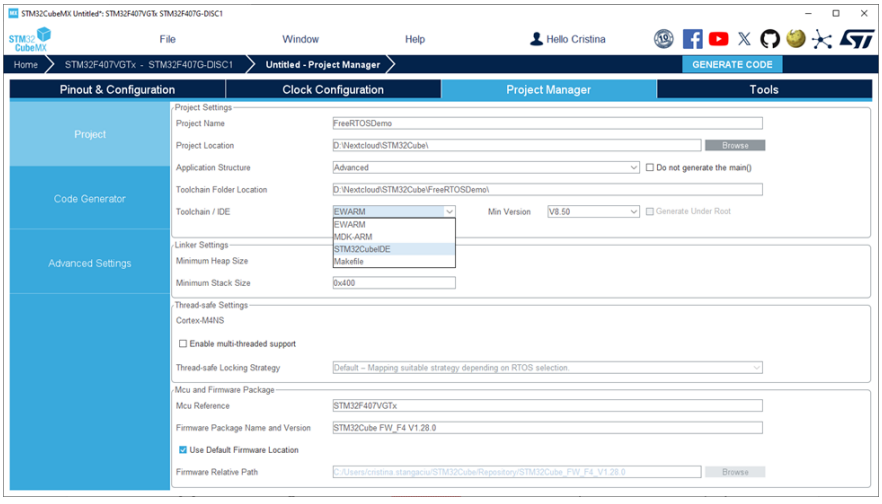

Figure 4.3. FreeRTOS project manager

- Generate code by saving the file or by clicking on Generate Code

4.2.4. Starting to Program in FreeRTOS

Introducing FreeRTOS project structure, data types and coding style

- The main FreeRTOS source files can be found in Core->Src, Middleware-> Third_Party-> FreeRTOS->Source and the header files in Core->Inc, Middleware-> Third_Party-> FreeRTOS->Include
- One of the most important file is FreeRTOSConfig.h found in Core->Inc
- FreeRTOS configures and uses a periodical timer interrupt, called SysTick. It counts the number of the tick interrupts that have occurred since system activation.
  Its rate is defined as configTICK_RATE_HZ in FreeRTOSConfig.h
- TickType_t is the data type used to hold the tick count variable and to specify times and its actual size is architecture-dependent. BaseType_t is also architecture-dependent and generally it is used for return types with a limited range of values.
- Functions are prefixed firstly with a few letters representing the type they return and then with the name of the file they are defined within. For xxxFileNameFunctionName(), xxx represents the return type and filename.c is the file that defines the function. Some common letters are v for void, x for BaseType_t, and pv for void pointer. E.g. xTaskCreate() -> x represents BaseType_t, and Task shows that this function is defined in tasks.c

Creating a task in FreeRTOS

- Make your own private defines if needed
- Create a task function. The task function should respect the following structure [75]:

```
1 void Name(void* pvParameters)
2 {
3         /* Declaration of local variables */
4         for (;;)
```

```
5           {
6           /* Implementation of the functionality */
7           }
8   }
```

Listing 4.1: Task function structure

- Use the following example, which toggles the green LED using a software delay.

```
1   void Led_Test(void* pvParameters)
2   {
3        volatile uint32_t ul;
4        for (;;)
5        {
6           /* Toggle green led. */
7           HAL_GPIO_TogglePin(GPIOD, GREEN_LED);
8           /* Delay for a period. */
9           for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul
               ++)
10          {
11                  ;
12          /* A software delay implemantation */
13          }
14       }
15  }
```

Listing 4.2: Task function structure

- Create a task using the FreeRTOS API function xTaskCreate
- Replace in the main function default TaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes); CMSIS API call with xTaskCreate(Led_Test, "led_Green", configMINIMAL_STACK_SIZE, NULL, 1, NULL); in our example.
- Check if the task creation was successful and start the scheduler by using the vTaskStartScheduler() API [76], and remove the CMSIS call osKernelStart() which was generated automatically.
- Follow the steps for running and debugging the applications presented in the first lesson.

4.2.5. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/04-RTOS-kernel-control/03-vTaskStartScheduler

Exercises

1. Change the code of the LED_Test by choosing different values for the loop limit used for the software delay. Observe how the LED blinking frequency changes.
2. Check the processor pinout for the STM32F407G-DISC1 board; note the pin for the blue LED. Make changes in the LED_Test function so that the blue LED blinks instead of the green one. Repeat the previous steps for the orange and red LEDs.

4.3. Lesson 2 - Task Parameters

4.3.1. Objectives and Application Requirements

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Creating tasks and assigning their parameters in FreeRTOS using xTaskCreate function

The application requirements are:

- Create several tasks that blink LEDs using FreeRTOS API. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to create tasks in FreeRTOS and assign task parameters.

4.3.2. Theoretical Aspects

In FreeRTOS each task has associated a pointer to a function that implements its functionality, a name, a stack size, a period, and a handle. In FreeRTOS the function that implements a task has a fixed header form:

void function_name (void * pvParameters). Parameters can be passed to the functions at task creation [74].

4.3.3. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that preemption is set and dynamic allocation is available
- Check the configMAX_PRIORITIES and configTOTAL_HEAP_SIZE values

4.3.4. Tasks in FreeRTOS

Creating tasks and assigning their parameters in FreeRTOS

- Make your own private defines if needed. Do not forget to put code only between "USER CODE BEGIN" and "USER CODE END" markers.
- Create two functions, each one for another task, as in the examples below:

```
1  void vTask1(void* pvParameters)
2  {
3
4      volatile uint32_t ul;
5      for (;;)
6      {
7          /* Toggle green led. */
8          HAL_GPIO_TogglePin(GPIOD, GREEN_LED);
9          /* Delay for a period. */
10         for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul
               ++)
11         {
12                 ; /* Software dealy */
13         }
14     }
15 }
16 /*-----------------------------------*/
```

```
17 void vTask2(void* pvParameters)
18 {
19      volatile uint32_t ul;
20      for (;;)
21      {
22         /* Toggle blue led. */
23         HAL_GPIO_TogglePin(GPIOD, BLUE_LED);
24         /* Delay for a period. */
25         for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul
              ++)
26         {
27                 ; /* Software dealy */
28         }
29      }
30 }
```

Listing 4.3: FreeRTOS task functions

- Create two tasks using the FreeRTOS API function xTaskCreate by replacing in the main function, the following call function: defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

```
1  uint16_t  Task1LED = GREEN_LED;
2  uint16_t  Task2LED = BLUE_LED;
3
4  BaseType_t xReturned1, xReturned2;
5  /* Create one of the two tasks. */
6  xReturned1 = xTaskCreate(vTask1, /* Pointer to the
      function that implements the task. */
7     "Task␣1",  /* Text name for the task.*/
8     128,       /* Stack depth*/
9     NULL,      /* We are not using the task
         parameter. */
10    1,         /* Priority 1. */
11    NULL);     /* We are not using the task handle.
         */
12
13 /* Create the other task in a similar way. */
14 xReturned2 = xTaskCreate(vTask2, "Task␣2", 128,
      NULL, 1, NULL);
15 If the task creation was successful, start the
```

```
   scheduler by using the vTaskStartScheduler() API
   , and remove the CMSIS call osKernelStart().
if ((xReturned1==pdPASS) && (xReturned2==pdPASS))
vTaskStartScheduler();
```

Listing 4.4: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.

Creating multiple tasks using the same function to implement their functionality in FreeRTOS

- Make your own private defines if needed, as in the case of the previous application.
- Create a task function, as in the examples below:

```
void vTaskFunction(void* pvParameters)
{
        uint16_t* LED;
        volatile uint32_t ul;

        LED = (uint16_t*)pvParameters;
        for (;; )
        {
                /* toggle the right LED. */
                HAL_GPIO_TogglePin(GPIOD, *LED);
                /* Delay for a period. */
                for (ul = 0; ul <
                   mainDELAY_LOOP_COUNT; ul++)
                {
                        ; /* Software dealy */
                }
        }
}
```

Listing 4.5: FreeRTOS task functions

- Create two tasks using the FreeRTOS API function xTaskCreate by replacing in the main function, the following call function: defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

- If the task creation was successful, start the scheduler by using the vTaskStartScheduler() API, and remove the CMSIS call osKernelStart().

```
uint16_t  Task1LED = GREEN_LED;
uint16_t  Task2LED = BLUE_LED;

BaseType_t xReturned1, xReturned2;
/* Create one of the two tasks. */
xReturned1 = xTaskCreate(vTaskFunction, "Task␣1",
   128, (void*)&Task1LED, 1, NULL);

/* Create the other task in a similar way. We are
   using the SAME task function, but passing in a
   different parameter.  */
xReturned2 = xTaskCreate(vTaskFunction, "Task␣2",
   128, (void*)&Task2LED, 1, NULL);
if ((xReturned1==pdPASS)&&(xReturned2==pdPASS))
vTaskStartScheduler();
```

Listing 4.6: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.

4.3.5. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/04-RTOS-kernel-control/03-vTaskStartScheduler

Exercises

- Starting from the previous application, increase the stack size for the tasks and create four tasks, each one blinking another LED. Empirically, determine the maximum stack size in this case (without making changes regarding the configTOTAL_HEAP_SIZE in the FreeRTOSConfig.h).
- Create four tasks, each one blinking another LED, using the same function to implement their functionality.
- Change the priority of the second task to a higher value. Observe the

behavior.

4.4. Lesson 3 - Periodic Tasks and Delay Functions

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Implementing periodical tasks and using vTaskDelay and vTaskDelayUntil functions

The application requirements are:

- Create several tasks that blink LEDs using FreeRTOS API and delay functions. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to implement periodic tasks in FreeRTOS.

4.4.1. Theoretical Aspects

According to Liu and Layland's model [22], for a task τ_i, we have the following time specifications:

$$\tau_i = C_i; T_i; D_i$$

, where parameter C_i is the computation time, T_i the period, and D_i the deadline for the task τ_i. An implicit deadline is considered to have the same value as the period, thus, a simplified model for tasks with implicit deadlines contains only the first two parameters:

$$\tau_i = C_i; T_i$$

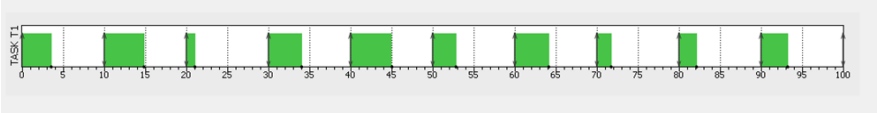

Figure 4.4. Periodic Task

While the computation time is dependent on the code instructions,

the period can be directly controlled using non-blocking delay functions (vTaskDelay and vTaskDelaUntil).

The difference between vTaskDelay and vTaskDelayUntil, is that vTaskDelay parameter specifies for how long the task should be suspended from the moment when the function is called, and vTaskDelayUntil last parameter specifies the time for how long the task should be suspended relative to a time moment given as a reference [74].

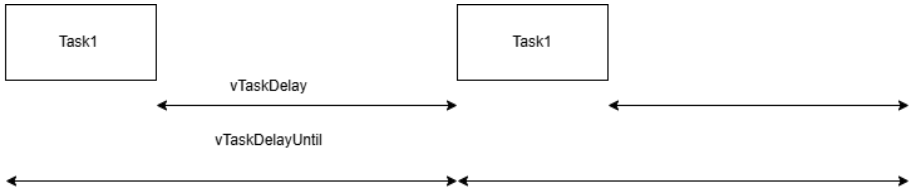

Figure 4.5. vTaskDelay vs vTaskDelayUntil

4.4.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that preemption is set and that the vTaskDelay and vTaskDelayUntil functions are included
- Check the value of configTICK_RATE_HZ

4.4.3. Using vTaskDelay function in FreeRTOS

- Make your own defines and create a function implementing the tasks' functionalities by writing the following code before the main function

```
1 uint16_t  Task1LED = GREEN_LED;
2 uint16_t  Task2LED = BLUE_LED;
3 BaseType_t xReturned1, xReturned2;
4 /* Create one of the two tasks. */
5 xReturned1 = xTaskCreate(vTaskFunction, "Task␣1",
     128, (void*)&Task1LED, 1, NULL);

6 xReturned2 = xTaskCreate(vTaskFunction, "Task␣2",
     128, (void*)&Task2LED, 2, NULL);
```

Listing 4.7: vTaskDelay example

Observation: FreeRTOS API calls specify the time in multiple of system tick periods. The pdMS_TO_TICKS() macro can be used to convert the time specified in ms into system ticks.

- Create two tasks, with different priorities, using the FreeRTOS API function xTaskCreate by replacing in the main function default TaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes); instruction

  If the task creation was successful, start the scheduler by using the vTaskStartScheduler() API, and remove the CMSIS call osKernelStart().

- Follow the steps for running and debugging the applications presented in the first lesson.

4.4.4. Using vTaskDelayUntil function in FreeRTOS

- Change the previous task function from the previous example, to:

```
1  void vTaskFunction(void* pvParameters)
2  {
3          uint16_t* LED;
4          TickType_t xLastWakeTime;
5          const TickType_t xDelay250ms =
              pdMS_TO_TICKS(250UL);
6          LED = (uint16_t*)pvParameters;
7          /* The xLastWakeTime variable retains the
              current tick count.  xLastWakeTime is
              given as a parameter to the
              vTaskDelayUntil task function */
8          xLastWakeTime = xTaskGetTickCount();
9          for (;;)
10         {
11              /* toggle the right LED. */
12              HAL_GPIO_TogglePin(GPIOD, *LED);
13              /* We want this task to execute
                   exactly every 250 milliseconds.*/
14              vTaskDelayUntil(&xLastWakeTime,
                   xDelay250ms);
15         }
16 }
```

Listing 4.8: vTaskDelayUntil example

- Follow the steps for running and debugging the applications presented in the first lesson.

4.4.5. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/04-RTOS-kernel-control/03-vTaskStartScheduler
https://www.freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/01-vTaskDelay
https://www.freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/02-vTaskDelayUntil

Exercises

- Change the lengths of the delay and observe the influence on the LED blinking frequency
- Change the computation time of the task by adding software delays (loops, like in the first lessons) before the delay function call, and observe the differences between vTaskDelay and vTaskDelayUntil function

4.5. Lesson 4 - Task Scheduling

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Using priority-based scheduling in FreeRTOS, assigning and changing task priorities

The application requirements are:

- Create two continuous and a periodical tasks, each one blinking a different LED, using FreeRTOS APIs. Observe the influence of the tasks' priorities on the application's behavior.
- Create two continuous tasks with different priorities (Task1 with high priority and Task2 with low priority). Increase the priority of Task2 from Task1's function to be higher than the priority of Task1. From Task2, decrease the priority back to the original value. Observe how

both tasks are running, due to priority change. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to properly assign priorities to tasks based on their time parameters and to change priorities during task execution.

4.5.1. Theoretical Aspects

Task scheduling determines the order in which tasks are to be executed by the system. By default, FreeRTOS uses fixed-priority preemptive execution, with round-robing for equal-priority tasks. A preemptive scheduler checks for task activation and suspends all the lower-priority tasks from execution if a higher-priority task is activated, thus always executing the highest-priority task that is ready. A non-preemptive scheduler, once it dispatches a task, will continue its execution until its execution finishes. Thus, each task executes without being preempted by any other tasks. In a fixed-priority scheduling algorithm, the priority of the task does not change during the system operation, and all the instances of a given task have the same priority. Round-robin algorithm shares processing time between ready tasks, giving each task a specific time slice [74].

Below is an example of a preemptive priority-based scheduling for three tasks, which have their priority assigned based on their activation rate (Task T1 having the highest priority).

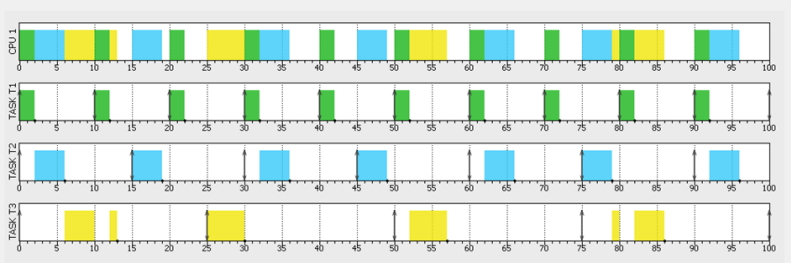

Figure 4.6. Rate Monotonic

If the priorities are assigned in the reverse order of their activation rate, the scheduling becomes unfeasible (i.e. not all deadlines are met as can be seen at time moment 10).

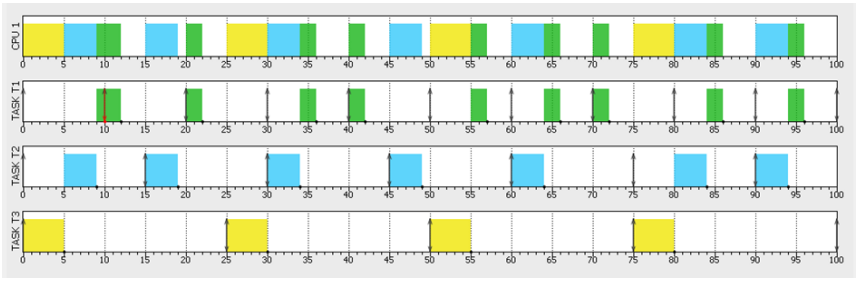

Figure 4.7. Unfeasible scheduling

The method of assigning priorities based on their activation rate is called Rate Monotonic (RM) and it was introduced by Liu and Layland [22]. The authors have also introduced a schedulability test based on a sufficiency condition: for a set of n periodic tasks:

$$U_{tot} \leq n(2^{\frac{1}{n}} - 1)$$

where U_{tot} is the total utilization:

$$U_{tot} = \sum \frac{C_i}{T_i}$$

where C_i is the computation time of task i and T_i is the period of the task. It means that as long as the total task utilization is below the threshold, the task set is schedulable. Thus properly assigning priorities for scheduling a set of tasks implies to determine the total utilization, check the feasibility tests and properly assign priorities to each task (e.g. based on their activation rate).

4.5.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that preemption is set and that the vTaskDelay and vTaskDelayUntil functions are included
- Check the value of configTICK_RATE_HZ

4.5.3. Assigning task priorities in FreeRTOS

- Make your own defines and create continuous and periodic functions implementing the tasks' functionalities by writing the following code before the main function

```
/* Private define ------------------------------*/
/* USER CODE BEGIN PD */
#define mainDELAY_LOOP_COUNT        ( 0xffffff )
#define GREEN_LED GPIO_PIN_12
#define BLUE_LED GPIO_PIN_15
#define RED_LED GPIO_PIN_14
#define ORANGE_LED GPIO_PIN_13
/* USER CODE END PD */
void vContinuousProcessingTask(void* pvParameters)
{
        uint16_t* LED;
        LED = (uint16_t*)pvParameters;
        for (;; )
        {
                /* toggle the right LED. */
                HAL_GPIO_TogglePin(GPIOD, *LED);
        }
}
void vPeriodicTask(void* pvParameters)
{
        uint16_t* LED;
        LED = (uint16_t*)pvParameters;
        TickType_t xLastWakeTime;
        const TickType_t xDelay250ms =
           pdMS_TO_TICKS(250UL);
        xLastWakeTime = xTaskGetTickCount();
        for (;; )
        {
                /* toggle the right LED. */
                HAL_GPIO_TogglePin(GPIOD, *LED);
                vTaskDelayUntil(&xLastWakeTime,
                   xDelay250ms);
        }
}
```

Listing 4.9: Task priorities example

- Create three tasks (two continuous and one periodic) using the FreeRTOS API function xTaskCreate by replacing in the main function defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes); instruction

```
uint16_t  Task1LED = GREEN_LED;
uint16_t  Task2LED = BLUE_LED;
uint16_t  Task3LED = RED_LED;

BaseType_t xReturned1, xReturned2, xReturned3;
/* Create two continuous processing task, both at
   priority      1. */
xReturned1 = xTaskCreate(vContinuousProcessingTask,
    "Task␣1", 128, (void*)&Task1LED, 1, NULL);
xReturned2 = xTaskCreate(vContinuousProcessingTask,
    "Task␣2", 128, (void*)&Task2LED, 1, NULL);
/* Create one periodic task at priority 2. */
xReturned3 = xTaskCreate(vPeriodicTask, "Task␣3",
   128, (void*)&Task3LED, 2, NULL);
```

Listing 4.10: Tasks creation

If the task creation was successful, start the scheduler by using the vTaskStartScheduler() API, and remove the CMSIS call osKernelStart().

- Follow the steps for running and debugging the applications presented in the first lesson.

4.5.4. Changing task priorities in FreeRTOS

- Change the previous task functions from the previous example, to:

```
/* Used to hold the handle of Task2. */
TaskHandle_t xTask2Handle;

void vTask1(void* pvParameters)
{
        UBaseType_t uxPriority;
        uint16_t* LED;
        LED = (uint16_t*)pvParameters;
        uint32_t ul;
```

```
10          /* Query this task's priority - passing in
               NULL means "return the caller task
               priority". */
11          uxPriority = uxTaskPriorityGet(NULL);
12
13          for (;; )
14          {
15                HAL_GPIO_TogglePin(GPIOD, *LED);
16                for (ul = 0; ul <
                     mainDELAY_LOOP_COUNT; ul++)
17                      ;  /*a software delay, just
                           to see the LED blink*/
18                /* Increase priority of Task 2 above
                     this task's priority */
19                vTaskPrioritySet(xTask2Handle, (
                     uxPriority + 1));
20
21          }
22  }
23  void vTask2(void* pvParameters)
24  {
25          UBaseType_t uxPriority;
26          uint16_t* LED;
27          LED = (uint16_t*)pvParameters;
28          uint32_t ul;
29          /* Query this task's priority */
30          uxPriority = uxTaskPriorityGet(NULL);
31
32          for (;; )
33          {
34                HAL_GPIO_TogglePin(GPIOD, *LED);
35                for (ul = 0; ul <
                     mainDELAY_LOOP_COUNT; ul++)
36                      ;/*a sofware delay, just to
                           see the LED blink */
37  /*Set the current task priority back down to its
       original value*/
38              vTaskPrioritySet(NULL,(uxPriority-2));
39          }
40  }
```

Listing 4.11: Priority change example

- Create two tasks using the FreeRTOS API function xTaskCreate and start the scheduler:

```
BaseType_t xReturned1, xReturned2;
/* Create the first task at priority 2.  The task
   handle is not used so is set to NULL. */
xReturned1 = xTaskCreate(vTask1, "Task 1", 1000, (
   void*)&Task1LED, 2, NULL);
/* Create the second task at priority 1 - which is
   lower than the priority given to Task1.  But
   this time we want to obtain a handle to the task
    so pass in the address of the xTask2Handle
   variable. */
xReturned2 = xTaskCreate(vTask2, "Task 2", 1000, (
   void*)&Task2LED, 1, &xTask2Handle);
/* The task handle is the last parameter */
if ((xReturned1==pdPASS) && (xReturned2==pdPASS))
vTaskStartScheduler();
```

Listing 4.12: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.

4.5.5. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/04-RTOS-kernel-control/03-vTaskStartScheduler
https://www.freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/01-vTaskDelay
https://www.freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/02-vTaskDelayUntil
https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/03-vTaskDelete

Exercises

- For the first application, change the priority of the periodic task and the priorities of the continuous processing tasks. Observe the behavior in case the continuous tasks have a greater priority than the periodic task.

- Disable preemption: #define configUSE_PREEMPTION 0 Rebuild the first application and observe if there are any differences.
- For the second application, create a new Task3 in the function implementing Task1 by using xTaskCreate(). Make Task3 blink the orange LED and then delete itself by using vTaskDelete() function.

4.6. Lesson 5 - Software Timers

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Using FreeRTOS software timers

The application requirements are:

- Create and use software timers for blinking LEDs with certain frequencies. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to reduce application execution jitter by using software timers.

4.6.1. Theoretical Aspects

One drawback of priority scheduling in general is that it introduces jitter. Jitter is the deviation of a periodic task from its periodic behavior. Release time jitter is the deviation of the task from being released at periodic time intervals.

One way of reducing jitter is to use timers. They can be configured to generate an interrupt that executes a callback function at a set time in the future, or periodically with a fixed frequency. As hardware timers, software timers do not use any processing time unless a callback function is executing. Thus, timers can be used to implement clock-driven scheduling mechanisms.

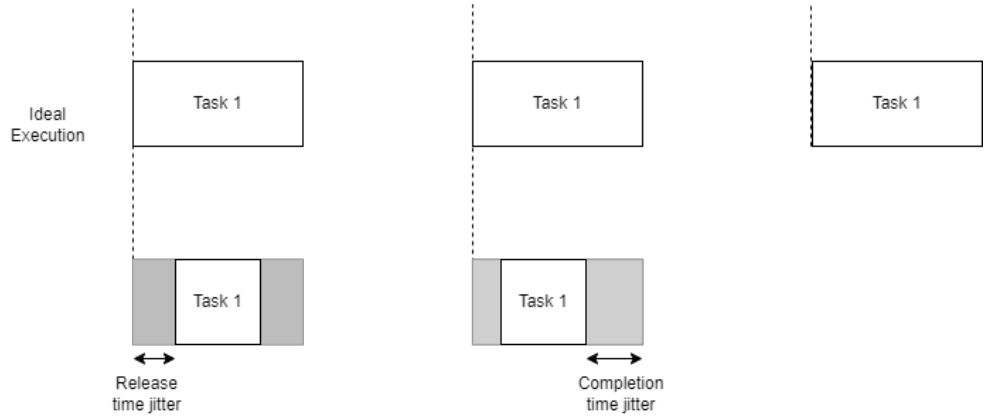

Figure 4.8. Jitter

Clock-driven schedulers make their scheduling decisions only at clock interrupt points. The scheduling points are determined by timer interrupts. They are off-line schedulers: the start time of each task is predetermined (i.e. known before the system starts to run).

In the next figure, there is an example of a perfectly periodic execution (i.e. the start time offset is constant). In this case the task execution can be triggered by a timer.

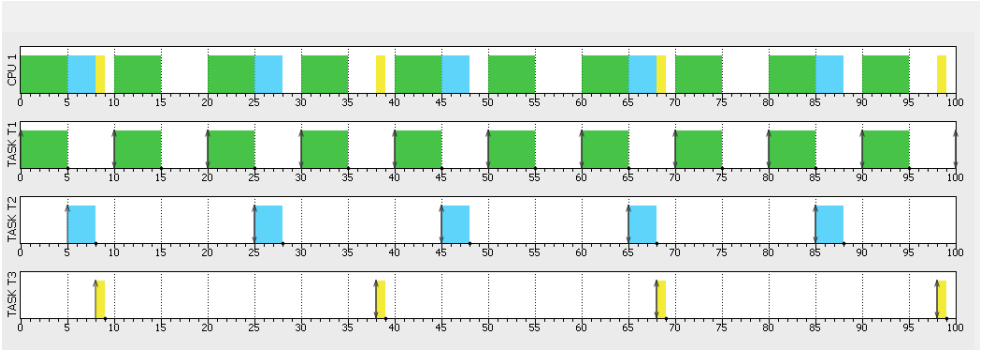

Figure 4.9. Jitterless execution

An example of an algorithm for computing the start time offset in a non-preemptive environment is Fixed Execution Non-Preemptive (FENP) [33].

4.6.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that the use of timers is enabled
- Make sure that the timer stack size is properly set: to minimum configMINIMAL_STACK_SIZE * 2

4.6.3. Using Software Timers in FreeRTOS

- Create two callback functions: one for a one-shot timer and one for an auto-reload (periodical) timer, respecting the callback function prototype: void ATimerCallback(TimerHandle_t xTimer); as in the following example:

```
1 static void prvOneShotTimerCallback(TimerHandle_t
     xTimer)
2 {
3 /* The LED will be only turned on,as this function
     will execute only once */
4         HAL_GPIO_TogglePin(GPIOD, RED_LED);
5 }
6 /*-----------------------------------*/
7
8 static void prvAutoReloadTimerCallback(
     TimerHandle_t xTimer)
9 {
10         /* Toggle LED */
11         HAL_GPIO_TogglePin(GPIOD, GREEN_LED);
12 }
```

Listing 4.13: Task creation

- Declare the timer handles and create one timer to function in one shot mode, and the other as a periodical timer:

```
1 TimerHandle_t xAutoReloadTimer, xOneShotTimer;
2 BaseType_t xTimer1Started, xTimer2Started;
3
4 /* Create the one shot software timer, storing the
     handle to the created
```

```
5 software timer in xOneShotTimer. */
6 xOneShotTimer = xTimerCreate("OneShot",
     mainONE_SHOT_TIMER_PERIOD,      /* The software
     timer's period in ticks. */
7          pdFALSE,        /* Setting uxAutoRealod to
              pdFALSE creates a one-shot software
              timer. */
8          0,      /* This example does not use the
              timer id. */
9          prvOneShotTimerCallback);       /* The
              callback function to be used by the
              software timer being created. */
10
11 /* Create the auto-reload software timer, storing
     the handle to the created software timer in
     xAutoReloadTimer. */
12 xAutoReloadTimer = xTimerCreate("AutoReload",
     mainAUTO_RELOAD_TIMER_PERIOD,     /* The
     software timer's period in ticks. */
13         pdTRUE, /* Set uxAutoRealod to pdTRUE to
              create an auto-reload software timer. */
14         0,      /* This example does not use the
              timer id. */
15         prvAutoReloadTimerCallback);    /* The
              callback function to be used by the
              software timer being created. */
```

Listing 4.14: Task creation

- 3. Check that the timers were created and start the timers, and then if the timers were started successfully, start the scheduler:

```
1 /* Check the timers were created. */
2 if ((xOneShotTimer != NULL) && (xAutoReloadTimer !=
     NULL))
3 {
4     /* Start the software timers, using a block
         time of 0 (no block time).The scheduler has
         not been started yet so any block time
         specified here
5 would be ignored anyway. */
6     xTimer1Started = xTimerStart(xOneShotTimer, 0);
```

```
7     xTimer2Started = xTimerStart(xAutoReloadTimer,
          0);
8
9     /* The implementation of xTimerStart() uses the
          timer command queue, and xTimerStart() will
          fail if the timer command queue gets full.
          The timer service task does not get created
          until the scheduler is started, so all
10 commands sent to the command queue will stay in the
      queue until after the scheduler has been
     started.  Check both calls to xTimerStart()
     passed. */
11 }
12 if ((xTimer1Started == pdPASS) && (xTimer2Started
     == pdPASS))
13 vTaskStartScheduler();
```

Listing 4.15: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.

4.6.4. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/11-Software-timers/01-xTimerCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/11-Software-timers/04-xTimerStart
https://www.freertos.org/Documentation/02-Kernel/04-API-references/11-Software-timers/05-xTimerStop
https://www.freertos.org/Documentation/02-Kernel/04-API-references/11-Software-timers/06-xTimerChangePeriod

Exercises

- Create more timers (maximum 4 in total), but with the same callback function and different IDs. Use the timers' callback function to toggle a different LED for each timer ID.
- Change the load value for each auto-reload timer by using the xTimerChangePeriod() function and observe the change in the frequency

with which the LEDs toggle.

- Add a maximum number of execution times for the software timer callback function and observe the behavior. Hint: use xTimerStop() function.

4.7. Lesson 6 - Interrupt Management

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Managing interrupts in FreeRTOS
- Using Semaphores

The application requirements are:

- Create a task for treating interrupt events and synchronize it with the proper interrupt service routine using semaphores. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to threat interrupts in FreeRTOS and deal with aperiodic events.

4.7.1. Theoretical Aspects

In order to treat aperiodic or sporadic events, they need to be detected in the first place. There are two ways in which the system can detect an event: by using interrupts or by polling the inputs. Each one of these methods has its advantages and disadvantages.

On one hand, tasks that poll after specific input states, usually called polling servers, can be easily integrated into the default priority-based scheduling mechanism, but they consume processor utilization when it is not needed.

On the other, interrupts are efficient from the resource management point of view, but they interfere with the scheduling mechanism, and if not treated correctly, they can introduce indeterminism in the system, which is not acceptable in the case of critical hard real-time systems.

A hybrid approach would be to use deferrable servers. They can be synchronized with very short interrupts. These servers are treated as sporadic tasks, and are also characterized by a period (minimum interarrival time) and computation time called server capacity. The period of the servers must be at least two times smaller than the event period, according to Nyquist-Shannon sampling theorem [77].

The difference between a deferrable server and a polling one is that unlike the polling servers, which replenish their capacity periodically, deferrable servers preserve their capacity if no requests are pending upon the invocation of the server.

In this case the feasibility condition for a set of n periodic tasks and a deferrable server becomes [78]:

$$U_p + U_s \leq U_s + ln\frac{U_s + 2}{2U_s + 1}$$

where U_p is the total utilization of the task set

$$U_p = \sum \frac{C_i}{T_i}$$

and U_s is the total utilization factor of the server

$$U_p = \frac{C_s}{T_s}$$

As a principle, in real-time systems, interrupt service routines (ISR), if used, should be kept as short as possible, and the actual functionality that treats the event should be implemented as a task (server), thus properly integrated into the scheduling mechanism.

FreeRTOS offers several mechanisms to synchronize tasks, or interrupts and tasks. One of them is represented by the semaphore [74].

4.7.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that preemption is set and that the vTaskDelay and vTaskDelayUntil functions are included
- Check the value of configTICK_RATE_HZ

4.7.3. Using Semaphores in FreeRTOS

- Set the user push button to generate an interrupt (see Chapter 3.3).
- Use a binary semaphore to synchronize the interrupt handler with another task, which represents the event handler that toggles the blue LED.

```
1  /* Declare a variable of type SemaphoreHandle_t.
      This is used to reference the
2  semaphore that is used to synchronize a task with
      an interrupt. */
3  SemaphoreHandle_t xBinarySemaphore;
4
5
6  /**
7    * @brief This function handles EXTI line0
       interrupt.
8          In our case it represents the ISR for the
             push button.
9    */
10 void EXTI0_IRQHandler(void)
11 {
12         BaseType_t xHigherPriorityTaskWoken;
13         /* USER CODE BEGIN EXTI0_IRQn 0 */
14         xHigherPriorityTaskWoken = pdFALSE;
15
16         /* The RED LED just marks the execution of
              the ISR. */
17         HAL_GPIO_TogglePin(GPIOD, RED_LED);
18         /* 'Give' the semaphore to unblock the task
              . */
19         xSemaphoreGiveFromISR(xBinarySemaphore, &
              xHigherPriorityTaskWoken);
20
21         /* USER CODE END EXTI0_IRQn 0 */
22         HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
23         /* USER CODE BEGIN EXTI0_IRQn 1 */
24
25         /* USER CODE END EXTI0_IRQn 1 */
26 }
27
```

```
28 static void vHandlerTask(void* pvParameters)
29 {
30         /* As per most tasks, this task is
              implemented within an infinite loop. */
31         for (;; )
32         {
33                 /* Use the semaphore to wait for
                      the event.  The semaphore was
                      created
34                 before the scheduler was started so
                       before this task ran for the
                      first
35                 time.  The task blocks indefinitely
                       meaning this function call will
                       only
36                 return once the semaphore has been
                      successfully obtained - so there
                       is
37                 no need to check the returned value
                      . */
38                 xSemaphoreTake(xBinarySemaphore,
                      portMAX_DELAY);
39
40                 HAL_GPIO_TogglePin(GPIOD, BLUE_LED)
                      ;
41         }
42 }
```

Listing 4.16: Task creation

- Create another function for a periodic task, that toggle the green LED.

```
1 static void vPeriodicTask(void* pvParameters)
2 {
3         const TickType_t xDelay = pdMS_TO_TICKS(50
             UL);
4
5         /* As per most tasks, this task is
             implemented within an infinite loop. */
6         for (;; )
7         {
```

```
8
9                   vTaskDelay(xDelay);
10
11                  HAL_GPIO_TogglePin(GPIOD, GREEN_LED
                       );
12          }
13  }
```

Listing 4.17: Task creation

- Create the semaphore and the two tasks (the one that treats the push button event and the periodic task). Give proper priorities.

```
1  BaseType_t xReturned1, xReturned2;
2  /* Before a semaphore is used it must be explicitly
       created.  In this
3  example a binary semaphore is created. */
4  xBinarySemaphore = xSemaphoreCreateBinary();
5
6  /* Check the semaphore was created successfully. */
7  if (xBinarySemaphore != NULL)
8  {
9          /* Create the 'handler' task, which is the
              task to which interrupt
10         processing is deferred, and so is the task
              that will be synchronized
11         with the interrupt.  The handler task is
              created with a high priority to
12         ensure it runs immediately after the
              interrupt exits.  In this case a
13         priority of 2 is chosen. */
14         xReturned1 = xTaskCreate(vHandlerTask, "
              Handler", 1000, NULL, 2, NULL);
15         /* Create the task that will periodically
              generate a software interrupt.
16         This is created with a priority below the
              handler task to ensure it will
17         get preempted each time the handler task
              exits the Blocked state. */
18         xReturned2 = xTaskCreate(vPeriodicTask, "
              Periodic", 1000, NULL, 1, NULL);
19 }
```

```
20 if ((xReturned1==pdPASS) && (xReturned2==pdPASS))
21 vTaskStartScheduler();
```

Listing 4.18: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.

4.7.4. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/10-Semaphore-and-Mutexes/00-Semaphores

Exercises

- Comment out the xSemaphoreGiveFromISR() and xSemaphoreTake() and observe the difference.
  Why the blue LED remains off in this situation?
  Explain the behavior.
- Replace the binary semaphore with a counting semaphore with a size of 3: SemaphoreHandle_t xCountingSemaphore;
  xCountingSemaphore = xSemaphoreCreateCounting(3, 0);
  Give the semaphore 3 times in the ISR to simulate a multiple event series. Observe if there are any differences in the behavior of the application.

4.8. Lesson 7 - Resource Sharing

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Learning how to share resources in exclusive mode in FreeRTOS
- Using Mutexes

The application requirements are:

- Create different tasks that share a resource in exclusive mode. Use proper resource sharing mechanisms. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to use Mutexes for sharing resources between real-time tasks in FreeRTOS

4.8.1. Theoretical Aspects

Sometimes real-time tasks need to share other resources than CPU among themselves (e.g. data structures, memory, files, peripheral devices, etc.).

Some resources can only be used in exclusive mode. If a task using a resource is preempted before it completes using that resource, the resource can become corrupted (e.g. writing a CD, printing, writing to an LCD).

To ensure data consistency is maintained at all times, access to a non-preemptable resource must be managed using a "mutual exclusion" technique. The goal is to ensure that, once a task starts to access a non-preemptable resource, the same task has exclusive access to the resource during its usage.

A piece of code executed under mutual exclusion constraints is called a critical section. Any task that needs to enter a critical section must wait until no other task is holding the resource. A task waiting for an exclusive resource is said to be blocked on that resource, otherwise, it proceeds by entering the critical section and holds the resource.

Besides data corruption, other unwanted situations are represented by (unbounded) priority inversion and deadlocks.

A deadlock is a situation in which two or more tasks sharing the same resource are effectively preventing each other from accessing the resource.

Priority inversion occurs when a high priority task is forced to wait for the release of a shared resource owned by a lower priority task.

In this case, the priority inversion is unbounded, due to the unbounded preemptions of the lower priority task by the medium priority tasks.

One of the simplest ways to avoid unbounded priority inversion is to disallow preemption during the execution of any critical section. This approach is equivalent to raising the priority of a task to the highest priority level whenever it enters a critical section. After exiting the critical section, the priority of the task returns to its original value. This method has the disadvantage that high priority tasks that do not require the resource are blocked by tasks executing critical sections.

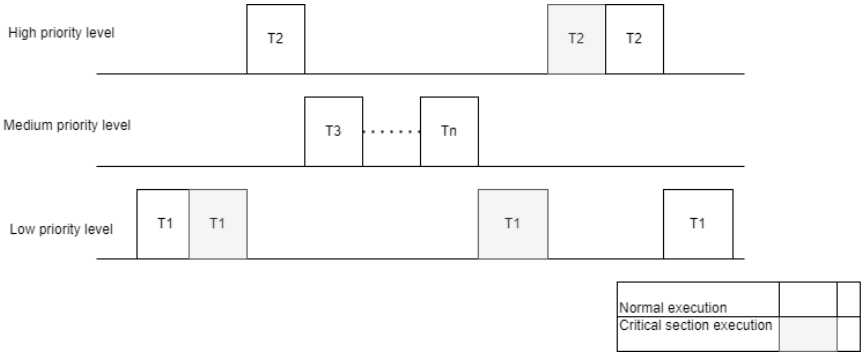

Figure 4.10. Unbounded priority inversion

Another way to avoid unbounded priority inversion is to only avoid the preemption of the low priority task holding a critical resource, by higher priority tasks that need that resource. This can be achieved by raising the priority level of the low priority task to be equal to that of the highest level task waiting for the resource. This is achieved by the usage of the Priority Inheritance Protocol.

FreeRTOS offers the following mutual exclusion mechanisms [74]:

- Functions for disabling task preemptions such as: taskENTER_CRITICAL() and taskEXIT_CRITICAL(), or the possibility to disable the scheduler by calling vTaskSuspendAll().
- MUTEXES, which are a special type of binary semaphore, used to control mutually exclusive access to a shared resource, and which implement Priority Inheritance Protocol for resource sharing.

4.8.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that preemption is set and that the vTaskDelay and vTaskDelayUntil functions are included

4.8.3. Using Mutexes in FreeRTOS

- Include stdlib.h, in order to be able to use the rand() function.
- Declare a variable of type SemaphoreHandle_t. This is used to reference the MUTEX type and create a MUTEX: SemaphoreHandle_t

xMutex; xMutex = xSemaphoreCreateMutex();

- Create a function that has a critical section guarded by the usage of the MUTEX. The critical section will access the red LED in an exclusive mode for blinking it five times with a certain delay, received as an input. The non-critical section of the function will go into a blocking state for a random amount of time.

```
1  /* The tasks block for a pseudo random time between
       0 and xMaxBlockTime ticks. */
2  const TickType_t xMaxBlockTimeTicks = 0x20;
3
4  static void prvBlinkLED(void* pvParameters)
5  {
6          uint32_t* delay;
7          /*The delay values ensure that the low
              priority task starts first*/
8          delay = (uint32_t*)pvParameters;
9          uint32_t cnt, ul;
10         for (;; )
11         {
12                 xSemaphoreTake(xMutex,
                      portMAX_DELAY);
13                 {
14                         /* The following line will
                              only execute once the
                              semaphore has been
15                         successfully obtained - so
                              standard out can be
                              accessed freely. */
16                         for (cnt = 0; cnt < 10; cnt
                              ++)
17                         {
18                                 HAL_GPIO_TogglePin(
                                      GPIOD, RED_LED);
19                                 /* Delay for a
                                      period. */
20                                 for (ul = 0; ul < *
                                      delay; ul++)
21                                 {
22                                 ;/* This loop is
                                      just a blocking
```

```
                                        delay
                                        implementation.
                                        */
23                                    }
24                            }
25                    }
26                    xSemaphoreGive(xMutex);
27                    /* Wait a pseudo random time.*/
28                    vTaskDelay(rand() %
                         xMaxBlockTimeTicks);
29            }
30    }
```

Listing 4.19: Task functions

- Create two tasks with different priorities. Activate the low priority task first, by sending a lower delay value as a parameter, as in the theoretical example illustrating priority inversion and then start the scheduler.

```
1  TickType_t delay1 = mainDELAY_LOOP_COUNT, delay2 =
       mainDELAY_LOOP_COUNT / 10;
2  /* Check the semaphore was created successfully. */
3  if (xMutex != NULL)
4  {
5          /* Create two instances of the tasks that
              attempt to use the same resource (the
              RED LED in our case).
6           * The tasks are created at different
               priorities so some pre-emption will
               occur. */
7           /*The delay values ensure that the low
               priority task starts first*/
8          xReturned1 = xTaskCreate(prvBlinkLED, "
              Task1", 128, &delay1, 1, NULL);
9          xReturned2 = xTaskCreate(prvBlinkLED, "
              Task2", 128, &delay2, 3, NULL);
10 }
11 if ((xReturned1 == pdPASS) && (xReturned2 == pdPASS
       ))
12 vTaskStartScheduler();
```

Listing 4.20: Task creation

- Follow the steps for running and debugging the applications presented in the first lesson.
- Observe the blinking frequency of the LED and how it changes, from one task to another.

4.8.4. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/10-Semaphore-and-Mutexes/06-xSemaphoreCreateMutex

Exercises

- Remove the usage of the Mutex (by commenting out xSemaphoreTake(xMutex, portMAX_DELAY); and xSemaphoreGive(xMutex); calls), build, run the application and observe the difference. Explain the behavior.
- After removing the usage of the Mutex, change the priorities of the tasks to be equal, build, run the application and observe the difference. Put back the Mutex and observe the difference again.
- Besides the two tasks, that share the red LED in an exclusive mode, introduce a medium task that does not use the red LED, but blinks the green LED (not in an exclusive mode). Activate the tasks in the same order as in the theoretical example. Build and run the application and observe the behavior. Remove the Mutex and observe the difference when running the application.
- Change the previous exercise and replace the Mutex with a binary semaphore. Observe the difference.

4.9. Lesson 8 - Queues

This lesson's main objectives consist of:

- Getting familiar with FreeRTOS API
- Getting familiar with FreeRTOS queues

The application requirements are:

- Create multiple tasks and a queue to transmit data from one task to another. Observation: We will use the native FreeRTOS API, not the CMSIS middleware API.

After going through the proposed exercises the reader will be able to use queues in FreeRTOS.

4.9.1. Theoretical Aspects

FreeRTOS queues offer support for thread-safe intertask communication. They can be used to send messages between tasks and between interrupts and tasks.

By default, they are used as FIFO (first-in-first-out) buffers.

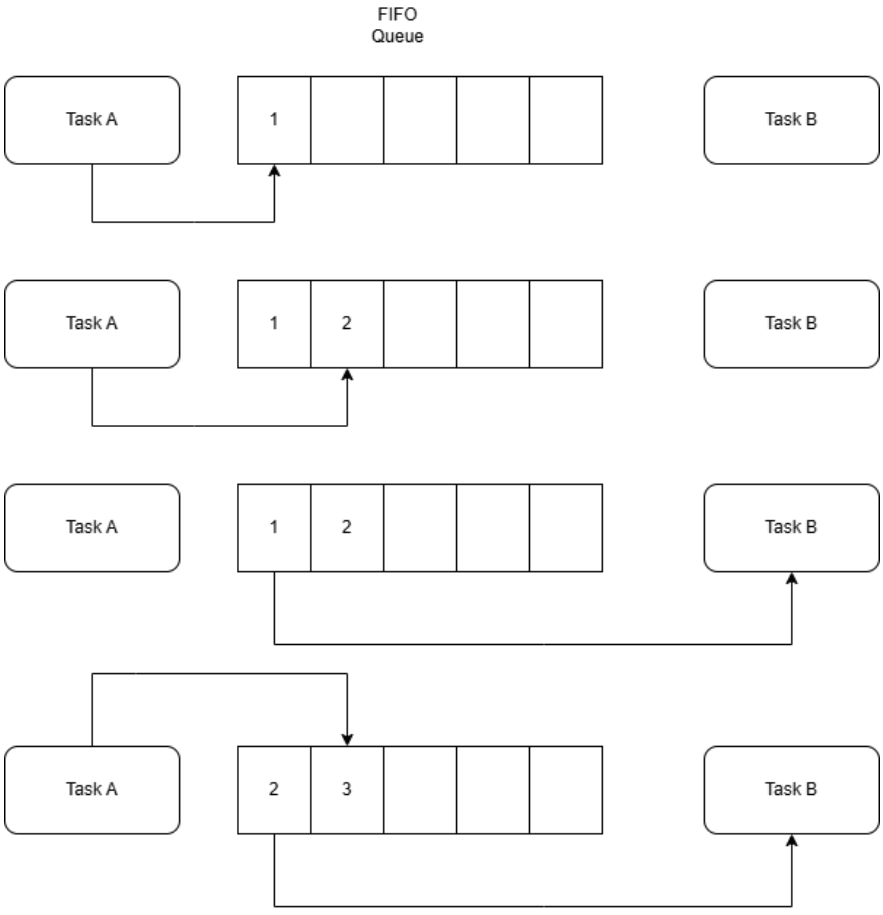

Figure 4.11. FIFO

4.9.2. Configuring FreeRTOS

- Open STMCube IDE and create a new project following the steps from the first lesson
- In FreeRTOSConfig.h, make sure that queues are enabled, preemption is set and that the vTaskDelay and vTaskDelayUntil functions are included

4.9.3. Using Queues in FreeRTOS

- Declare a queue handler as a global variable of type QueueHandle_t: QueueHandle_t xQueue;
- Create a function for a sender task that pushes to the back of the queue the parameter it receives at creation (which will be the number of the pin for blinking a certain LED in this example).

```
1  static void vSenderTask(void* pvParameters)
2  {
3          uint16_t* lValueToSend;
4          /* Cast the parameter to the required type.
               */
5          lValueToSend = (uint16_t*)pvParameters;
6          const TickType_t xTicksToWait =
              pdMS_TO_TICKS(100UL);
7
8          for (;; )
9          {
10                 /* The first parameter is the queue
                       to which data is being sent.
                      The
11                 queue was created before the
                      scheduler was started, so before
                       this task
12                 started to execute.
13
14                 The second parameter is the address
                       of the data to be sent.
15
16                 The third parameter is the Block
                      time - the time the task should
```

```
                    be kept
17                  in the Blocked state to wait for
                       space to become available on the
                        queue
18                  should the queue already be full.
                       In this case we don't specify a
                       block
19                  time because there should always be
                        space in the queue. */
20                  xQueueSendToBack(xQueue,
                       lValueToSend, 0);
21                  vTaskDelay(xTicksToWait);
22
23          }
24  }
```

Listing 4.21: Task functions

- Create a function for a receiver task that pulls the first parameter from the queue (which is the pin number for the LED to blink).

```
1   static void vReceiverTask(void* pvParameters)
2   {
3           /* Declare the variable that will hold the
                values received from the queue. */
4           uint16_t lReceivedValue;
5           const TickType_t xTicksToWait =
                pdMS_TO_TICKS(500UL);
6           BaseType_t xStatus;
7           for (;; )
8           {
9                   /* The first parameter is the queue
                        from which data is to be
                       received.  The queue is created
                       before the scheduler is started,
                        and therefore before this task
                       runs for the first time.
10
11                  The second parameter is the buffer
                       into which the received data
                       will be
12                  placed.  In this case the buffer is
```

```
                    simply the address of a
                    variable that has the required
                    size to hold the received data.
13
14                  the last parameter is the block
                    time - the maximum amount of
                    time that the task should remain
                     in the Blocked state to wait
                    for data to be available should
                    the queue already be empty. */
15                  xStatus = xQueueReceive(xQueue, &
                    lReceivedValue, xTicksToWait);
16
17                  if (xStatus == pdPASS)
18                  {
19                          /* Data was successfully
                            received from the queue.
                             */
20                          HAL_GPIO_TogglePin(GPIOD,
                            lReceivedValue);
21
22                  }
23
24          }
25  }
```

Listing 4.22: Task creation

- Create the queue, two sender tasks, and one receiving task, by using the previously defined functions.

```
1  uint16_t led1 = GREEN_LED;
2  uint16_t led2 = BLUE_LED;
3
4
5  /* The queue is created to hold a maximum of 5
      short values. */
6  xQueue = xQueueCreate(5, sizeof(int16_t));
7  BaseType_t xReturned1, xReturned2, xReturned3;
8  if (xQueue != NULL)
9  {
10         /* Create two instances of the task that
```

```
            will write to the queue.  The parameter
            is used to pass the value that the task
            should write to the queue,
Both tasks are created at priority 1. */
        xReturned1 = xTaskCreate(vSenderTask, "
            Sender1", 128, (void*)&led1, 1, NULL);
        xReturned2 = xTaskCreate(vSenderTask, "
            Sender2", 128, (void*)&led2, 1, NULL);

        /* Create the task that will read from the
            queue.  The task is created with
            priority 2, so above the priority of the
             sender tasks. */
        xReturned3 = xTaskCreate(vReceiverTask, "
            Receiver", 128, NULL, 2, NULL);
}
if ((xReturned1 == pdPASS) && (xReturned2 == pdPASS
   ) && (xReturned3 == pdPASS))
vTaskStartScheduler();
```

Listing 4.23: Queues creation

- Follow the steps for running and debugging the applications presented in the first lesson.

4.9.4. Bibliography, Further Reading and Exercises

https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
https://www.freertos.org/Documentation/02-Kernel/04-API-references/06-Queues/00-QueueManagement

Exercises

- Change the priorities of each task (e.g. make the receiver have a lower priority than the sender tasks) and observe the behavior.
- Remove the waiting time for the xQueueReceive call and observe the behavior.
- Modify the example from this lesson, in order to send data structures instead of integer values. The data structure will contain one uint16_t field with the pin number, and an enum field with the ID

of the sender. Blink different LEDs depending on the sender ID.

- Modify the example from this lesson by adding an ISR for the push button and use a queue to communicate between the ISR and a task. Do not forget to use the proper API functions in the IRS (e.g. xQueueSendToBackFromISR)!

- Modify the example from this lesson in order to use a queue set made of two queues.

```
1  static QueueHandle_t xQueue1 = NULL, xQueue2 = NULL
       ;
2  /* Declare a variable of type QueueSetHandle_t.
       This is the queue set to which the two queues
       are added. */
3  static QueueSetHandle_t xQueueSet = NULL;
4  xQueue1 = xQueueCreate(1, sizeof(uint16_t*));
5  xQueue2 = xQueueCreate(1, sizeof(uint16_t*));
6
7  /* Create the queue set.*/
8  xQueueSet = xQueueCreateSet(1 * 2);
9
10 /* Add the two queues to the set. */
11 xQueueAddToSet(xQueue1, xQueueSet);
12 xQueueAddToSet(xQueue2, xQueueSet);
```

Listing 4.24: Queue sets

Each sender task will send in its own queue from the set (i.e. Sender1 task will send in Queue1, by calling xQueueSend(xQueue1, &led1, 0); and Sender2 task will send in Queue2 xQueueSend(xQueue2, &led2, 0);). The receiver task will take data from the task set by using xQueueSelectFromSet() function and xQueueReceive().

```
1  QueueHandle_t xQueueThatContainsData;
2  xQueueThatContainsData = (QueueHandle_t)
       xQueueSelectFromSet(xQueueSet, portMAX_DELAY);
3  xQueueReceive(xQueueThatContainsData, &
       ReceivedLedNumber, 0);
```

Listing 4.25: Queue set receive

4.10. Lesson 9 - Tracing and Analysing

This lesson's main objectives consist of:

- Learning how to trace a FreeRTOS application using Tracealyzer
- Getting familiar with the Percepio Tracealyzer

The application requirements are:

- Rebuild the previous FreeRTOS applications, add Tracealyzer tracing facility into the projects, and trace their execution.

After going through the proposed exercises the reader will be able to trace and analyse FreeRTOS applications with Tracealyzer.

4.10.1. Theoretical Aspects

Feasibility and determinism are the most critical aspects in real-time systems. While formal analysis is used to check and guarantee feasibility and determinism for different execution models, tracing tools can be used to check that the actual execution is behaving according to the model. Moreover, tracing tools provide empirical insights into system behavior during execution, helping to discover different execution patterns and thus improving security and reliability. They can also be used for benchmarking real-time systems by collecting and analyzing data for specific time parameters such as task switching time, task preemption time, interrupt latency time, etc.

4.10.2. Installing Percepio Tracealyzer

- Go to https://percepio.com/tracealyzer/download-tracealyzer/
- Make an evaluation account
- Students can request a free Academic Individual License, which can be installed on only one computer
- After receiving the download link, via email, run the downloaded Tracealyzer installation file and install the application
- Add the (free) license and register the application

4.10.3. Adding the Trace Recorder Library in an STM32CubeIDE project

- Open Tracealyzer [79]

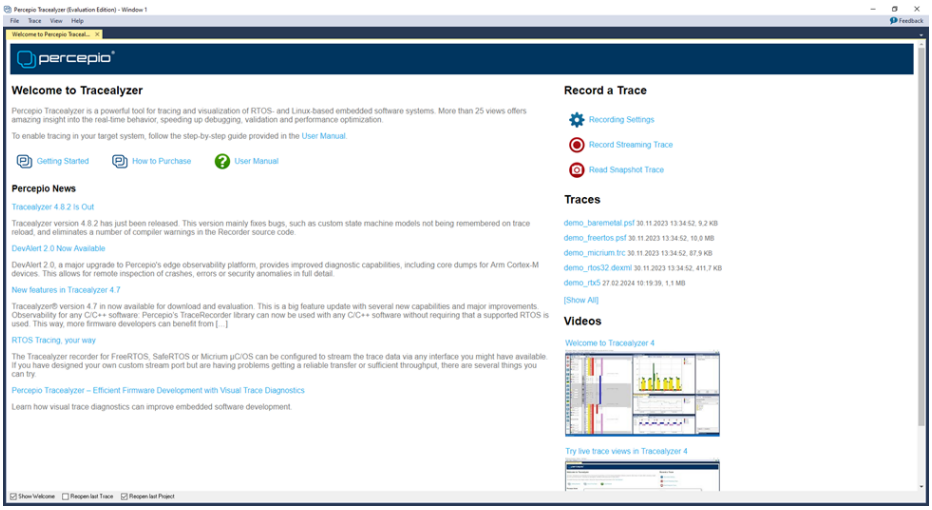

Figure 4.12. Tracealyzer

- Go to Help->FreeRTOS Trace Recorder and open the folder with the library files

Figure 4.13. Tracealyzer Library

By default, the files are located at:
C:\Program Files\Percepio\Tracealyzer 4\FreeRTOS

- Open your project with FreeRTOS applications in STM32CubeIDE
- Right click on the Core folder-> New-> Folder in order to keep your project organized. Call this folder Tracealyzer

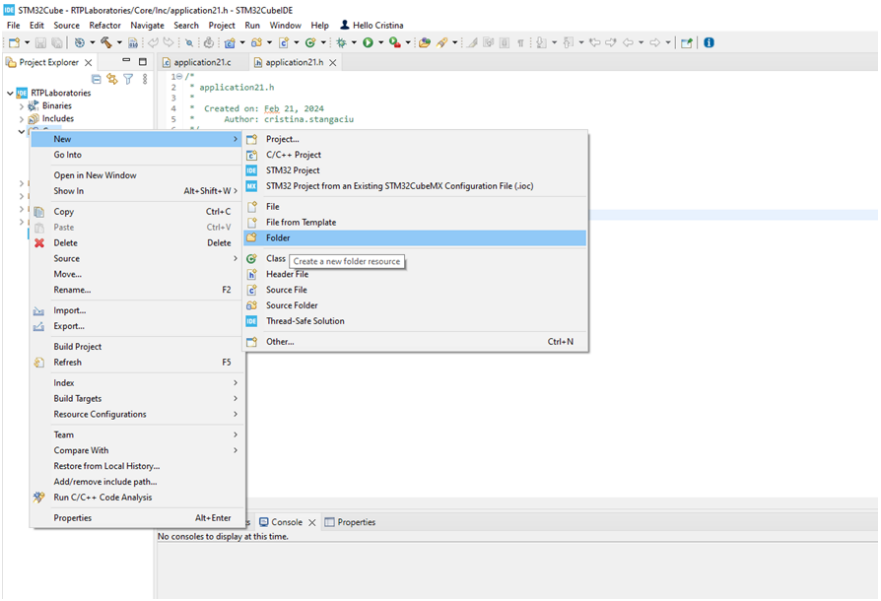

Figure 4.14. Tracealyzer Folder

- Copy the folders and files from C:\Program Files\Percepio\ Tracealyzer 4\FreeRTOS\TraceRecorder (except the License files, extras and streamports) into the new folder in your project called Tracealyzer

- Update the trcConfig.h file:

- Remove line: "#error "Trace Recorder: Please include your processor's header file here and remove this line."

- Add an include directive for your processor e.g. #include "stm32f4xx.h"

- Remove line: #define TRC_CFG_HARDWARE_PORT TRC_HARDWARE _PORT_NOT_SET

- Add the corresponding hardware port e.g. #define TRC_CFG_HARDWARE_PORT TRC_HARDWARE_PORT_ARM_Cortex_M

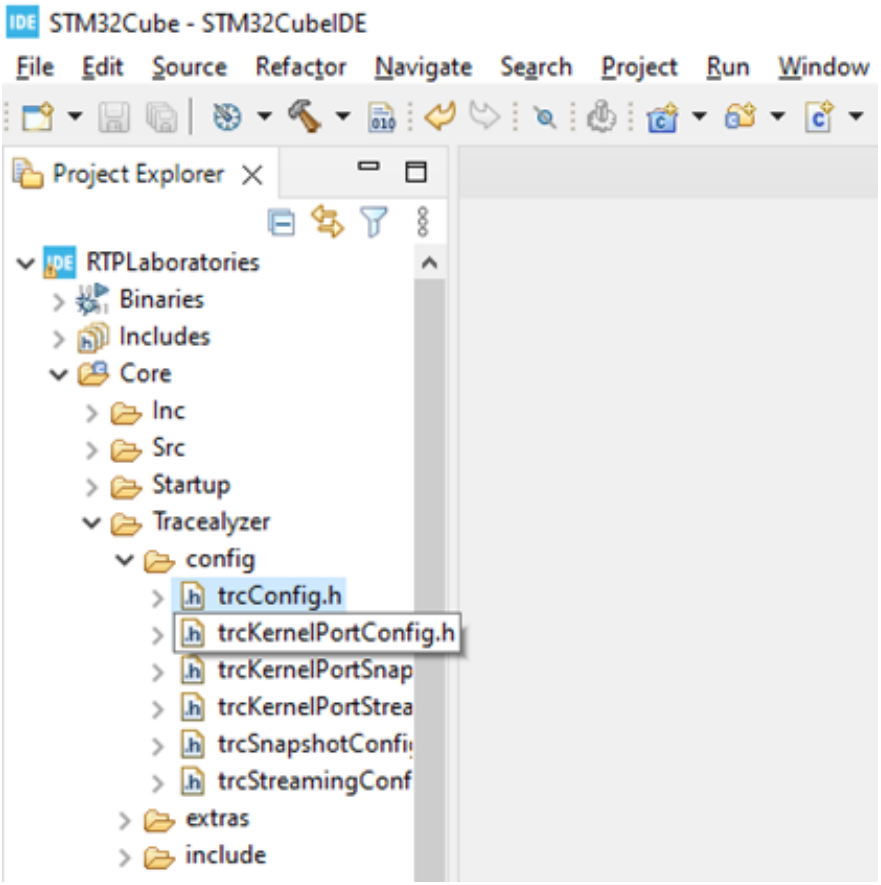

Figure 4.15. Tracealyzer Files

- Configure the system so that it uses the correct version of FreeRTOS. Get the version from FreeRTOSConfig.h file
- Change #define TRC_CFG_FREERTOS_VERSION FREERTOS_VERSION_NOT_SET to the corresponding version e.g. #define TRC_CFG_FREERTOS_VERSION TRC_FREERTOS_VERSION_10_3_1

  Depending on the library version, this can be found in trcKernelPort-Config.h or other files
- In the main file, in the main function, before initialization of the configured peripherals, add xTraceEnable(TRC_START)
- If using FreeRTOS v10.3.0 or v10.3.1, please make sure that the trace point in prvNotifyQueueSetContainer() in queue.c is renamed from

traceQUEUE_SEND to traceQUEUE_SET_SEND in order to tell them apart from other traceQUEUE_SEND trace points. Then set TRC_CFG_ACKNOWLEDGE_QUEUE_SET_SEND in trcKernelPortConfig.h to TRC_ACKNOWLEDGED to get rid of this error.

4.10.4. Configuring Tracealyzer Mode

- The recommended mode is streaming mode, a continuous tracing mode
- In trcKernelPortConfig.h Change the #define TRC_CFG_ RECORDER_MODE TRC_RECORDER_MODE_ SNAPSHOT to #define TRC_CFG_RECORDER_MODE TRC_RECORDER _MODE_STREAMING
- Choose and integrate a stream port. From the TraceRecord folder at C:\Program Files\Percepio\Tracealyzer 4\FreeRTOS\Trace Recorder\ choose the streamport available for your board. (Recommended to use JLink_RTT after enabling it). Copy the files included in this folder and the ones from the subfolders and add them into your STM32IDE project in the Tracealyzer folder, preserving the file structure (the files from config to TreceRecorder\config, the ones from include to TreceRecorder\include).
- Enabling and using onboard J-Link_RTT. Several boards from STM use ST-Link to upload the applications on the board. SEGGER company provides firmware that makes the ST-LINK on-board compatible with J-Link OB, allowing users to take advantage of most J-Link features, like tracing, in our case. In order to be able to use the tracing facility, please follow the tutorial at https://www.segger.com/products/debug-probes/j-link/models /other-j-links/st-link-on-board/
- Update the list of include files and rebuild the project. Go to Project -> Properties

  In C/C++ Build -> Settings->MCU GCC Compiler -> Include paths, include Tracealyzer/config and Tracealyzer/include

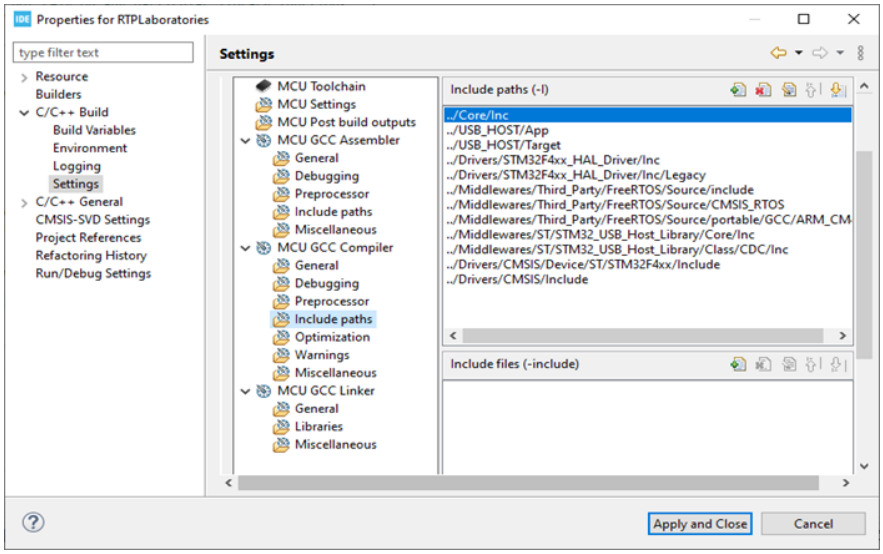

Figure 4.16. Include paths

- Rebuild the index, if asked

4.10.5. Adding Percepio tools in the STM32CubeIDE

- In STM32CubeIDE click Help -> Install New Software
- Enter this URL (https://percepio.com/stm32cubeide_exporter) in the "Work with:" box and hit Enter
- After a short while, the Percepio category, with Percepio Trace Exporter for STM32CubeIDE feature, will be shown in the list
- Select (check the check box) and hit Next to start the installation process
- Select the two URLs as trusted sites
- If the installation has succeeded, you will be asked to restart STM32CubeIDE, and after the restart you will see Percepio as a new tab in the menu

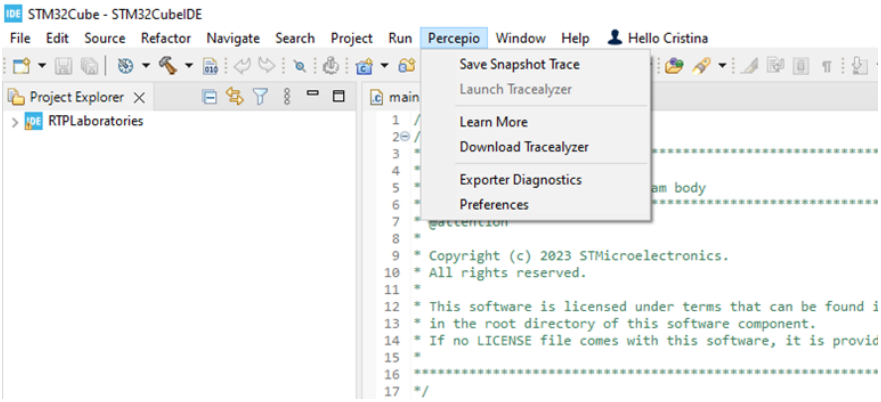

Figure 4.17. Percepio menu

- Add the path to the Trecealyzer.exe by clicking on Percepio-> Preferences

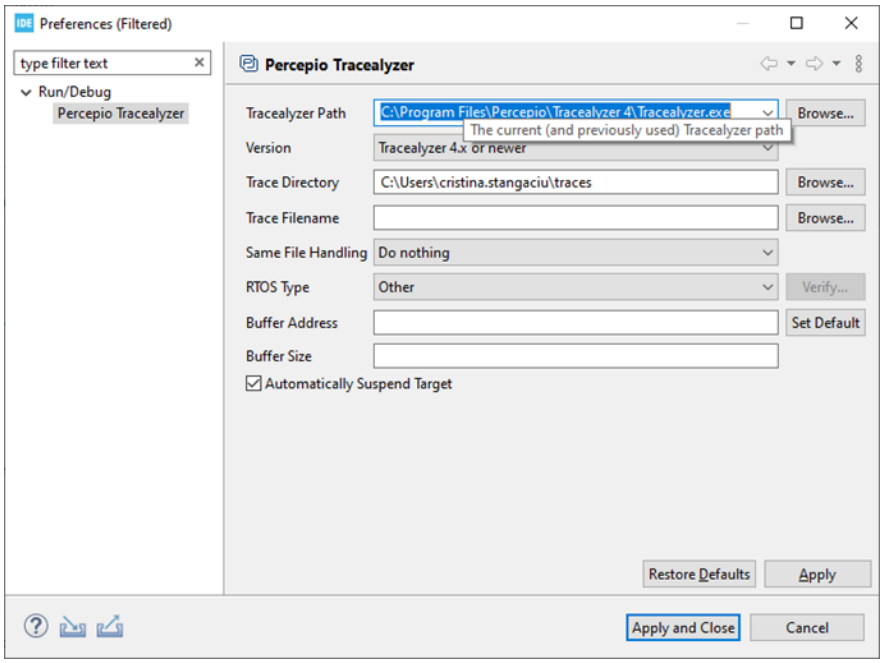

Figure 4.18. Tracealyzer path

4.10.6. Using Tracealyzer in STM32Cube with FreeRTOS

- Run the application in debug mode and check that xTraceEnable is called in the main function and that the trace mode is set to streaming

mode. Build the project and check that you have no errors and no warnings

- Check that a debugger is set in the Debug Configuration ... by right-clicking on the project and selecting Debug Configuration ...

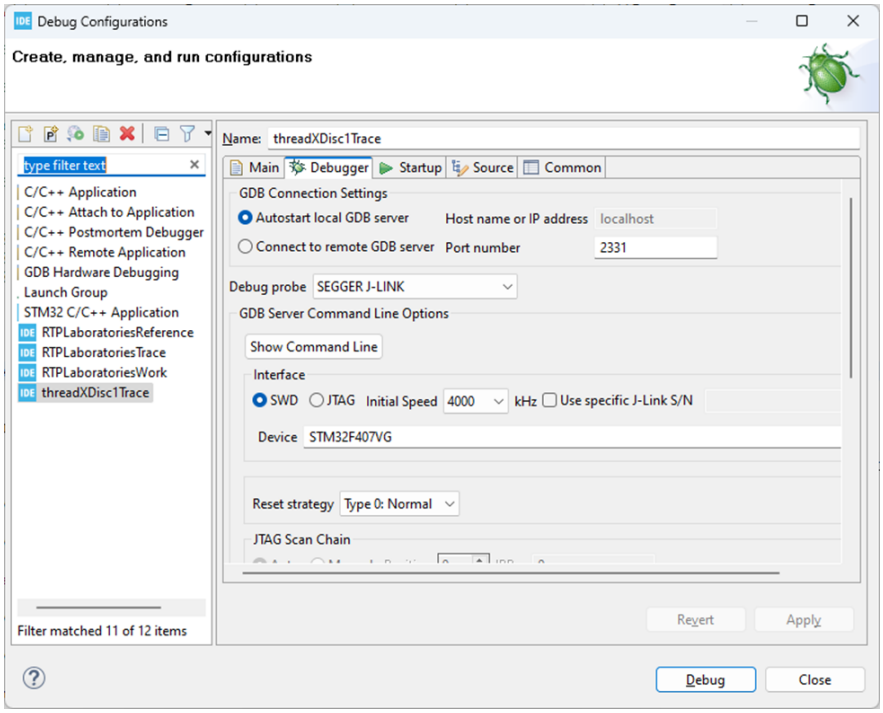

Figure 4.19. Debugger configuration

- In the Debugger tab select a debugging interface that supports tracing (J-Link in our case) Note: if J-Link is not an alternative, please rewrite the firmware of the STM board by following the next tutorial https://www.segger.com/products/debug-probes/j-link/models/other-j-links/st-link-on-board/

- Start the debugging session by right-clicking on the project and then select the STM32 C/C++ Application

- Run the application in debug mode by using the shortcuts (E.g. F8 for Resume)

4.10.7. Using Tracealyzer to visualize the execution of the application

- Make sure that the application is still running in debug mode in STM32IDE
- Launch Tracealyzer either directly or from the STM32 IDE

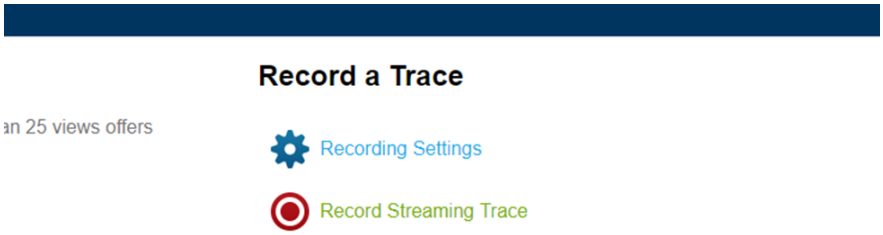

Figure 4.20. Record streaming trace

- From Percepio Tacealyzer select Record Streaming Trace
- In the Trace Menu select Open Live Stream Tool

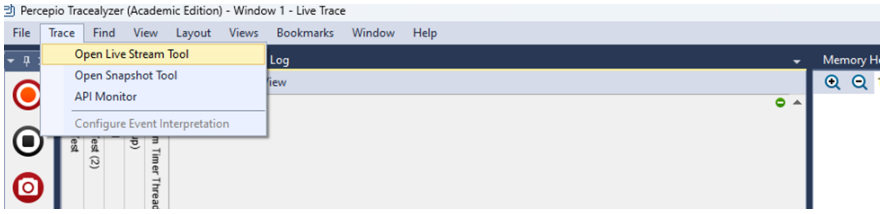

Figure 4.21. Open live streaming tool

- Select Settings

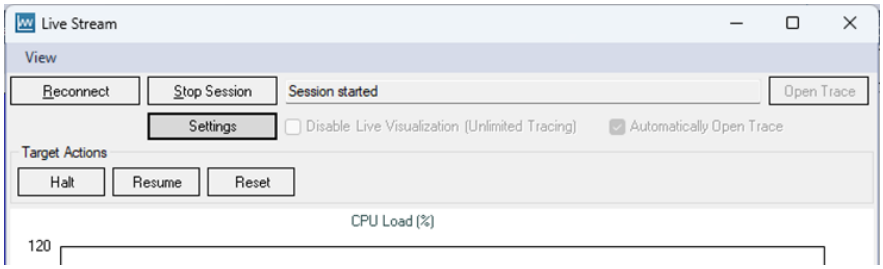

Figure 4.22. Streaming settings

- Configure the following parameter
- PSF Streaming Settings - Target connection Segger RTT
- Select the proper device and debugging interface and click OK
- Click on Reconnect and after Start Session

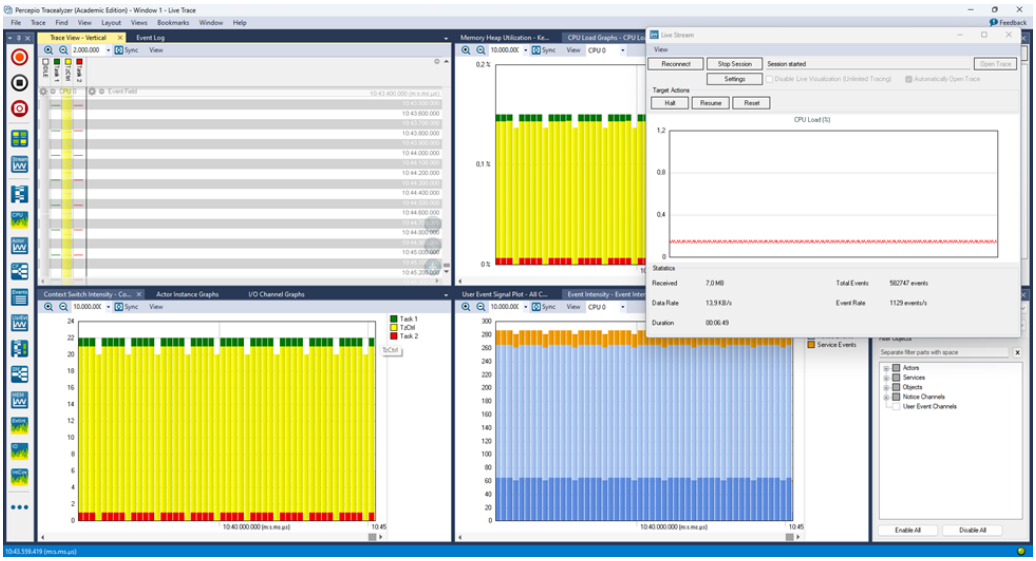

Figure 4.23. Start session

- Use the Tracealyzer menus for visualization and analysis

4.10.8. Bibliography, Further Reading and Exercises

https://percepio.com/tracealyzer/freertostrace/
https://percepio.com/getstarted/latest/html/freertos.html

Exercises

- Add Tracealyzer library to the previous application projects and monitor their execution in live streaming mode.
- Create or modify existing FreeRTOS applications in order to blink the four LEDs, two of them by using tasks, and two by using software timers, and compare the jitter of tasks versus software timer interrupts with the help of Tracealyzer. Is there any noticeable difference regarding the release jitter? Add a random blocking delay (for no

more than 20% of the LEDs' blinking frequencies), for both the tasks and the interrupt routines. How is the release jitter influenced in this situation?

- Monitor the execution of task synchronization applications from previous lessons and analyze the usage of semaphores using Tracealyzer.
- Monitor the execution of resource sharing applications from previous lessons and analyze the usage of mutexes using Tracealyzer.

4.11. Lesson 10 - Benchmarks

This lesson's main objectives consist of:

- Determine and compare different performance parameters for a real-time system
- Implement benchmarking tests

The application requirements are:

- Determine the performance of your system in terms of preemption time, interrupt latency, and semaphore shuffling time with the help of timers and Tracealyzer

After going through the proposed exercises the reader will be able to determine and analyze the performance of FreeRTOS in terms of switching time.

4.11.1. Theoretical Aspects

We use benchmarks or (standard) program sets in order to determine and compare the performance of different systems.

A benchmark developed for real-time systems is Rhealstone [80]. It consists of a set of programs for measuring six performance parameters: average task switch time, average preemption time, average interrupt latency, semaphore shuffle time, deadlock break time, and inter-task message latency. Based on these parameters average value, the system receives a score which can be used to compare different systems' performance.

Task Switching Time is defined as the time it takes for one context switch among equal priority tasks.

Task Preemption Time is defined as the time elapsed from the moment a higher priority task (compared to the currently running task) becomes available until the moment it starts to execute. Interrupt Latency Time is defined as the time it takes to start the execution of the required ISR after an interrupt occurs.

Semaphore Shuffle Time is defined as the time that elapses between a lower priority task releasing a semaphore and a higher priority task requesting the same semaphore starting to run.

Deadlock breaking occurs when a higher priority task preempts a lower priority task that holds a resource needed by the higher priority task. The deadlock-breaking metric measures the average time it takes the executive to resolve the conflict.

Inter-task massage latency measures the time delay between the sending and receiving of a message in inter-task communication situations.

4.11.2. Configuring the project

- Open STMCube IDE and create a new project following the steps from Chapter 4.2.
- Add serial communication by configuring the UART following the steps from Chapter 3.6.

4.11.3. Adding serial communication functions

- Include the following:

```
1 /* USER CODE BEGIN Includes */
2 #include "stm32f4xx_hal_gpio.h"
3 #include "usbd_cdc_if.h"
4 #include <stdarg.h>
5 #include <stdio.h>
6 /* USER CODE END Includes */
```

Listing 4.26: Includes

- Implement a function similar to printf, in order to send messages on the serial interface

```
1 the serial interface
2 /* USER CODE BEGIN 0 */
3 #define LOG_BUFFER_SIZE 128
```

```
4 uint8_t buf[LOG_BUFFER_SIZE];
5 void vPrintSerial(const char* format, ...)
6 {
7         va_list ap;
8         va_start(ap, format);
9         size_t len = vsnprintf((char*)buf, sizeof(
              buf), format, ap);
10        va_end(ap);
11        if (len > sizeof(buf) - 1)
12                len = sizeof(buf) - 1;
13        CDC_Transmit_FS(buf, len); /* send string
              to serial interface */
14 }
15 /* USER CODE END 0 */
```

Listing 4.27: vPrintSerial

- Comment out the CMSIS API function calls from the main function.

```
1 /* osKernelInitialize(); */
2 /* defaultTaskHandle = osThreadNew(
3 StartDefaultTask,NULL,&defaultTask_attributes); */
4 /* osKernelStart(); */
```

Listing 4.28: CMSIS calls

- Add the following function call after the initialization of the configured peripherals

```
1 /* USER CODE BEGIN 2 */
2 MX_USB_DEVICE_Init();
3 /* USER CODE END 2 */
```

Listing 4.29: MX_USB_DEVICE_Init

- Now you can use vPrintSerial function to send messages to the serial terminal on your host PC: e.g. vPrintSerial("Test\n");

4.11.4. Configuring a timer in order to measure different time intervals

- Just configure the TIM2 timer to count to the maximum value and enable the auto-reload.

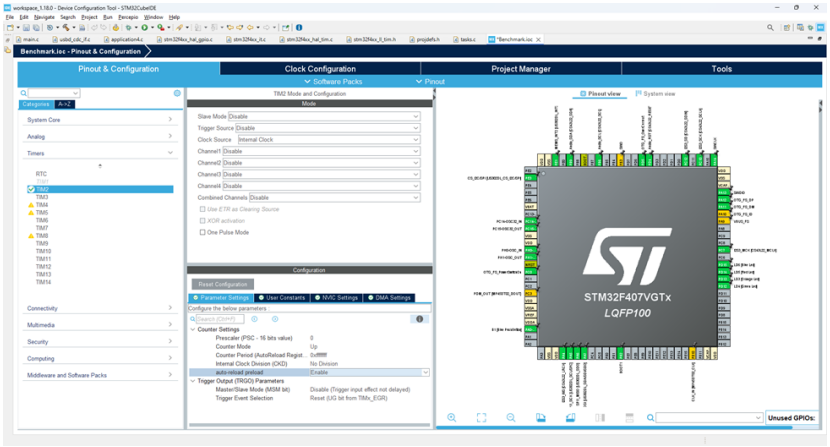

Figure 4.24. TIM2 configuration

- Initialize and start the timers, after the MX_USB_DEVICE_Init(); call

```
/* USER CODE BEGIN 2 */
MX_USB_DEVICE_Init();
HAL_TIM_Base_Init(&htim2);
HAL_TIM_Base_Start(&htim2);
/* USER CODE END 2 *
```

Listing 4.30: TIM2 Init

- Measure time by resetting the timer counter at the beginning of the code block for which you wish to determine the execution time, and read the timer value at the end of the block

```
TIM2->CNT = 0;
/* Reset timer counter */

/* Blocking delay to simulate some workload to be
   measured */
for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul++)
{
        ;
}
/* Get the time value in timer ticks */
measuredTime = TIM2->CNT;
```

Listing 4.31: TIM2 time measurement

- Determine the timer frequency and make the time value conversion from tics to time units. Example: for TIM2 we have 84MHz frequency, thus if we divide measuredTime by 84 we obtain time in ns.
- Express the value to ns and send it to the PC serial communication terminal: vPrintSerial("MeasuredTime in ns %u\n", measuredTime/84UL);

4.11.5. Implementing a test for task switching time

- Create a task and measure its execution time using timers as shown above

```
1  void vTask1(void* pvParameters)
2  {
3
4      volatile uint32_t ul;
5      for (;;)
6      {
7          TIM2->CNT = 0;
8          /* Reset timer counter
9          Blocking delay to simulate some workload */
10         for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul
               ++)
11         {
12                 ;
13                 //taskYIELD();
14         }
15         /* Get the time value in timer ticks */
16         measuredTime = TIM2->CNT;
17         vPrintSerial("MeasuredTime␣in␣ns␣%u\n",
               measuredTime / 84UL);
18     }
19 }
```

Listing 4.32: Task switching time

- Make the first task give the execution context to another task by uncommenting taskYIELD() and create another task that only yields (gives the execution context to the other task without doing anything else).

```
1 void vTask2(void* pvParameters)
2 {
3         for (;;)
4         {
5                 /*taskYIELD(); */
6         }
7 }
```

Listing 4.33: vTask2

- Measure again the time in this case. From the measured time, subtract the first value (the one for the task execution without task YIELD();) and divide the obtained value by 2*mainDELAY_LOOP _COUNT and you will obtain the value for the average task switching time.

4.11.6. Bibliography, Further Reading and Exercises

https://community.st.com/t5/stm32-mcus/how-to-use-the-input-capture-feature/ta-p/704161
https://jacobfilipp.com/DrDobbs/articles/DDJ/1989/8902/8902a/8902a.htm
https://amdls.dorsal.polymtl.ca/files/RTOS%20%20Benchmarking.pdf

Exercises

- Provide implementations for measuring the parameters targeted by Rhealstone benchmark (except deadlock breaking time), on the current system (STM32F407G-Disc1 board running FreeRTOS).
- Measure each parameter using the proposed implementation and timers.
- Make the configurations needed and use Tracealyzer for visualizing the tests and analyzing the parameters, where possible. Compare the values with the ones obtained in the previous exercise, when timers were used.

5. PROJECT PROPOSALS

True skills only develop through practice. In this chapter the reader will find several project proposals which can serve as future directions for developing more practical real-time programming skills.

1. Implement a Morse code interpreter, considering the symbols in Table 5.1, using the user push button from the STM32F407G-Disc1 board for receiving the Morse code signals as input and the serial communication to send the decoded characters. Consider a short press (< 500 ms) to represent a dot (.) in Morse code and a long press (> 500 ms) to represent a dash (-). Also consider a short pause > 1000 ms to indicate a new character and a longer pause to indicate a new word.

Table 5.1. Morse Code

Char	**Morse Code**	**Char**	**Morse Code**	**Char**	**Morse Code**
A	.-	J	.—	S	...
B	-...	K	-.-	T	-
C	-.-.	L	.-..	U	..-
D	-..	M	–	V	...-
E	.	N	-.	W	.–
F	..-.	O	—	X	-..-
G	–.	P	.–.	Y	-.–
H	....	Q	–.-	Z	–..
I	..	R	.-.		
0	——	4	....-	8	—..
1	.—-	5	.....	9	—-.
2	..—	6	-....		
3	...–	7	–...		

2. Implement the previous requirement in the following situations:

   - No operating systems is used

- FreeRTOS is used as an RTOS and no power management is required
- FreeRTOS is used and power management is required

Analyse and compare the performance of the previous implementations using Tracealyzer.

3. Implement an API, based on existing FreeRTOS API, in order to create periodic, sporadic and aperiodic tasks. The API should contain the following functions:

   - BaseType_t xTaskCreatePeriodic(TaskFunction_t pvTaskCode, const char * const pcName, const configSTACK_DEPTH_TYPE uxStackDepth, void *pvParameters, UBaseType_t uxPriority, TaskHandle_t *pxCreatedTask, const TickType_t xTicksPeriod) /*xTicksPeriod represents the period of the task */
   - BaseType_t xTaskCreateSporadic(TaskFunction_t pvTaskCode, const char * const pcName, const configSTACK_DEPTH_TYPE uxStackDepth, void *pvParameters, UBaseType_t uxPriority, TaskHandle_t *pxCreatedTask, const TickType_t xTicksPeriod, TichType_t *pxActivationTimes) /* pxActivationTimes represents a pointer to a list of absolute activation times for the task */
   - BaseType_t xTaskCreateSporadic(TaskFunction_t pvTaskCode, const char * const pcName, const configSTACK_DEPTH_TYPE uxStackDepth, void *pvParameters, UBaseType_t uxPriority, TaskHandle_t *pxCreatedTask, TichType_t xActivationTime) /* xActivationTime represents the absolute activation time */

4. Fixed Execution Non-Preemptive (FENP) [81] is a non-preemptive time-triggered cycling scheduling algorithm used for scheduling real-time perfectly periodical tasks. In an offline step, (i.e. before starting the scheduler), it statically assigns a star

time (an offset) to each task. This start time is a constant offset (phase), calculated from the start of the system. All the next execution instances (jobs) of a FENP task can be further calculated by adding the task period to the start time of the previous instance. The algorithm computes the start times in such a manner that no jobs overlap. Considering that you have the start times computed a priori, implement a FENP execution support in FreeRTOS using software timers.

5. Earliest Deadline First (EDF) is an optimal dynamic priority based preemptive scheduling algorithm. In this algorithm, the task with the earliest absolute deadline is the one with the highest priority. Considering that the relative deadline of each task is equal to the period of the task and following the specifications from [82], implement a EDF scheduling and execution support in FreeRTOS using task lists.

6. By default FreeRTOS does not offer support for sporadic and aperiodic real-time task execution.
   Implement a system of servers (polling/deferrable)[83] to support aperiodic real-time task execution.

7. Mixed Criticality Systems (MCS) [84] are a special case of real-time systems, where tasks have associated besides time parameters, also a criticality level. Considering the Vestal's task model [31], implement a mixed criticality task scheduling and execution support in FreeRTOS.

List of Figures

List of Tables

BIBLIOGRAPHY

[1] Sanjoy Baruah, Marko Bertogna, and Giorgio Buttazzo. *Multiprocessor scheduling for real-time systems*. Springer, 2015.

[2] Arezou Mohammadi and Selim G Akl. Scheduling algorithms for real-time systems. *School of Computing Queens University, Tech. Rep*, 2005.

[3] Maryline Chetto. *Real-time systems scheduling 1: fundamentals*, volume 1. John Wiley & Sons, 2014.

[4] Steiner Wilfried Kopetz Hermann. *Real-time systems: design principles for distributed embedded applications*. Springer Nature, 2022.

[5] Phillip A Laplante et al. *Real-time systems design and analysis*, volume 3. Wiley New York, 2004.

[6] Parameswaran Ramanathan. Overload management in real-time control applications using (m, k)-firm guarantee. *IEEE Transactions on parallel and distributed systems*, 10(6):549–559, 2002.

[7] Jian Li, YeQiong Song, and Françoise Simonot-Lion. Providing real-time applications with graceful degradation of qos and fault tolerance according to (m, k)-firm model. *IEEE Transactions on Industrial Informatics*, 2(2):112–119, 2006.

[8] Jian Lin and Albert MK Cheng. Maximizing guaranteed qos in (m, k)-firm real-time systems. In *12th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA'06)*, pages 402–410. IEEE, 2006.

[9] Xiong Wang, Zhijun Yang, and Hongwei Ding. Application of polling scheduling in mobile edge computing. *Axioms*, 12(7):709, 2023.

[10] Haitao Zhu, Steve Goddard, and Matthew B Dwyer. Response time analysis of hierarchical scheduling: The synchronized deferrable servers approach. In *2011 IEEE 32nd Real-Time Systems Symposium*, pages 239–248. IEEE, 2011.

[11] Chaman Verma, Veronika Stoffová, and Zoltán Illés. Rate-monotonic vs early deadline first scheduling: A review. *Proceeding of Education Technology-Computer science in building better future*, pages 188–193, 2018.

[12] Jiwen Dong and Yang Zhang. A modified rate-monotonic algorithm for scheduling periodic tasks with different importance in embedded system. In *2009 9th International Conference on Electronic Measurement & Instruments*, pages 4–606. IEEE, 2009.

[13] Neil C Audsley, Alan Burns, Mike F Richardson, and Andy J Wellings. Hard real-time scheduling: The deadline-monotonic approach. *IFAC Proceedings Volumes*, 24(2):127–132, 1991.

[14] Parimalam Viswanath. *Online assignment of firm aperiodic tasks using least slack time first (LST) priority*. Texas A&M University-Kingsville, 2005.

[15] Manal A El Sayed, El Sayed M Saad, Rasha F Aly, and Shahira M Habashy. Energy-efficient task partitioning for real-time scheduling on multi-core platforms. *Computers*, 10(1):10, 2021.

[16] Andrea Bastoni, Bjorn B Brandenburg, and James H Anderson. Is semi-partitioned scheduling practical? In *2011 23rd Euromicro Conference on Real-Time Systems*, pages 125–135. IEEE, 2011.

[17] Arvind Easwaran, Insik Shin, and Insup Lee. Optimal virtual cluster-based multiprocessor scheduling. *Real-Time Systems*, 43(1):25–59, 2009.

[18] AM Gruzlikov, NV Kolesov, Yu M Skorodumov, and MV Tolmacheva. Task scheduling in distributed real-time systems. *Journal of Computer and Systems Sciences International*, 56:236–244, 2017.

[19] Koen Langendoen. Medium access control in wireless sensor networks. *Medium access control in wireless networks*, 2:535–560, 2008.

[20] Hermann Kopetz and Günter Grunsteidl. Ttp-a time-triggered protocol for fault-tolerant real-time systems. In *FTCS-23 The twenty-third international symposium on fault-tolerant computing*, pages 524–533. IEEE, 1993.

[21] Hermann Kopetz, Astrit Ademaj, Petr Grillinger, and Klaus Steinhammer. The time-triggered ethernet (tte) design. In *Eighth IEEE International Symposium on Object-Oriented Real-Time Distributed Computing (ISORC'05)*, pages 22–33. IEEE, 2005.

[22] Chung Laung Liu and James W Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM (JACM)*, 20(1):46–61, 1973.

[23] Damien Masson, Laurent George, and Joël Goossens. Dual priority and edf: a closer look. In *Work-in-Progress Session of IEEE Real-Time Systems Symposium (WiP-RTSS)*, 2014.

[24] Martin Stigge, Pontus Ekberg, Nan Guan, and Wang Yi. The digraph real-time task model. In *2011 17th IEEE real-time and embedded technology and applications symposium*, pages 71–80. IEEE, 2011.

[25] Alexander Zuepke, Marc Bommert, and Daniel Lohmann. Autobest: a united autosar-os and arinc 653 kernel. In *21st IEEE real-time and embedded technology and applications symposium*, pages 133–144. IEEE, 2015.

[26] Aloysius Ka-Lau Mok. *Fundamental design problems of distributed systems for the hard-real-time environment*. PhD thesis, Massachusetts Institute of Technology, 1983.

[27] Dorin Maxim and Liliana Cucu-Grosjean. Response time analysis for fixed-priority tasks with multiple probabilistic parameters. In *2013 IEEE 34th Real-Time Systems Symposium*, pages 224–235. IEEE, 2013.

[28] Aloysius K Mok and Deji Chen. A multiframe model for real-time tasks. *IEEE transactions on Software Engineering*, 23(10):635–645, 1997.

[29] Sanjoy K Baruah. A general model for recurring real-time tasks. In *Proceedings 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279)*, pages 114–122. IEEE, 1998.

[30] Hanwoong Jung, Hyunok Oh, and Soonhoi Ha. Multiprocessor scheduling of a multi-mode dataflow graph considering mode transition delay. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 22(2):1–25, 2017.

[31] Steve Vestal. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *28th IEEE international real-time systems symposium (RTSS 2007)*, pages 239–243. IEEE, 2007.

[32] Alan Burns and Robert Ian Davis. Mixed criticality systems-a review:(february 2022). 2022.

[33] Mihai V Micea, V-I Cretu, and Voicu Groza. Maximum predictability in signal interactions with haretick kernel. *IEEE transactions on instrumentation and measurement*, 55(4):1317–1330, 2006.

[34] Eugenia Ana Capota, Cristina Sorina Stangaciu, Mihai Victor Micea, and Daniel-Ioan Curiac. Towards fully jitterless applications: Periodic scheduling in multiprocessor mcss using a table-driven approach. *Applied Sciences*, 10(19):6702, 2020.

[35] Giuseppe Lipari. Earliest deadline first. *Scuola Superiore Sant Anna, Pisa-Italy*, 2005.

[36] Robert I Davis. A review of fixed priority and edf scheduling for hard real-time uniprocessor systems. *ACM SIGBED Review*, 11(1):8–19, 2014.

[37] John Lehoczky, Lui Sha, and Yuqin Ding. The rate monotonic scheduling algorithm: Exact characterization and average case behavior. In *RTSS*, volume 89, pages 166–171, 1989.

[38] Morteza Mohaqeqi, Mitra Nasri, Yang Xu, Anton Cervin, and Karl-Erik Årzén. Optimal harmonic period assignment: complexity results and approximation algorithms. *Real-Time Systems*, 54(4):830–860, 2018.

[39] Enrico Bini, Giorgio Buttazzo, and Giuseppe Buttazzo. A hyperbolic bound for the rate monotonic algorithm. In *Proceedings 13th Euromicro Conference on Real-Time Systems*, pages 59–66. IEEE, 2001.

[40] Olivier Certner. rtprio (2) and posix (. 1b) priorities, and their freebsd implementation: A deep dive (and sweep).

[41] Lui Sha, Ragunathan Rajkumar, and John Lehoczky. Real-time scheduling support in futurebus+. In *[1990] Proceedings 11th Real-Time Systems Symposium*, pages 331–340. IEEE, 1990.

[42] Klein Mark, Ralya Thomas, Pollak Bill, Obenza Ray, and Harbour Michael González. *A practitioner's handbook for real-time analysis: guide to rate monotonic analysis for real-time systems*. Springer Science & Business Media, 2012.

[43] Lui Sha, Ragunathan Rajkumar, and John P Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on computers*, 39(9):1175–1185, 2002.

[44] Kyle Singer, Noah Goldstein, Stefan K Muller, Kunal Agrawal, I-Ting Angelina Lee, and Umut A Acar. Priority scheduling for interactive applications. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures*, pages 465–477, 2020.

[45] Neil C Audsley, Alan Burns, and Andy J Wellings. Deadline monotonic scheduling theory and application. *Control Engineering Practice*, 1(1):71–78, 1993.

[46] Yoshifumi Manabe and Shigemi Aoyagi. A feasibility decision algorithm for rate monotonic and deadline monotonic scheduling. *Real-Time Systems*, 14(2):171–181, 1998.

[47] Paolo Pazzaglia, Alessandro Biondi, and Marco Di Natale. Simple and general methods for fixed-priority schedulability in optimization problems. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1543–1548. IEEE, 2019.

[48] Michael L Dertouzos. Control robotics: The procedural control of physical processes. In *Proceedings IF IP Congress, 1974*, 1974.

[49] Jesse Patterson and Thidapat Chantem. Edf-hv: An energy-efficient semi-partitioned approach for hard real-time systems. In *Proceedings of the 24th International Conference on Real-Time Networks and Systems*, pages 267–276, 2016.

[50] Andreas Stahlhofen and Dieter Zöbel. Linux sched deadline vs. martop-edf. In *2015 IEEE 13th International Conference on Embedded and Ubiquitous Computing*, pages 168–172. IEEE, 2015.

[51] Dario Faggioli, Michael Trimarchi, Fabio Checconi, Marko Bertogna, and Antonio Mancina. An implementation of the earliest deadline first algorithm in linux. In *Proceedings of the 2009 ACM symposium on Applied Computing*, pages 1984–1989, 2009.

[52] Giorgio C Buttazzo. Rate monotonic vs. edf: Judgment day. *Real-Time Systems*, 29(1):5–26, 2005.

[53] Ernesto Massa and George Lima. A bandwidth reservation strategy for multiprocessor real-time scheduling. In *2010 16th IEEE Real-Time and Embedded Technology and Applications Symposium*, pages 175–183. IEEE, 2010.

[54] Alessandro Biondi, Alessandra Melani, Marko Bertogna, and Giorgio Buttazzo. Optimal design for reservation servers under shared resources. In *2014 26th Euromicro Conference on Real-Time Systems*, pages 153–164. IEEE, 2014.

[55] Paolo Valente and Giuseppe Lipari. An upper bound to the lateness of soft real-time tasks scheduled by edf on multiprocessors. In *26th IEEE International Real-Time Systems Symposium (RTSS'05)*, pages 10–pp. IEEE, 2005.

[56] James H Anderson, Vasile Bud, and UC Devi. An edf-based scheduling algorithm for multiprocessor soft real-time systems. In *17th Euromicro Conference on Real-Time Systems (ECRTS'05)*, pages 199–208. IEEE, 2005.

[57] Chia-Hui Wang, Jan-Ming Ho, Ray-I Chang, Shun-Chin Hsu, et al. *A feedback-controlled EDF scheduling algorithm for real-time multimedia transmission.* Institute of Information Science, Academia Sinica, 2001.

[58] Chenyang Lu, John A Stankovic, Gang Tao, and Sang Hyuk Son. Design and evaluation of a feedback control edf scheduling algorithm. In *Proceedings 20th IEEE Real-Time Systems Symposium (Cat. No. 99CB37054)*, pages 56–67. IEEE, 1999.

[59] David Matschulat, Cesar AM Marcon, and Fabiano Hessel. Er-edf: A qos scheduler for real-time embedded systems. In *18th IEEE/IFIP International Workshop on Rapid System Prototyping (RSP'07)*, pages 181–188. IEEE, 2007.

[60] Tao Qian, Frank Mueller, and Yufeng Xin. Hybrid edf packet scheduling for real-time distributed systems. In *2015 27th Euromicro Conference on Real-Time Systems*, pages 37–46. IEEE, 2015.

[61] Sung-Heun Oh and Seung-Min Yang. A modified least-laxity-first scheduling algorithm for real-time tasks. In *Proceedings Fifth International Conference on Real-Time Computing Systems and Applications (Cat. No. 98EX236)*, pages 31–36. IEEE, 1998.

[62] Wei Zhang, Shaohua Teng, Zhaohui Zhu, Xiufen Fu, and Haibin Zhu. An improved least-laxity-first scheduling algorithm of variable time slice for periodic tasks. In *6th IEEE International Conference on Cognitive Informatics*, pages 548–553. IEEE, 2007.

[63] Yorie Nakahira, Niangjun Chen, Lijun Chen, and Steven H Low. Smoothed least-laxity-first algorithm for ev charging. In *Proceedings of the Eighth International Conference on Future Energy Systems*, pages 242–251, 2017.

[64] Xuefeng Piao, Sangchul Han, Heeheon Kim, Minkyu Park, Yookun Cho, and Seongje Cho. Predictability of earliest deadline zero laxity algorithm for multiprocessor real-time systems. In *Ninth IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC'06)*, pages 6–pp. IEEE, 2006.

[65] Andreas Naderlinger. Simulating preemptive scheduling with timing-aware blocks in simulink. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 758–763. IEEE, 2017.

[66] Shamit Bansal, Yecheng Zhao, Haibo Zeng, and Kehua Yang. Optimal implementation of simulink models on multicore architectures with partitioned fixed priority scheduling. In *2018 IEEE Real-Time Systems Symposium (RTSS)*, pages 242–253. IEEE, 2018.

[67] Kaouther Gasmi, Asma Rebaya, Imen Amari, and Salem Hasnaoui. Workflow for multi-core architecture: From matlab/simulink models to hardware mapping/scheduling. In *2016 7th International Conference on Sciences of Electronics, Technologies of Information and Telecommunications (SETIT)*, pages 142–148. IEEE, 2016.

[68] MathWorks. The MathWorks Simulink and Stateflow user manuals. `http://www.mathworks.com`, 2025. Accessed: 2025-07-28.

[69] GPIO Tutorial embetronicx. https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-gpio-tutorial-using-stm32cubeide/. Accessed: 2025.

[70] Push-botton Interfacing Tutorial embetronicx. https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-gpio-tutorial-using-stm32cubeide/Push-button_Interfacing. Accessed: 2025.

[71] STM32F407 Timer Tutorial Using STM32CubeIDE embetronicx. https://embetronicx.com/tutorials/microcontrollers/stm32/stm32f407-timer-tutorial-using-stm32cubeide/. Accessed: 2025.

[72] PWM Generation in STM32F407 Using STM32CubeIDE embetronicx. https://embetronicx.com/tutorials/microcontrollers/stm32/pwm-generation-in-stm32f407-using-stm32cubeide/. Accessed: 2025.

[73] STM32: USB Virtual COM Port (VCP) willfrank. https://willfrank.co.uk/stm32-vcp.html. Accessed: 2025.

[74] Richard Barry. *Mastering the FreeRTOS Real Time Kernel*. 2016.

[75] API References Task Creation freertos. https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate. Accessed: 2025.

[76] API References Task Scheduler freertos. https://www.freertos.org/Documentation/02-Kernel/04-API-references/04-RTOS-kernel-control/03-vTaskStartScheduler. Accessed: 2025.

[77] Emiel Por, Maaike van Kooten, and Vanja Sarkovic. Nyquist–shannon sampling theorem. *Leiden University*, 1(1):1–2, 2019.

[78] Giorgio C Buttazzo and Giorgio C Buttazzo. Fixed-priority servers. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, pages 119–159, 2011.

[79] Percepio tracealyzer. https://percepio.com/tracealyzer/freertostrace/. Accessed: 2025.

[80] JA Heide and Wolfgang A Halang. Performance metrics for real-time systems. In *PEARL 91-Workshop über Realzeitsysteme: 12. Fachtagung des PEARL-Vereins eV unter Mitwirkung von GI und GMA, Boppard, 28./29. November 1991 Proceedings*, pages 121–127. Springer, 1991.

[81] Cristina S Stangaciu, Mihai V Micea, and Vladimir I Cretu. Hard real-time execution environment extension for freertos. In *2014 IEEE International Symposium on Robotic and Sensors Environments (ROSE) Proceedings*, pages 124–129. IEEE, 2014.

[82] Enrico Carraro. Implementation and test of edf and llref schedulers in freertos. 2016.

[83] Robin Kase. Efficient scheduling library for freertos, 2016.

[84] Cristina-Sorina Stângaciu. Task allocation techniques for real-time mixed criticality multicore systems: A survey. *Mastering Time-Innovative Solutions to Complex Scheduling Problems: Innovative Solutions to Complex Scheduling Problems*, page 91, 2025.